

中药材热风烘干过程的热湿分布规律研究

摘要

随着中医药产业的持续发展，药材的质量日益受到重视。干燥是决定中药材成品品质的关键工序，而热风烘干是一种常见的干燥方式，其分为预热平衡阶段与恒温干燥阶段。在烘干过程中，温度与水分浓度在药材内部的时空分布以及干燥过程中的尺寸收缩，直接影响干燥效率和成品品质。本文针对中药材热风烘干过程中的热湿传递问题，建立数学模型，分析药材内部温度场与水分浓度场的演化规律，并确定达到干燥要求所需的时间。

针对问题一，本文将药材简化为固定半径的轴对称圆柱体，根据能量守恒和水分质量守恒分别建立温度传导模型和水分扩散模型，对附件 1 环境数据采用**拉伸指数模型**进行连续拟合，建立**固定半径径向热湿模型**，并采用**有限体积法**和**隐式 BDF 法**求解，得到预热平衡阶段各时刻、各径向位置的温度和水分浓度。计算结果已存入 *result1.xlsx* 中，部分结果展示在表 2、表 3。

针对问题二，考虑预热平衡与恒温干燥两个阶段，先利用 Savitzky-Golay 平滑和连续窗口判别温度、水分浓度的稳定分界点，再用分段拉伸指数模型平滑连接两阶段边界，并将物性参数表示为温度、水分浓度的函数，建立**变物性热湿耦合模型**。通过有限体积法与隐式 BDF 联立求解，得到 0~3h 内的场分布。计算结果已存入 *result2.xlsx* 中，部分结果展示在表 5、表 6。

针对问题三，沿用问题二的变物性热湿耦合模型，以药材全域最大水分浓度首次低于 0.15kg/kg 作为干燥达标条件，采用表面加密网格和事件检测确定临界时刻。计算得到固定半径条件下的干燥时间为 **57.6790h**，完整结果已存入 *result3.xlsx* 中，部分结果展示在表 7。

针对问题四，先对附件 2 的半径时序数据进行物理约束筛选和 **PCHIP 插值**，得到连续半径函数，再引入材料坐标，将随时间变化的物理区域映射到固定区间，建立考虑收缩的**移动边界热湿耦合模型**。结果表明，考虑药材收缩后达到同一干燥要求的时间缩短为 **51.2683h**。

最后，我们对提出的模型进行全面的分析，综合评价了模型的优缺点。

关键词：药材烘干 热湿耦合 有限体积法 隐式 BDF 拉伸指数模型 PCHIP 插值

一、问题重述

1.1 问题背景

随着中医药产业的持续发展，药材的质量管控日益受到重视。干燥是决定中药材成品品质的关键工序，热风烘干作为应用广泛的干燥方式，分为预热平衡与恒温干燥两个阶段，其核心是通过调节烘房内的温度与水分浓度实现药材干燥。

然而实际生产过程中，烘干工艺参数若选取不当，容易出现干燥效率低、能耗高、成品品质不稳定等问题。传统的试验优化模式，存在试验成本高、研发周期长等局限性，因此迫切需要借助数理分析与数值仿真方法揭示药材的干燥规律。为此，本文基于题目所给的工况场景及数据信息，建立数学模型开展科学分析。

1.2 问题要求

以某近似圆柱形的中药材为研究对象，已知药材几何尺寸、初始温度与初始水分浓度，烘房温湿度时序数据以及烘干过程中药材半径变化数据，以及相关物性参数与经验公式。现需据此并结合实际情况建立数学模型，解决以下问题：

问题一：根据题目给定的中药材的初始状态、附件 1 提供的烘房温度和水分浓度变化数据，以及附录 2 给出的相关参数，建立预热平衡阶段药材温度和水分浓度随时间及空间位置变化的数学模型。计算 100、300、600、900、1200、1500、1800s 时，距离药材中心 0、0.5、1、1.5、2cm 处的温度和水分浓度，并将每隔 1s、每隔 0.1cm 的完整结果保存至 *result1.xlsx* 中。

问题二：考虑预热平衡阶段与恒温干燥阶段的参数差异，根据附录 3 给出的经验公式，建立描述整个烘干过程中药材温度和水分浓度变化规律的数学模型。给出 0~3.0h 内每隔 0.5h，以及距离药材中心 0、0.5、1、1.5、2cm 处的计算结果，并将每隔 1s、每隔 0.1cm 的完整结果保存至 *result2.xlsx* 中。

问题三：以药材各处水分浓度均低于 0.15kg/kg 为烘干达标条件，确定药材达到该要求所需的烘干时间。按照每隔 6h、距离药材中心每隔 0.5cm 的方式给出烘干过程中的水分浓度，并将每隔 60s、每隔 0.1cm 的完整结果保存至 *result3.xlsx* 中。

问题四：考虑药材在烘干过程中因水分流失而产生的尺寸变化。根据附件 2 提供的半径变化数据和附录 4 给出的经验公式，确定药材达到烘干要求所需的时间。按照每隔 6h、从药材中心起每隔 0.5cm 直至药材表面的方式给出水分浓度，并将每隔 60s、每隔 0.1cm 的完整结果保存至 *result4.xlsx* 中。

二、问题分析

2.1 问题一：

问题一要求研究固定半径药材在预热平衡阶段的温度与水分浓度变化的数学模型。药材为圆柱形，半径固定，长度远大于半径，可忽略轴向效应，简化为沿半径方向的一维问题。附件 1 以 60s 为间隔给出一段时间内的烘房环境记录，而模型求解需要 0~1800s 内连续可求的边界输入。因此首先需要先对烘房温度和水分浓度数据进行连续化拟合。

相关研究揭示了干燥过程的一般物理规律^[2]。对题给烘干过程，根据能量守恒和水分质量守恒分别建立温度和水分浓度的径向守恒方程，方程为圆柱坐标下的非稳态偏微分方程，没有解析解，故采用有限体积法^[7]和隐式 BDF^[3]进行求解。由于温度场和水分浓度场的物性参数与边界条件相对独立，可分别求解，再通过表面边界相互关联。

2.2 问题二：

问题二要求建立整个烘干过程药材温度与水分浓度变化的数学模型，并给出前 3h 的结果。由于整个烘干过程包含预热平衡和恒温干燥两个不同阶段，因此需要分别识别烘房温度和水分浓度的稳定分界点，并分别连续化拟合。为保证平滑，两阶段间需通过平滑过渡连接。

其次，采用附录 3 给出的经验公式建立变物性热湿耦合模型，其中 ρ 、 c_p 、 k 随局部水分浓度变化， D 同时随局部温度和水分浓度变化。此时温度场和水分浓度场相互影响，不能再分别求解，应在使用有限体积法后用隐式 BDF 联立求解。

2.3 问题三：

问题三在问题二模型的基础上，将计算时间延长至药材完全达到干燥要求的时刻。由于附件 1 的记录时间有限，稳定阶段环境边界的外推会影响长期计算，因此需要明确尾段边界的构造方式，此处仍用问题二的分段拟合方法构造。

题目要求药材各处的水分浓度均低于 0.15 kg/kg，因此以全域最大水分浓度的首次低于阈值时刻作为干燥终点，求得对应时间。

2.4 问题四：

相关研究揭示了半径收缩下干燥过程的物理规律^[4]。考虑药材在烘干过程中的半径收缩并重新建模后，采用附录 4 给出的经验公式。附件 2 中的半径数据是离散记录，因此需要和温度和水分浓度数据一样先进行插值，并比较不同方法的拟合误差、单调性和越界情况，得到符合物理变化规律的半径变化函数。

然后，在假设长度不变且药材径向均匀收缩的条件下，引入材料坐标 ξ ，将移动区域变换为固定区域进行求解。半径变化会同时影响内部传递项和表面边界条件，数值求解后还需将材料坐标结果转换为实际距离，并用全域最大水分浓度确定收缩状态下的干燥终点。

文章总体思路如图 1 所示：

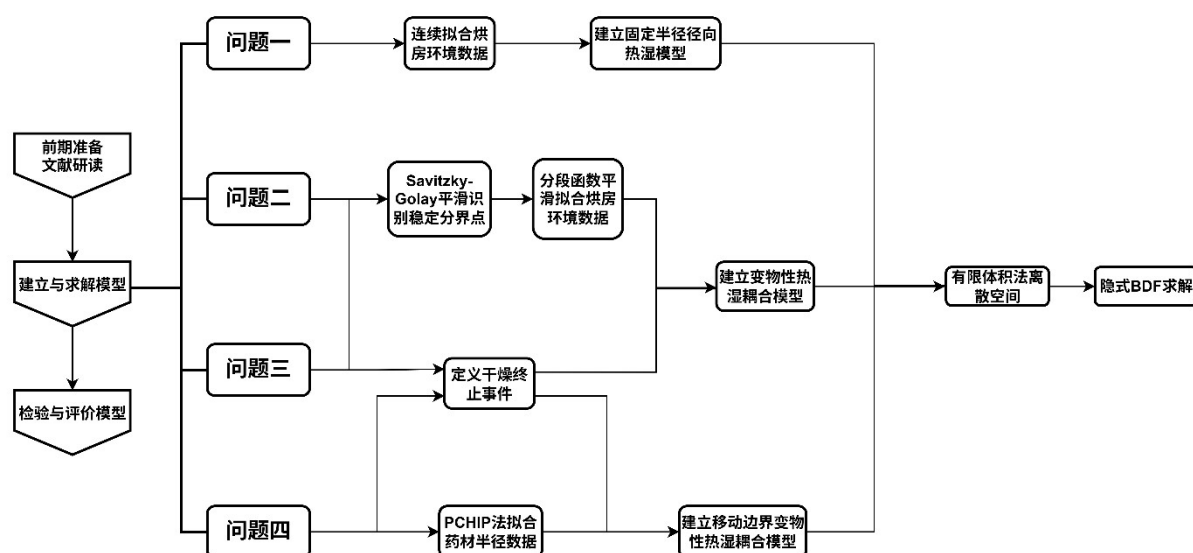


图 1 总体思路图

三、模型假设

- (1) 假设药材为均匀、各向同性的近似圆柱体，忽略轴向和周向差异，仅考虑径向传热传质。
- (2) 假设传热遵循傅里叶定律，水分迁移遵循等效扩散定律，忽略蒸发潜热、湿分迁移显热及内部热源等影响。
- (3) 假设药材与烘房只通过对流换热和对流传质交换热量与水分，表面满足 Robin 边界条件，环境数据用连续函数拟合。
- (4) 假设问题三、问题四中进入恒温干燥阶段后烘房温度、水分浓度一直趋于稳定。
- (5) 假设问题四中药材长度不变且沿径向均匀收缩，忽略开裂、翘曲等局部变形。

四、符号说明

符号	含义	单位
t	时间	s
R	药材半径	m
L	药材长度	m
ρ	药材密度	kg/m^3
c_p	药材比热容	$J/(kg \cdot K)$
k	热传导系数	$W/(m \cdot K)$
h	对流换热系数	$W/(m^2 \cdot K)$
h_m	对流传质系数	m/s
D	水分扩散系数	m^2/s
m_w	药材所含水分质量	kg
m_d	药材干物质质量	kg
$T(r, t)$	t 时刻距离药材中心距离 r 处的温度	$^{\circ}C$
$C(r, t)$	t 时刻距离药材中心距离 r 处的水分浓度	kg/kg
$\Theta(\xi, t)$	t 时刻距离药材中心距离 ξR 处的温度	$^{\circ}C$
$U(\xi, t)$	t 时刻距离药材中心距离 ξR 处的水分浓度	kg/kg
$T_a(t)$	t 时刻的烘房温度	$^{\circ}C$
$C_a(t)$	t 时刻的烘房水分浓度	kg/kg

五、模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型准备

在问题一中，要求研究固定半径药材在预热平衡阶段的温度与水分浓度变化的数学模型，该阶段药材内部温度 $T(r, t)$ 和水分浓度 $C(r, t)$ 的变化，直接受烘房环境温度 $T_a(t)$ 和水分浓度 $C_a(t)$ 控制，二者若作为模型的边界输入，必须对任意时间 $t \in [0, 1800]$ 连续可求。

附件 1 仅给出间隔为 60s 的环境记录，记录范围为 0~14400s，可绘制为原始环境边界散点图，如图 2。

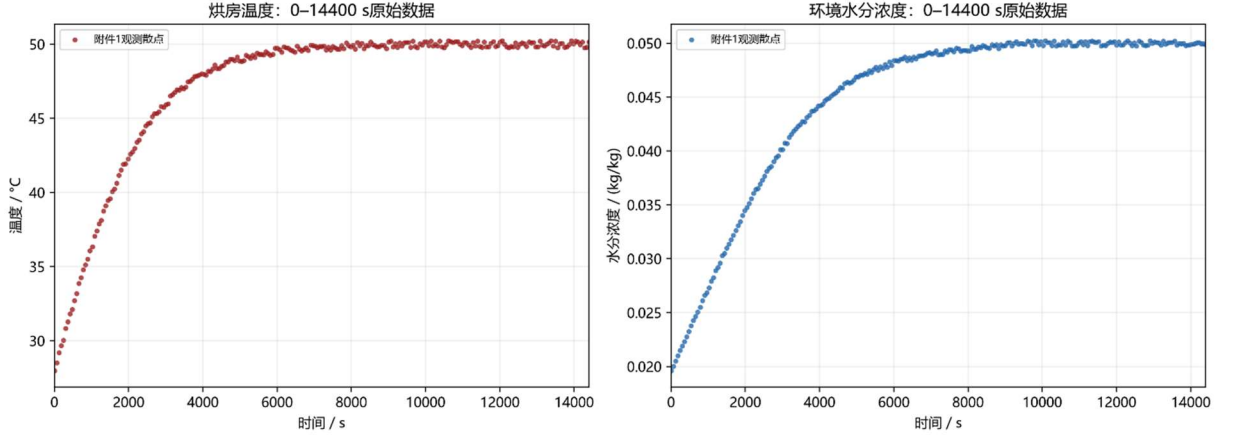


图 2 原始环境边界散点图

可以看到温度和环境水分浓度均在早期快速上升，随后逐渐变慢并趋于稳定。若直接线性插值，边界输入在节点处不可导，且可能引入折点，影响数值求解的稳定性和精度。因此，需要先根据附件 1 的数据，建立连续、光滑的环境边界函数。

本文先比较六类候选模型：分段线性、PCHIP、Akima、自然三次样条、平滑样条和初值固定的拉伸指数模型。六者的求解机制并不相同：前四者是由观测节点直接确定的插值方法，不需要非线性参数优化，平滑样条通过残差平方和和曲率惩罚确定平滑函数，属于正则化最小二乘，拉伸指数模型需要对幅值、时间尺度和形状参数进行非线性最小二乘估计。

对候选方法采用时间阻塞交叉验证，在不随机打乱记录的前提下，从 0~1800s 的 31 个观测点中每隔 5 个点留出 1 个点，得到 25 个训练点和 6 个验证点。随后在 6 个验证点上计算得到均方根误差，作为阻塞验证 RMSE 的结果。

具体地，若验证样本为 (t_i, y_i) ，预测值为 $\hat{y}$ 则

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

其中 n 是验证样本数。为比较曲线的局部弯折程度，另在验证区间内取 721 个等间距点，定义二阶差分粗糙度

$$\mathcal{R}(\hat{y}) = \frac{1}{N_d - 2} \sum_{j=1}^{N_d-2} \left| \frac{\hat{y}_{j+1} - 2\hat{y}_j + \hat{y}_{j-1}}{\Delta t^2} \right| \quad N_d = 721$$

该指标是平均绝对二阶差分，数值越小表示曲线越平滑。温度和水分浓度的粗糙度因量纲不同，只在同一序列内比较。候选方法比较和拉伸指数参数估计均只使用 0~1800s 数据，结果如表 1。

表 1 六类候选边界拟合方法的时间阻塞交叉验证比较表

方法	温度 RMSE/ $^{\circ}C$	水分浓度 RMSE	温度粗糙度/ $(^{\circ}C \cdot s^{-2})$	水分浓度粗糙度/ $((kg/kg) \cdot s^{-2})$
分段线性	0.133411	5.01×10^{-5}	3.62×10^{-5}	3.48×10^{-8}
PCHIP	0.161469	4.57×10^{-5}	5.10×10^{-5}	5.06×10^{-8}
Akima	0.147783	5.22×10^{-5}	5.17×10^{-5}	5.05×10^{-8}
自然三次样条	0.186352	4.13×10^{-5}	5.16×10^{-5}	5.07×10^{-8}
平滑样条	0.078165	6.00×10^{-5}	2.12×10^{-6}	6.93×10^{-10}
拉伸指数模型	0.057243	5.54×10^{-5}	2.20×10^{-6}	7.30×10^{-10}

温度序列中拉伸指数模型的 RMSE 最低 ($0.057243^{\circ}C$)。水分浓度序列中自然三次样条的 RMSE 为 4.13×10^{-5} ，略低于拉伸指数模型的 5.54×10^{-5} ，但其粗糙度约为拉伸指数模型的 69.5 倍，平滑样条的粗糙度虽更小，误差却略高于拉伸指数模型。综合 RMSE、粗糙度和平滑性，以及希望用同一函数族表示两类环境边界，本文选择初值固定的拉伸指数模型。

对任意环境变量 $y(t)$ ，固定 $t=0$ 的观测初值 y_0 ，拉伸指数模型标准式如下

$$y(t) = y_0 + A \left[1 - \exp \left(- \left(\frac{t}{\tau} \right)^p \right) \right] \quad A > 0, \tau > 0, p > 0 \quad (1)$$

其中 t 和时间尺度 τ 用秒计， A 为幅值， p 为无量纲形状参数。通过非线性最小二乘法估计后，烘房温度边界和水分浓度边界分别为

$$T_a(t) = 28 + 27.3279502 \left[1 - \exp \left(- \left(\frac{t}{2617.1875} \right)^{1.0429163} \right) \right]$$

$$C_a(t) = 0.01963 + 0.06247822 \left[1 - \exp \left(- \left(\frac{t}{6927.1507} \right)^{1.0519591} \right) \right]$$

最终拟合效果如图 3。

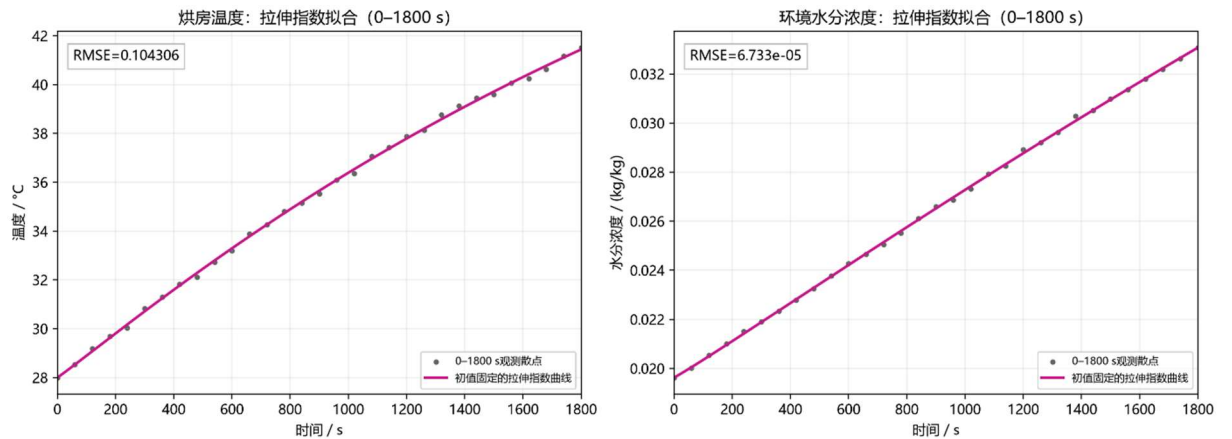


图 3 拉伸指数模型拟合效果图

5.1.2 模型建立

Step 1 建立径向热传导方程

在问题一中，要求研究固定半径药材在预热平衡阶段的温度与水分浓度分布。根据傅里叶定律，沿径向向外的导热通量为

$$q_r = -k \frac{\partial T}{\partial r}$$

取半径为 r 、厚度为 dr 、长度为 L 的微小圆环控制体。由能量守恒，控制体内能的增加率等于导热净流入量：

$$\rho c_p (2\pi r L dr) \frac{\partial T}{\partial t} = - \frac{\partial}{\partial r} (2\pi r L q_r) dr$$

代入傅里叶定律并约去公共因子，得到圆柱径向热传导方程

$$\rho c_p \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k \frac{\partial T}{\partial r} \right) \quad 0 < r < R$$

问题一中 ρ 、 c_p 、 k 均为常数。令热扩散率 $\alpha = k/(\rho c_p)$ ，上式也可写成

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r} \right) \quad 0 < r < R \quad (2)$$

Step 2 建立水分浓度扩散方程

对于水分迁移，设单位体积干物质质量为 ρ_d ，根据菲克定律定义径向水分质量通量

$$j_w = -\rho_d D(C) \frac{\partial C}{\partial r}$$

对同一圆环应用水分质量守恒，并假设 ρ_d 在空间上均匀，可得

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right] \quad 0 < r < R \quad (3)$$

其中问题一的有效水分扩散系数为

$$D(C) = 7 \times 10^{-9} \exp\left(-\frac{0.89}{C}\right)$$

Step 3 初始条件与边界条件

药材初始状态均匀，初始条件为

$$T(r, 0) = T_0 \quad C(r, 0) = C_0 \quad (4)$$

圆柱中心满足轴对称条件

$$\left. \frac{\partial T}{\partial r} \right|_{r=0} = 0 \quad \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0 \quad (5)$$

表面采用有限速率的对流换热和有效传质边界条件

$$\begin{aligned} -k \left. \frac{\partial T}{\partial r} \right|_{r=R} &= h [T(R, t) - T_a(t)] \\ -D(C(R, t)) \left. \frac{\partial C}{\partial r} \right|_{r=R} &= h_m [C(R, t) - C_a(t)] \end{aligned} \quad (6)$$

当空气温度高于药材表面温度时，热量由环境传入药材，当药材表面水分浓度高于环境参考值时，水分由药材向外迁移。问题一中温度方程的系数不依赖水分浓度，水分扩散系数也不依赖温度，因此两个场可分别求解。

Step 4 模型汇总

综上，建立固定半径径向热湿模型：

$$\left\{ \begin{array}{l} \frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r} \right) \\ \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C) \frac{\partial C}{\partial r} \right] \\ D(C) = 7 \times 10^{-9} \exp \left(-\frac{0.89}{C} \right) \\ T(r, 0) = T_0 \\ C(r, 0) = C_0 \\ \left. \frac{\partial T}{\partial r} \right|_{r=0} = 0 \\ \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0 \\ -k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h[T(R, t) - T_a(t)] \\ -D(C(R, t)) \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m[C(R, t) - C_a(t)] \end{array} \right. \quad 0 < r < R$$

5.1.3 模型求解

Step 1 有限体积法处理偏微分方程

采用径向有限体积法，将固定半径区间 $[0, R]$ 均分为 N 个小区间，节点为

$$r_i = i \Delta r \quad \Delta r = \frac{R}{N} \quad i = 0, 1, \dots, N$$

以节点 r_i 为中心建立环状控制体，其左右边界定义为

$$a_i = \max \left(0, r_i - \frac{\Delta r}{2} \right) \quad b_i = \min \left(R, r_i + \frac{\Delta r}{2} \right)$$

圆柱体积元为 $2\pi L r \, dr$ 。约去控制方程两侧共有的 $2\pi L$ 后，第 i 个控制体的几何权重为

$$w_i = \int_{a_i}^{b_i} r \, dr = \frac{b_i^2 - a_i^2}{2}$$

为统一温度与水分浓度方程，令 $u = T$ 、 $a = \alpha$ 表示温度问题，令 $u = C$ 、 $a = D(C)$ 表示水分浓度问题，则两者均可写为

$$u_t = \frac{1}{r} \frac{\partial}{\partial r} (r a u_r)$$

将上式乘以 r ，并在控制体 $[a_i, b_i]$ 上积分，得到

$$w_i \frac{du_i}{dt} = F_{i+1/2} - F_{i-1/2}$$

中心对称条件对应 $F_{-1/2} = 0$ 。温度和水分浓度的表面交换条件分别写成

$$F_{N+1/2}^T = R \frac{h}{\rho c_p} [T_a(t) - T_N] \quad F_{N+1/2}^C = R h_m [C_a(t) - C_N]$$

由此得到统一的半离散方程

$$\frac{du_i}{dt} = \frac{F_{i+1/2} - F_{i-1/2}}{w_i}$$

其中, $F_{i+1/2}$ 为节点 i 与 $i+1$ 之间的界面通量变量。界面位置及其通量分别近似为

$$r_{i+1/2} = \frac{r_i + r_{i+1}}{2} \quad F_{i+1/2} = r_{i+1/2} a_{i+1/2} \frac{u_{i+1} - u_i}{\Delta r}$$

对于温度方程, $a_{i+1/2} = \alpha$ 。对于水分浓度方程, $a = D(C)$, 在低水浓度分区域呈指数级下降, 节点处扩散系数的算术平均仅是对界面系数的近似, 可能在水分浓度梯度较大时产生偏差。而 Kirchhoff 积分平均能够更准确地反映扩散系数在节点间的连续变化, 并保持有限体积元的通量守恒性^[1]。故本文采用 Kirchhoff 积分平均。

在节点 r_i 与 r_{i+1} 之间, 令

$$\Delta r = r_{i+1} - r_i \quad \Delta C = C_{i+1} - C_i$$

并在该节点段上采用线性水分浓度路径

$$C(s) = C_i + s \Delta C \quad 0 \leq s \leq 1$$

定义 Kirchhoff 变换

$$\Psi(C) = \int_C D(\eta) d\eta$$

则 Kirchhoff 界面平均扩散系数

$$\begin{aligned} D_{i+1/2}^K &= \frac{\Psi(C_{i+1}) - \Psi(C_i)}{C_{i+1} - C_i} \\ &= \frac{1}{C_{i+1} - C_i} \int_{C_i}^{C_{i+1}} D(\eta) d\eta \end{aligned}$$

将变量代换 $\eta = C_i + s \Delta C$ 、 $d\eta = \Delta C ds$ 带入上式可以得到

$$D_{i+1/2}^K = \int_0^1 D(C_i + s(C_{i+1} - C_i)) ds \quad (7)$$

对于该积分式采用 8 点 Gauss-Legendre 求积: 若 s_j 和 ω_j 分别为变换到 $[0, 1]$ 后的求积节点和权重, 则

$$D_{i+1/2}^K \approx \sum_{j=1}^8 \omega_j D(C_i + s_j(C_{i+1} - C_i))$$

当 $C_{i+1} = C_i$ 时, 不使用差商形式, 而由积分定义直接得到连续极限

$$D_{i+1/2}^K = D(C_i)$$

从而避免相邻水分浓度很接近时的分母除零问题。

为构造水分浓度方程的解析 Jacobian 供后文使用, 对积分式(7)在节点水分浓度处求偏导。沿积分路径

$$C(s) = C_i + s(C_{i+1} - C_i)$$

有

$$\frac{\partial C(s)}{\partial C_i} = 1 - s \quad \frac{\partial C(s)}{\partial C_{i+1}} = s$$

记 $D_C = \frac{dD}{dC}$, 根据链式法则, 得到

$$\frac{\partial D_{i+1/2}^K}{\partial C_i} = \int_0^1 D_C (C_i + s \Delta C) (1-s) ds, \quad \frac{\partial D_{i+1/2}^K}{\partial C_{i+1}} = \int_0^1 D_C (C_i + s \Delta C) s ds \quad (8)$$

数值求积使用与积分式(7)相同 8 个 Gauss-Legendre 节点计算。

Step 2 常微分方程组求解

空间离散后, 将全部节点值组成状态向量, 将全部节点温度和水分浓度组成状态向量

$$\mathbf{y}(t) = \begin{bmatrix} \mathbf{T}(t) \\ \mathbf{C}(t) \end{bmatrix}$$

得到常微分方程组

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y})$$

本文使用自适应隐式 BDF 方法求解^[6]。以一阶 BDF 为例,

$$\frac{\mathbf{y}^{n+1} - \mathbf{y}^n}{\Delta t} = \mathbf{f}(t_{n+1}, \mathbf{y}^{n+1})$$

右端含有新时刻状态, 每个时间步长都需要隐式求解。计算中提供稀疏解析 Jacobian 矩阵 $\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{y}}$ ^[3], 以提高迭代效率。具体到第 i 个控制体, 有

$$f_i(\mathbf{y}) = \frac{F_{i+1/2}(\mathbf{y}) - F_{i-1/2}(\mathbf{y})}{w_i}$$

则右端 Jacobian 矩阵的第 i 行由下式得到:

$$\frac{\partial f_i}{\partial y_j} = \frac{1}{w_i} \left(\frac{\partial F_{i+1/2}}{\partial y_j} - \frac{\partial F_{i-1/2}}{\partial y_j} \right)$$

问题一的温度场和水分浓度场彼此解耦, 因此两个场分别形成一个稀疏三对角矩阵, 而不出现热湿耦合分块矩阵。下面直接对界面通量 $F_{i+1/2}$ 求导, 并记 $\Delta u = u_{i+1} - u_i$ 。令

$$a_i^L = \frac{\partial a_{i+1/2}}{\partial u_i} \quad a_i^R = \frac{\partial a_{i+1/2}}{\partial u_{i+1}}$$

则

$$\frac{\partial F_{i+1/2}}{\partial u_i} = \frac{r_{i+1/2}}{\Delta r} (a_i^L \Delta u - a_{i+1/2}) \quad \frac{\partial F_{i+1/2}}{\partial u_{i+1}} = \frac{r_{i+1/2}}{\Delta r} (a_i^R \Delta u + a_{i+1/2})$$

温度方程中 $a_{i+1/2} = \alpha$ 为常数, 故 $a_i^L = a_i^R = 0$ 。水分浓度方程中, $a_{i+1/2} = D_{i+1/2}^K$,

$a_i^L = \frac{\partial D_{i+1/2}^K}{\partial C_i}$ 与 $a_i^R = \frac{\partial D_{i+1/2}^K}{\partial C_{i+1}}$ 由式(8)的 Kirchhoff 平均的端点导数公式给出。

若将表面处的通量统一写成 $F_{N+1/2} = R\beta(u_a - u_N)$, 对温度场, $\beta = h/(\rho c_p)$ 。对水分浓度场, $\beta = h_m$, 则它对最外节点右端的边界贡献为

$$\left. \frac{\partial f_N}{\partial u_N} \right|_{r=R} = - \frac{R\beta}{w_N}$$

这就是三对角矩阵最外侧对角元的表面边界修正。

问题一在 $0 \leq t \leq 1800$ s 内积分, 最终采用 $N = 6400$ 个径向区间, 即 6401 个节点, 空间步长为 3.125×10^{-6} m。BDF 相对容差和绝对容差分别取 2×10^{-10} 和 2×10^{-12} , 最大内部步长为 5s。

5.1.4 求解结果与分析

100、300、600、900、1200、1500、1800 s，到药材中心距离 0、0.5、1、1.5、2 cm 处的结果如表 2、表 3，1800s 内每隔 1s、到药材中心距离每隔 0.1cm 的完整结果已保存到文件 *result1.xlsx* 中。

表 2 30 分钟内药材的温度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	28.0001	28.0003	28.0039	28.0318	28.1698
300	28.0386	28.0604	28.1466	28.3645	28.8353
600	28.4463	28.5292	28.7997	29.3204	30.1872
900	29.3232	29.4570	29.8737	30.6169	31.7512
1200	30.5480	30.7160	31.2299	32.1166	33.4151
1500	31.9960	32.1846	32.7557	33.7237	35.1079
1800	33.5689	33.7676	34.3658	35.3685	36.7806

表 3 30 分钟药材的水分浓度

时间/s	到药材中心的距离/cm				
	0	0.5	1	1.5	2
100	2.5500	2.5500	2.5500	2.5500	2.2470
300	2.5500	2.5500	2.5500	2.5492	2.0508
600	2.5500	2.5500	2.5500	2.5353	1.8770
900	2.5500	2.5500	2.5497	2.5045	1.7547
1200	2.5500	2.5500	2.5482	2.4646	1.6586
1500	2.5500	2.5499	2.5445	2.4206	1.5787
1800	2.5500	2.5497	2.5383	2.3755	1.5102

由表可见 1800 s 时药材中心温度为 33.5689°C 、表面温度为 36.7806°C ，中心水分浓度为 2.5500kg/kg 、表面水分浓度为 1.5102kg/kg 。药材表面先升温失水，中心却仍接近初始状态。

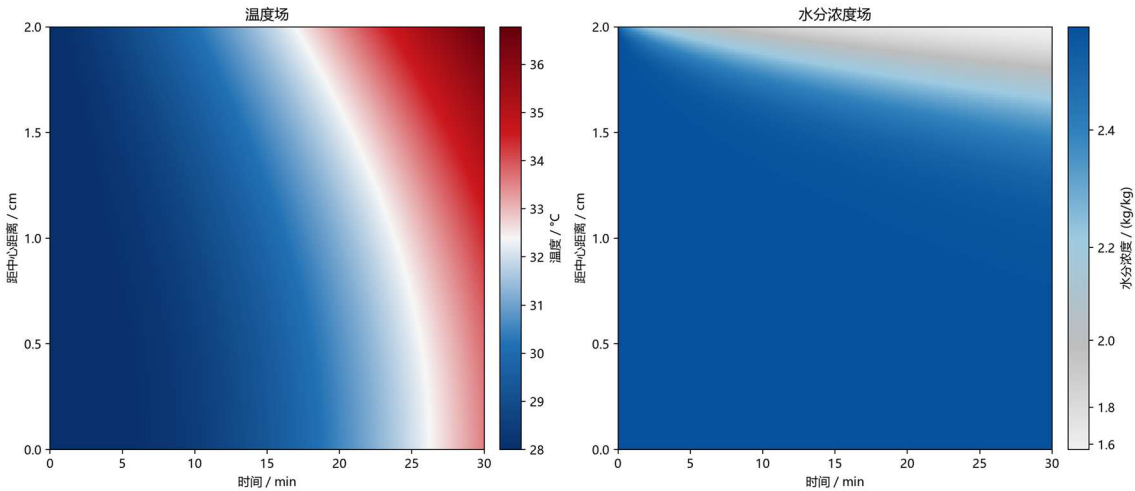


图 4 前 30 分钟药材温度、水分浓度场变化图

将 *result1.xlsx* 结果可视化得到图 4。图中左右两个面板分别显示温度场和水分浓度场随横轴时间、纵轴距中心距离的变化情况。可直接看到温度由表面向内部传播，以及水分浓度首先在表面降低并逐步向中心扩展。

5.2 问题二模型的建立与求解

5.2.1 模型准备

第二问要求建立整个烘干过程药材温度与水分浓度变化的数学模型。整个烘干过程主要包括预热平衡和恒温干燥两个阶段。图 2 的原始散点也表明，烘房温度和环境水分浓度都经历了从快速增长到逐渐平稳的过程。因此需要分别识别烘房温度和水分浓度的稳定分界点，并分别连续化拟合。为保证平滑，两阶段间需通过平滑过渡连接。

此处沿用问题一定义的 RMSE、粗糙度和六类候选方法，并通过比较最终仍选用拉伸指数函数，具体过程不再赘述。稳定分界点探索使用附件一中完整的 0~14400s 记录，然后稳定分界点前后的数据分别用于求解预热平衡阶段的环境边界函数和恒温干燥阶段的均值估计。

Step 1 稳定分界点的识别

本步骤的目标是从附件一的全量数据中确定预热平衡和恒温干燥两阶段之间的稳定分界点。令 $y \in \{T_a, C_a\}$ ，其中 $T_a(t)$ 为环境温度， $C_a(t)$ 为环境水分浓度。两者的稳定时间应分别由各自的观测序列确定。

为区分真实趋势与短时波动，首先使用 11 点二阶 Savitzky-Golay 滤波拟合法对原始序列作平滑处理，记平滑结果为 $\tilde{y}_i$ 。然后从 $t = 1800$ s 开始采样候选窗口。取采样步长 $W = 1800$ s，对应 $n_W = \frac{W}{\Delta t} + 1 = 31$ 个观测点。若窗口起点为 $s = t_k$ ，记其观测下标集合为 $I_k = \{k, k+1, \dots, k+n_W-1\}$ ，则短时波动可被描述为窗口极差

$$R_y(s) = \max_{i \in I_k} \tilde{y}_i - \min_{i \in I_k} \tilde{y}_i$$

同时，用窗口内平滑序列的最小二乘直线斜率 $b_y(s)$ 描述边界量的残余趋势。令 $\bar{t}_k$ 和 $\bar{y}_k$ 分别为窗口内时间和平滑值的均值，则

$$b_y(s) = \frac{\sum_{i \in I_k} (t_i - \bar{t}_k) (\tilde{y}_i - \bar{y}_k)}{\sum_{i \in I_k} (t_i - \bar{t}_k)^2}$$

$R_y(s)$ 控制局部波动幅度， $|b_y(s)|$ 控制窗口内的平均变化速度。二者分别对应变化幅度大小和趋势停止与否。

由于温度和水分浓度的量纲不同，为使阈值随不同量纲的噪声尺度自动调整，取末段样本 $I_{\text{tail}} = \{i: t_i \geq 0.75t_{\text{max}}\}$ ，对残差 $e_i = y_i - \tilde{y}_i$ 作稳健估计：

$$m_e = \text{median}_{i \in I_{\text{tail}}} (e_i)$$

$$MAD_y = \text{median}_{i \in I_{\text{tail}}} |e_i - m_e|$$

$$\hat{\sigma}_y = \frac{1}{\Phi^{-1}(0.75)} MAD_y = 1.4826 MAD_y$$

其中 $\frac{1}{\Phi^{-1}(0.75)}$ 是在正态噪声假设下将 MAD 校准为标准差的无量纲系数。

再以全序列相邻记录差的中位数表示记录分辨率尺度 d_y ，并设置极差阈值 $\varepsilon_{R,y}$ 和斜率阈值 $\varepsilon_{b,y}$ ，具体为

$$d_y = \text{median}_{i=1,\dots,n} |y_i - y_{i-1}|$$

$$\varepsilon_{R,y} = \max(\hat{2\sigma}_y, 4d_y), \quad \varepsilon_{b,y} = \frac{\varepsilon_{R,y}}{W}$$

于是，候选窗口的稳定标志定义为

$$\mathcal{S}_y(s) = \begin{cases} 1, & R_y(s) \leq \varepsilon_{R,y} \text{ 且 } |b_y(s)| \leq \varepsilon_{b,y}, \\ 0, & \text{其他情况.} \end{cases}$$

为避免把一次偶然的平静当作稳定阶段，要求稳定标志在时间上连续通过三个窗口。对离散扫描而言，稳定分界点取为

$$t_{s,y} = \min \{t_k : t_k \geq 1800 \text{ s}, \mathcal{S}_y(t_k) = \mathcal{S}_y(t_{k+n_W}) = \mathcal{S}_y(t_{k+2n_W}) = 1\}$$

由于窗口两端的观测点均计入，单个窗口包含 31 个点，程序中连续窗口起点相隔 $n_W \Delta t = 1860 \text{ s}$ 。确定 $t_{s,y}$ 后，再用分界点之后的原始记录计算稳定尾段均值作为后续分段边界的稳定水平，即

$$y^* = \frac{1}{n_*} \sum_{t_i \geq t_{s,y}} y_i, \quad n_* = \#\{i : t_i \geq t_{s,y}\}$$

$$y^* = \begin{cases} T_a(t_i) & y = T_a \\ C_a(t_i) & y = C_a \end{cases}$$

即分界点作为外部边界的趋势项和波动项同时低于相应尺度后，连续保持三个窗口的最早时刻。上述 $t_{s,y}$ 与 y^* 将分别作为下一步分段边界函数的转接点和稳定尾段值。识别结果如表 4。

表 4 烘房温度与水分浓度稳定分界点判定表

序列	σ_y	$\varepsilon_{R,y}$	$\varepsilon_{b,y}$	连续窗口 起点/s	稳定分 界点 $t_{s,y}/s$	尾段均值
烘房 温度	$0.1809^\circ C$	$0.720000^\circ C$	$4.00 \times 10^{-4}^\circ C/s$	5280, 7140, 9000	5280	$49.8858^\circ C$
烘房 水分 浓度	$1.7007 \times 10^{-4} \text{ kg/kg}$	$8.60 \times 10^{-4} \text{ kg/kg}$	$4.78 \times 10^{-7} (\text{kg/kg})/s$	6780, 8640, 10500	6780	0.0498 kg/kg

由此，烘房温度和水分浓度的稳定分界点相差 1500 s，不能用一个共同分界点或两个序列的均值代替。分界点探索的具体过程见图 5，散点是原始记录，黑线是平滑曲线，浅色窗口表示连续通过判据的候选窗口，红色虚线是最终选定的起点。

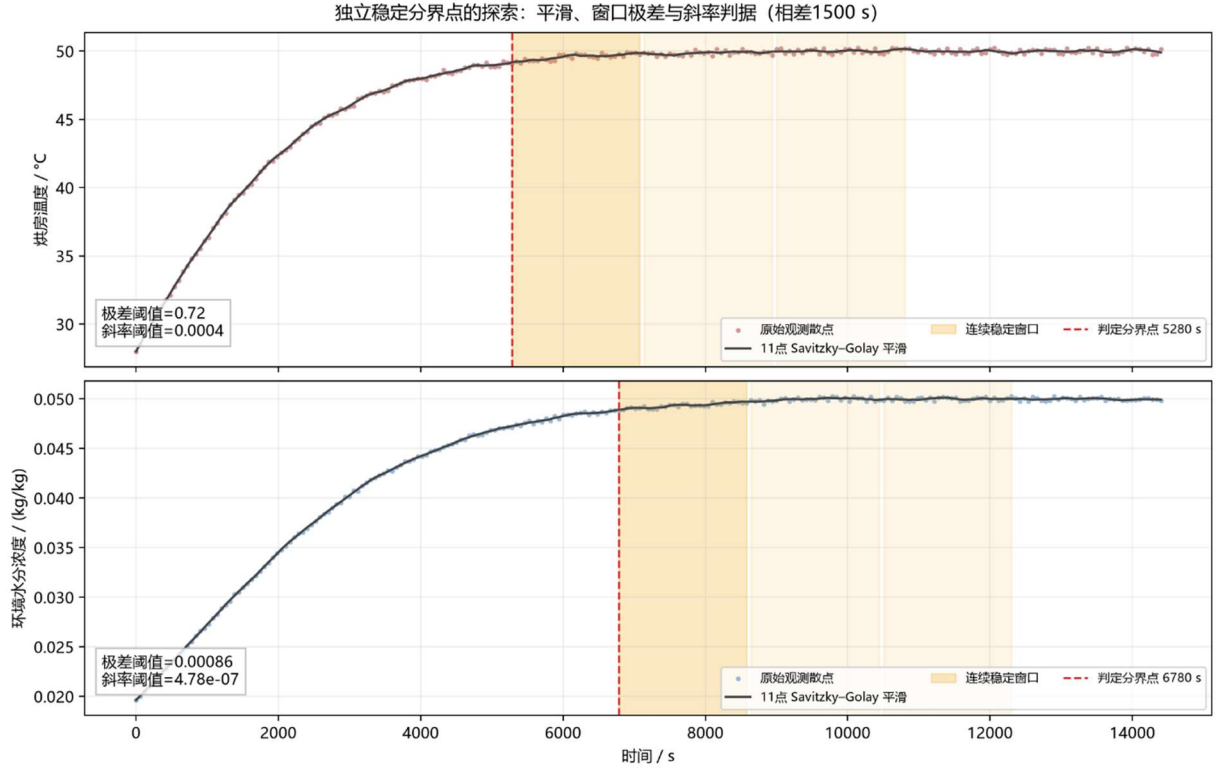


图 5 烘房温度、水分浓度稳定分界判定图

Step 2 考虑分界点的连续边界函数

此处拟合方式仍使用拉伸指数模型，温度 and 水分浓度采用各自不同的拟合窗口。温度拟合窗口为0~5280 s，水分浓度拟合窗口为0~6780 s。得到：

$$T_a(t) = 28 + 22.19188797 \left[1 - \exp \left(- \left(\frac{t}{1912.6396} \right)^{1.1352503} \right) \right], \quad 0 \leq t \leq 5280$$

$$C_a(t) = 0.01963 + 0.03089867 \left[1 - \exp \left(- \left(\frac{t}{2770.4870} \right)^{1.2435327} \right) \right], \quad 0 \leq t \leq 6780$$

拟合后分别计算温度和水分浓度得 RMSE，为 0.13270585 °C 和 1.82417×10^{-4} kg/kg。

为保证函数平滑过渡，此处把稳定分界点后的第一个采样步长（ $W = 1800s$ ）作为过渡区，平滑接入尾段均值。求得温度和水分浓度的过渡区为：

$$t_{s,T} = 5280 \text{ s}, [t_{-,T}, t_{+,T}] = [5280, 7080] \text{ s}$$

$$t_{s,C} = 6780 \text{ s}, [t_{-,C}, t_{+,C}] = [6780, 8580] \text{ s}$$

最终得到 T 和 C 的函数表达式为（时间 t 以 s 计）：

$$T_a(t) = \begin{cases} 28 + 22.19188797 \left[1 - \exp \left(- \left(\frac{t}{1912.6396} \right)^{1.1352503} \right) \right], & 0 \leq t \leq 5280, \\ \left(1 - \frac{t-5280}{1800} \right) \left\{ 28 + 22.19188797 \left[1 - \exp \left(- \left(\frac{t}{1912.6396} \right)^{1.1352503} \right) \right] \right\} + \frac{t-5280}{1800} 49.88579739, & 5280 < t < 7080, \\ 49.88579739, & t \geq 7080, \end{cases}$$

$$C_a(t) = \begin{cases} 0.01963 + 0.03089867 \left[1 - \exp\left(-\left(\frac{t}{2770.4870}\right)^{1.2435327}\right) \right], & 0 \leq t \leq 6780, \\ \left(1 - \frac{t-6780}{1800}\right) \left\{ 0.01963 + 0.03089867 \left[1 - \exp\left(-\left(\frac{t}{2770.4870}\right)^{1.2435327}\right) \right] \right\} + \frac{t-6780}{1800} 0.04982109, & 6780 < t < 8580, \\ 0.04982109, & t \geq 8580. \end{cases}$$

其中 T_a 的单位为 $^{\circ}\text{C}$, C_a 的单位为 kg/kg 。

最终, 分段拟合效果图如图 6, 图中蓝色段为稳定分界点前拟合函数, 橙色段为 1800s 线性平滑连接, 绿色段为稳定尾段均值, 散点为附件 1 原始观测。

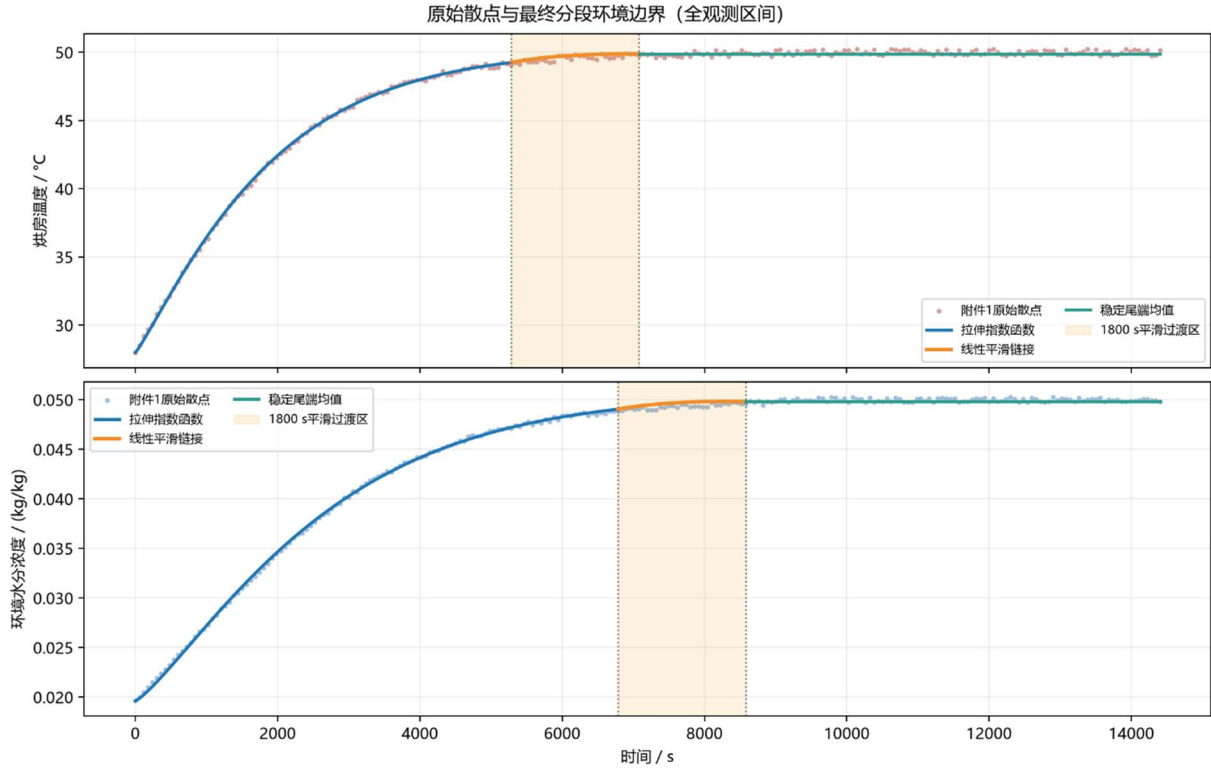


图 6 烘房温度、水分浓度分段拟合效果图

5.2.2 模型建立

问题二仍采用固定半径的一维圆柱径向模型, 但药材物性会随局部状态变化。各物性关系为

$$\begin{aligned} \rho(C) &= 650 + 128C \\ c_p(C) &= 1450 + 2736 \frac{C}{C+1} \\ k(C) &= 0.21 + 0.38 \frac{C}{C+1} \\ D(C, T) &= 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T+273.15}\right] \end{aligned}$$

将上述经验公式代入问题一的式(2)、式(3), 得到非线性热湿耦合模型:

$$\begin{aligned} \rho(C) c_p(C) \frac{\partial T}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left[r k(C) \frac{\partial T}{\partial r} \right] \\ \frac{\partial C}{\partial t} &= \frac{1}{r} \frac{\partial}{\partial r} \left[r D(C, T) \frac{\partial C}{\partial r} \right] \end{aligned}$$

问题二采用与问题一相同的初始条件、中心对称条件及表面交换条件。

综上，建立变物性热湿耦合模型：

$$\left\{ \begin{array}{l} \rho(C)c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[rk(C) \frac{\partial T}{\partial r} \right] \\ \frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left[rD(C,T) \frac{\partial C}{\partial r} \right] \\ \rho(C) = 650 + 128C \\ c_p(C) = 1450 + 2736 \frac{C}{C+1} \\ k(C) = 0.21 + 0.38 \frac{C}{C+1} \\ D(C,T) = 2.4 \times 10^{-3} \exp\left(-\frac{0.45}{C}\right) \exp\left[-\frac{3850}{T+273.15}\right] \quad 0 < r < R \\ T(r, 0) = T_0 \\ C(r, 0) = C_0 \\ \left. \frac{\partial T}{\partial r} \right|_{r=0} = 0 \\ \left. \frac{\partial C}{\partial r} \right|_{r=0} = 0 \\ -k \left. \frac{\partial T}{\partial r} \right|_{r=R} = h[T(R,t) - T_a(t)] \\ -D(C(R,t)) \left. \frac{\partial C}{\partial r} \right|_{r=R} = h_m[C(R,t) - C_a(t)] \end{array} \right.$$

5.2.3 模型求解

Step 1 将偏微分方程化为半离散方程

此处沿用问题一的径向有限体积法，但不再化为常系数形式，而是分别保留随水分浓度 C 变化的 $\rho_i c_{p,i}$ 和热传导系数 k_i 。由于 $k(C)$ 随水分浓度平滑变化，故在内部界面 $r_{i+1/2}$ 处采用算数平均

$$k_{i+1/2} = \frac{k_i + k_{i+1}}{2}$$

对于水分浓度，先取 $T_{i+1/2} = (T_i + T_{i+1})/2$ ，再作 Kirchhoff 平均

$$D_{i+1/2}^K = \int_0^1 D(T_{i+1/2}, C_i + s(C_{i+1} - C_i)) ds$$

这里的 $D_{i+1/2}^K$ 与问题一具有相同的积分含义，只是扩散系数还依赖界面的算术平均温度。

由此定义温度和水分的界面通量变量为

$$F_{i+1/2}^T = r_{i+1/2} k_{i+1/2} \frac{T_{i+1} - T_i}{\Delta r}$$

$$F_{i+1/2}^C = r_{i+1/2} D_{i+1/2} \frac{C_{i+1} - C_i}{\Delta r}$$

中心界面仍满足

$$F_{-1/2}^T = F_{-1/2}^C = 0$$

表面交换条件则转化为

$$F_{N+1/2}^T = Rh[T_a(t) - T_N] \quad F_{N+1/2}^C = Rh_m[C_a(t) - C_N]$$

因此，第 i 个控制体上的半离散方程为

$$\rho_i c_{p,i} w_i \frac{dT_i}{dt} = F_{i+1/2}^T - F_{i-1/2}^T$$

$$w_i \frac{dC_i}{dt} = F_{i+1/2}^C - F_{i-1/2}^C$$

Step 2 常微分方程组求解

全部节点温度 and 水分浓度组成状态向量

$$\mathbf{y}(t) = \begin{bmatrix} \mathbf{T}(t) \\ \mathbf{C}(t) \end{bmatrix}$$

得到常微分方程组

$$\frac{d\mathbf{y}}{dt} = \mathbf{f}(t, \mathbf{y})$$

再采用与问题一相同的隐式 BDF 方法求解。每次计算 $\mathbf{f}(t, \mathbf{y})$ 时，均根据当前节点状态 T_i, C_i 重新计算 ρ_i 、 $c_{p,i}$ 、 k_i 、 D_i 。将物理状态右端记为

$$\mathbf{f} = \begin{bmatrix} \mathbf{f}^T \\ \mathbf{f}^C \end{bmatrix}$$

隐式迭代求解器所需的 Jacobian 矩阵可以写成热湿耦合的四分块形式。先记

$$b_i = \rho_i c_{p,i} \quad b_i' = \frac{d(\rho c_p)}{dC}(C_i) \quad k_i' = \frac{dk}{dC}(C_i)$$

并用 $D_C = \frac{\partial D}{\partial C}$ 、 $D_T = \frac{\partial D}{\partial T}$ 表示扩散系数的两个偏导。于是

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{y}} = \begin{bmatrix} \mathbf{J}_{TT} & \mathbf{J}_{TC} \\ \mathbf{J}_{CT} & \mathbf{J}_{CC} \end{bmatrix}$$

这里第一个下标表示右端方程的类型，第二个下标表示被求导的状态变量，例如

$$\mathbf{J}_{TC} = \frac{\partial \mathbf{f}^T}{\partial \mathbf{C}}, \quad \mathbf{J}_{CT} = \frac{\partial \mathbf{f}^C}{\partial \mathbf{T}}$$

为表示方便，记

$$\mathcal{L}_i^T = F_{i+1/2}^T - F_{i-1/2}^T \quad \mathcal{L}_i^C = F_{i+1/2}^C - F_{i-1/2}^C$$

则

$$f_i^T = \frac{\mathcal{L}_i^T}{w_i b_i} \quad f_i^C = \frac{\mathcal{L}_i^C}{w_i}$$

其中， w_i 为第 i 个控制体的几何权重， $b_i = \rho_i c_{p,i}$ 随 C_i 变化，所以求 $\frac{\partial f_i^T}{\partial C_i}$ 时还必须对分母 b_i 求导。

接下来求 Jacobian 矩阵的 4 个分块矩阵。

求 $\mathbf{J}_{TT}$ 。在内部界面 $r_{i+1/2}$ 处，有

$$\frac{\partial F_{i+1/2}^T}{\partial T_i} = -r_{i+1/2} \frac{k_{i+1/2}}{\Delta r} \quad \frac{\partial F_{i+1/2}^T}{\partial T_{i+1}} = r_{i+1/2} \frac{k_{i+1/2}}{\Delta r}$$

因此，对 $j = i-1, i, i+1$ ，Jacobian 矩阵的元素由相邻两个界面通量相减得到：

$$\frac{\partial f_i^T}{\partial T_j} = \frac{1}{w_i b_i} \left(\frac{\partial F_{i+1/2}^T}{\partial T_j} - \frac{\partial F_{i-1/2}^T}{\partial T_j} \right)$$

这构成 $\mathbf{J}_{TT}$ 的三对角部分。

求 $\mathbf{J}_{TC}$ 。由于

$$k_{i+1/2} = \frac{k_i + k_{i+1}}{2}$$

有

$$\frac{\partial F_{i+1/2}^T}{\partial C_i} = r_{i+1/2} \frac{k'_i}{2} \frac{T_{i+1} - T_i}{\Delta r} \quad \frac{\partial F_{i+1/2}^T}{\partial C_{i+1}} = r_{i+1/2} \frac{k'_{i+1}}{2} \frac{T_{i+1} - T_i}{\Delta r}$$

这些是热通量中由 $k(C)$ 引起的交叉项。除此之外， f_i^T 的分 $w_i b_i$ 也依赖 C_i ，故完整求导式为：

$$\begin{aligned} \frac{\partial f_i^T}{\partial C_i} &= \frac{1}{w_i b_i} \frac{\partial \mathcal{L}_i^T}{\partial C_i} + \frac{\mathcal{L}_i^T}{w_i} \frac{\partial (b_i^{-1})}{\partial C_i} \\ &= \frac{1}{w_i b_i} \frac{\partial \mathcal{L}_i^T}{\partial C_i} - \frac{\mathcal{L}_i^T}{w_i b_i^2} b_i' \\ &= \frac{1}{w_i b_i} \frac{\partial \mathcal{L}_i^T}{\partial C_i} - f_i^T \frac{b_i'}{b_i} \end{aligned}$$

求 $\mathbf{J}_{CC}$ 。记 $\Delta C = C_{i+1} - C_i$ 。Kirchhoff 平均的端点处水分浓度导数推导流程同问题一的式(8)，但需要把界面温度

$$T_{i+1/2} = \frac{T_i + T_{i+1}}{2}$$

视为当前界面的已知算术平均，即将问题一中的 $D_C(C)$ 替换为 $D_C(T_{i+1/2}, C)$ ，并用同一组的 8 点 Gauss-Legendre 节点计算这两个积分。

由 $F_{i+1/2}^C = r_{i+1/2} D_{i+1/2}^K \Delta C / \Delta r$ ，可得

$$\frac{\partial F_{i+1/2}^C}{\partial C_i} = r_{i+1/2} \frac{\Delta C}{\Delta r} \frac{\partial_{C_i} D_{i+1/2}^K - D_{i+1/2}^K}{\Delta r} \quad \frac{\partial F_{i+1/2}^C}{\partial C_{i+1}} = r_{i+1/2} \frac{\Delta C}{\Delta r} \frac{\partial_{C_{i+1}} D_{i+1/2}^K + D_{i+1/2}^K}{\Delta r}$$

最后只需做通量差分：

$$\frac{\partial f_i^C}{\partial C_j} = \frac{1}{w_i} \left(\frac{\partial F_{i+1/2}^C}{\partial C_j} - \frac{\partial F_{i-1/2}^C}{\partial C_j} \right)$$

这构成 $\mathbf{J}_{CC}$ 的三对角部分。

求 $\mathbf{J}_{CT}$ 。对温度求导时，界面算术平均的导数为

$$\frac{\partial T_{i+1/2}}{\partial T_j} = \frac{1}{2}, \quad j = i, i+1$$

故

$$\frac{\partial D_{i+1/2}^K}{\partial T_i} = \frac{1}{2} \int_0^1 D_T(T_{i+1/2}, C_i + s \Delta C) ds \quad \frac{\partial D_{i+1/2}^K}{\partial T_{i+1}} = \frac{1}{2} \int_0^1 D_T(T_{i+1/2}, C_i + s \Delta C) ds$$

再对水分浓度界面通量求导：

$$\frac{\partial F_{i+1/2}^C}{\partial T_j} = r_{i+1/2} \frac{\Delta C}{\Delta r} \frac{\partial D_{i+1/2}^K}{\partial T_j}, \quad j = i, i+1$$

于是

$$\frac{\partial f_i^C}{\partial T_j} = \frac{1}{w_i} \left(\frac{\partial F_{i+1/2}^C}{\partial T_j} - \frac{\partial F_{i-1/2}^C}{\partial T_j} \right)$$

由表面通量

$$F_{N+1/2}^T = Rh(T_a - T_N), \quad F_{N+1/2}^C = Rh_m(C_a - C_N)$$

可知，表面项对最外节点右端的导数分别为

$$\left. \frac{\partial f_N^T}{\partial T_N} \right|_{\text{surface}} = -\frac{Rh}{w_N b_N}, \quad \left. \frac{\partial f_N^C}{\partial C_N} \right|_{\text{surface}} = -\frac{Rh_m}{w_N}$$

它们只修正 $\mathbf{J}_{TT}$ 和 $\mathbf{J}_{CC}$ 的最外侧对角元；中心零通量的导数为零。

问题二在 $0 \leq t \leq 10800$ s 内积分，最终采用 $N = 400$ 个均匀径向区间，相对容差和绝对容差分别为 2×10^{-10} 和 2×10^{-12} ，最大内部步长为 5s。时间积分结果按 1s 采样保存。

5.2.4 求解结果与分析

求得 3h 内每隔 0.5h、到药材中心距离 0、0.5、1、1.5、2cm 处的结果如表 5、表 6，每隔 1s、到药材中心距离每隔 0.1cm 的完整结果已保存到文件 *result2.xlsx* 中。

由表可见，3h 时中心与表面温差约 0.0421°C ，说明传热已接近均匀，水分浓度由中心 1.7658 递减至表面 1.0075kg/kg，水分扩散明显慢于升温。

表 5 3 小时内药材的温度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
0.5	32.1841	32.3803	32.9740	33.9804	35.4291
1.0	40.3657	40.5363	41.0412	41.8616	42.9725
1.5	45.8446	45.9350	46.2006	46.6265	47.1945
2.0	48.5554	48.5923	48.6996	48.8682	49.0859
2.5	49.5062	49.5168	49.5478	49.5965	49.6594
3.0	49.7828	49.7857	49.7942	49.8076	49.8249

表 6 3 小时内药材的水分浓度

时间/h	到药材中心的距离/cm				
	0		0		0
0.5	2.5499	2.5489	2.5256	2.3257	1.6488
1.0	2.5257	2.4948	2.3579	2.0231	1.4709

1.5	2.3862	2.3258	2.1345	1.8020	1.3477
2.0	2.1704	2.1080	1.9232	1.6258	1.2320
2.5	1.9557	1.8997	1.7355	1.4720	1.1171
3.0	1.7658	1.7162	1.5701	1.3333	1.0075

将 *result2.xlsx* 结果可视化得到图 7。图中左右两个面板分别显示温度场和水分浓度场随横轴时间、纵轴距中心距离的变化情况。

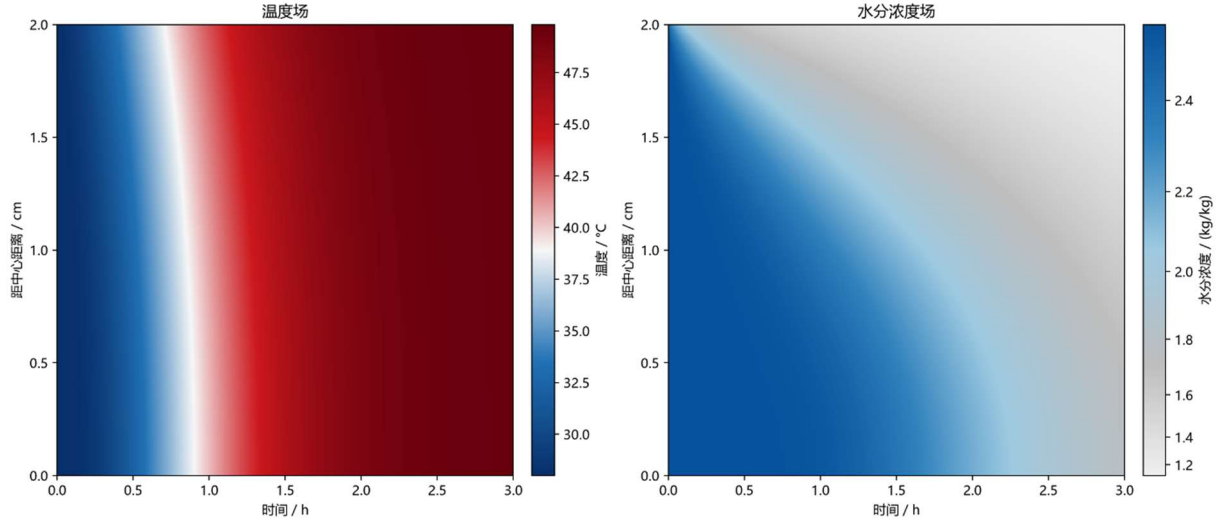


图 7 前 3 小时药材温度、水分浓度场变化图

5.3 问题三模型的建立与求解

5.3.1 模型建立

问题三不再建立新的传热传质方程，而是沿用问题二的变物性热湿耦合模型以及独立边界处理和附录 3 的经验公式。固定半径为 $R = 0.02 \text{ m}$ ，初始和边界条件仍为式(4)、式(5)、式(6)。

题目要求的是药材各处都低于阈值 $C_{cr} = 0.15$ ，因此定义达标时间为

$$t_* = \inf \left\{ t \geq 0 : \max_{0 \leq r \leq R} C(r, t) < C_{cr} \right\} \quad (9)$$

5.3.2 模型求解

为解析后期药材表面附近的陡峭水分浓度梯度，问题三采用非均匀的网格加密：

$$r_i = R \frac{1 - \exp(-4i/N)}{1 - \exp(-4)} \quad i = 0, \dots, N, \quad N = 300$$

时间上仍采用隐式 BDF，环境变化阶段最大步长 5s，稳定尾段最大步长 300s。每一时间步都检查水分浓度正值性（所有节点 $C_i \geq 0$ ）、径向单调性（从中心到表面的离散增量 $C_{i+1} - C_i$ 不出现可见的正值）和累计表面交换量（将边界传质通量按干物质基准累计的水分浓度变化量）。

5.3.3 求解结果与分析

解得药材烘干所需时间为

$$t_* = 207644.4000 \text{ s} \approx 57.6790 \text{ h}$$

此时最大水分浓度为 0.1500kg/kg ，位于药材中心。

每隔 6h 、到药材中心距离每隔 0.5cm 的水分浓度如表 7，药材内部水分浓度每隔 60s 、到药材中心距离每隔 0.1cm 的完整结果已保存到文件 *result3.xlsx* 中。

表 7 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm				
	0	0.5	1	1.5	2
6	1.0182	0.9892	0.9023	0.7553	0.5322
12	0.4571	0.4440	0.4038	0.3302	0.1641
18	0.2995	0.2921	0.2692	0.2257	0.0844
24	0.2383	0.2332	0.2171	0.1859	0.0663
30	0.2062	0.2022	0.1895	0.1643	0.0597
36	0.1862	0.1828	0.1721	0.1506	0.0565
42	0.1723	0.1694	0.1599	0.1409	0.0547
48	0.1621	0.1594	0.1509	0.1336	0.0536
54	0.1541	0.1517	0.1438	0.1279	0.0528
57.6790	0.1500	0.1477	0.1402	0.1249	0.0525

将 *result3.xlsx* 结果可视化得到图 8。图中，以黑色虚线标出 $C = 0.15\text{kg/kg}$ 的干燥参考线。

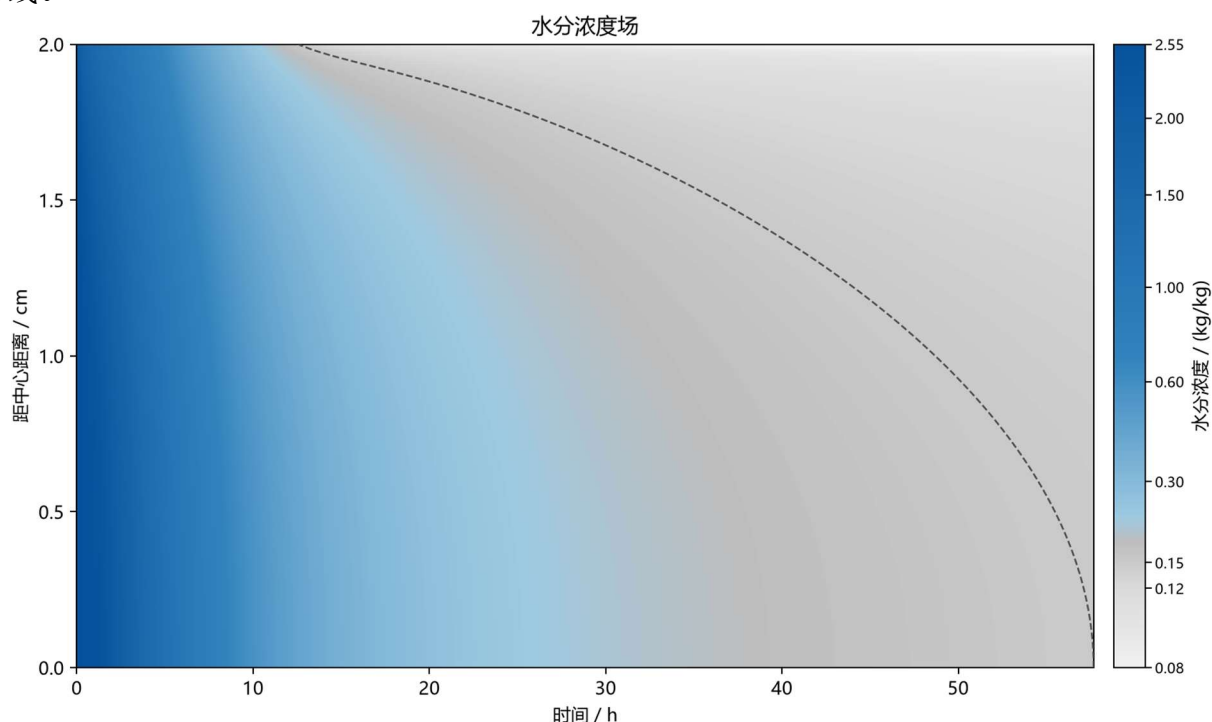


图 8 药材水分浓度场变化图

5.4 问题四模型的建立与求解

5.4.1 模型准备

附件 2 每隔 $\Delta t_R = 1800\text{s}$ 给出一个半径记录，共 145 条，记录范围为 $0 \sim 259200\text{s}$ ，可绘制为半径观测散点图，如图 9。记第 k 个观测时刻和半径分别为 $t_k = k \Delta t_R$ 与 R_k ，单位分别为 s 、 cm 。

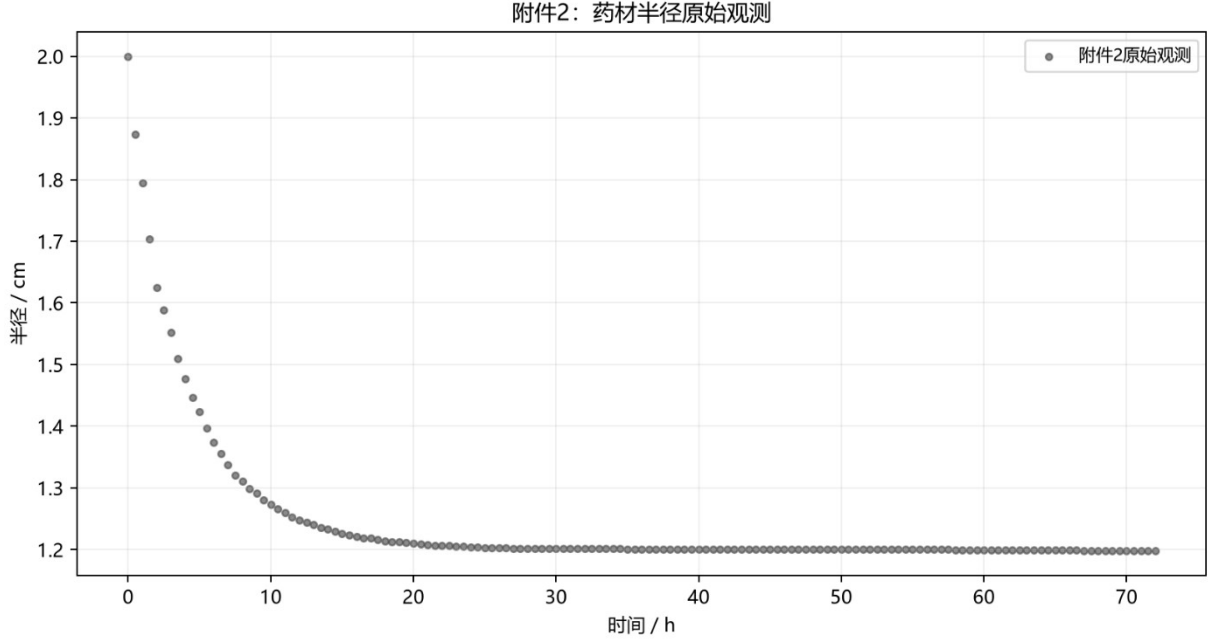


图 9 药材半径变化图

由于第四问的实际计算区域 $\Omega(t) = \{r: 0 \leq r \leq R(t)\}$ 是随时间变化的，所以在长度不变、径向均匀收缩的假设下，材料点的相对位置保持不变，于是记 ξ 为材料坐标，则

$$r(t; \xi) = \xi R(t), \quad 0 \leq \xi \leq 1$$

因而， $R(t)$ 不只是对散点的曲线拟合，更是移动区域的几何输入。若 $R(t)$ 出现半径回升、负半径或数据范围外的过冲，就会把非物理几何误差传递到温度场和水分浓度场中。所以，半径函数应同时满足贴合观测、保持正值、单调不增和不过度越界等要求。

设 $g(t)$ 为候选半径函数，目标是将半径连续化的同时满足物理约束，候选方法包括分段线性、PCHIP^[5]、Akima、自然三次样条和平滑样条。验证采用与第一问相同的时间阻塞交叉验证，从内部观测中每隔 5 个记录留出 1 个点，记留出下标集合为 $\mathcal{V}$ ，其余记录组成训练集 $\mathcal{I}_{\text{tr}}$ 。对每一种方法，先用 $\mathcal{I}_{\text{tr}}$ 构造 $g_{-\mathcal{V}}(t)$ ，再在留出点上计算交叉验证误差。误差越小，表示函数对未参与拟合的半径记录预测越准确；同时还要在加密时间网络上检查半径是否回升以及是否超出训练样本范围。

若将留出点上的预测记为 $g_{-\mathcal{V}}(t_j)$ ，则交叉验证均方根误差为

$$RMSE = \sqrt{\frac{1}{|\mathcal{V}|} \sum_{j \in \mathcal{V}} [g_{-\mathcal{V}}(t_j) - R_j]^2}$$

设 $\{\tau_\ell\}$ 为覆盖观测区间的加密时间网格，定义半径回升段数和最大越界量为

$$N_{\uparrow}(g) = \#\{\ell: g(\tau_{\ell+1}) - g(\tau_\ell) > 10^{-9} \text{ m}\}$$

$$E_{\text{out}}(g) = \max_{\tau_\ell} ([R_{\min}^{\text{tr}} - g(\tau_\ell)]_+ + [g(\tau_\ell) - R_{\max}^{\text{tr}}]_+), \quad [x]_+ = \max(x, 0)$$

其中 $R_{\min}^{\text{tr}}$ 和 $R_{\max}^{\text{tr}}$ 为训练样本的最小、最大半径。程序在 721 点网格上计算越界量，在 5000 点网格上统计回升段数。于是，候选函数首先要满足 $N_{\uparrow} = 0$ 且 $E_{\text{out}} = 0$ ，再在物理可接受的候选中比较 E_{cv} 。比较结果如表 8。

表 8 基于物理约束的候选方法比较表

方法	RMSE/cm	半径增大段数 $N_{\uparrow}$	最大越界 E_{out}/cm
分段线性	0.004505	0	0
PCHIP	0.003829	0	0
Akima	0.003987	41	0
自然三次样条	0.003942	687	0.000114
平滑样条	0.003455	870	0.000105

平滑样条、自然三次样条和 Akima 均出现局部回升，因而首先被排除。满足单调性和无越界约束的候选只有分段线性和 PCHIP，其中 PCHIP 的 RMSE 为 0.003829 cm，低于分段线性的 0.004505 cm，且拥有分段线性所没有的连续曲率，因此选择该模型。

在上述物理约束筛选后，连续半径函数写为

$$R(t) = \begin{cases} \mathcal{I}_{\text{PCHIP}}\{(t_k, R_k)\}, & 0 \leq t \leq 259200 \text{ s} \\ R(259200), & t > 259200 \text{ s} \end{cases}$$

其中 $\mathcal{I}_{\text{PCHIP}}$ 表示由 (t_k, R_k) 构造的保形分段三次插值，输出单位为 cm。

5.4.2 模型建立

Step 1 建立收缩药材的移动边界描述

考虑药材收缩，物理区域变为

$$0 \leq r \leq R(t)$$

假设药材长度不变并沿径向均匀收缩，材料点的相对径向位置保持不变。对相对位置为 ξ 的材料点，有

$$r(t) = \xi R(t)$$

对时间求导，得到内部材料速度

$$v(r, t) = \frac{dr}{dt} = \xi \dot{R}(t) = \frac{\dot{R}(t)}{R(t)} r \quad (10)$$

对于同时依赖空间和时间的一般场量 $\phi(r, t)$ ，沿材料点轨迹使用多元复合函数的链式法则：

$$\frac{d}{dt} \phi(r(t), t) = \frac{\partial \phi}{\partial t} + \frac{dr}{dt} \frac{\partial \phi}{\partial r}$$

由此定义材料导数

$$\frac{D\phi}{Dt} = \frac{\partial \phi}{\partial t} + v \frac{\partial \phi}{\partial r}$$

其中，第一项表示固定空间位置处的局部变化，第二项表示材料运动到不同位置造成的变化。用材料导数描述收缩材料内部状态，实际空间中的控制方程为

$$\rho c_p (T_t + v T_r) = \frac{1}{r} \frac{\partial}{\partial r} (rk T_r)$$

$$C_t + vC_r = \frac{1}{r} \frac{\partial}{\partial r} (rDC_r)$$

Step 2 将移动区域化为固定材料坐标区域

为避免不断移动计算网格，引入归一化材料坐标

$$\xi = \frac{r}{R(t)}, \quad 0 \leq \xi \leq 1 \quad (11)$$

并定义

$$\Theta(\xi, t) = T(\xi R(t), t) \quad U(\xi, t) = C(\xi R(t), t)$$

固定实际位置 r 时，

$$\left. \frac{\partial \xi}{\partial t} \right|_r = - \frac{\xi \dot{R}}{R}$$

由链式法则，

$$C_t|_r = U_t - \frac{\xi \dot{R}}{R} U_\xi \quad C_r = \frac{1}{R} U_\xi$$

结合 $v = \xi \dot{R}$ ，有

$$vC_r = \frac{\xi \dot{R}}{R} U_\xi$$

所以坐标运动项和材料运动项恰好抵消：

$$C_t + vC_r = U_t$$

同理，

$$T_t + vT_r = \Theta_t$$

又因为

$$\frac{\partial}{\partial r} = \frac{1}{R} \frac{\partial}{\partial \xi}$$

于是

$$\begin{aligned} \frac{1}{r} \frac{\partial}{\partial r} (rkT_r) &= \frac{1}{R^2 \xi} \frac{\partial}{\partial \xi} (\xi k \Theta_\xi), \\ \frac{1}{r} \frac{\partial}{\partial r} (rDC_r) &= \frac{1}{R^2 \xi} \frac{\partial}{\partial \xi} (\xi D U_\xi). \end{aligned}$$

因此，固定材料坐标区域上的移动边界模型为

$$\rho(U) c_p(U) \Theta_t = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} [\xi k(U) \Theta_\xi] \quad (12)$$

$$U_t = \frac{1}{R(t)^2 \xi} \frac{\partial}{\partial \xi} [\xi D(\Theta, U) U_\xi] \quad (13)$$

Step 3 物性经验公式、初始条件与边界条件

问题四采用新的局部物性关系：

$$\rho(U) = 760 + 90U$$

$$c_p(U) = 1850 + 2150 \frac{U}{U+1}$$

$$k(U) = 0.12 + 0.20 \frac{U}{U+1}$$

$$D(\Theta, U) = 4.2 \times 10^{-4} \exp\left(-\frac{0.30}{U}\right) \exp\left[-\frac{3850}{\Theta + 273.15}\right]$$

初始条件与中心对称条件为

$$\Theta(\xi, 0) = T_0 \quad U(\xi, 0) = C_0$$

$$\Theta_\xi(0, t) = 0 \quad U_\xi(0, t) = 0$$

由于 $T_r = \frac{\Theta_\xi}{R}$ 、 $C_r = \frac{U_\xi}{R}$ ，移动表面 $\xi = 1$ 处的交换条件为

$$\begin{aligned} -\frac{k(U_s)}{R(t)} \Theta_\xi(1, t) &= h[\Theta_s - T_a(t)] \\ -\frac{D(\Theta_s, U_s)}{R(t)} U_\xi(1, t) &= h_m[U_s - C_a(t)] \end{aligned} \quad (14)$$

Step 4 建立材料坐标下的水分守恒关系

水分浓度定义为 $U = \frac{m_w}{m_d}$ ，缩小材料体积不会改变水质量与干物质质量之比。长度不变且干物质总质量守恒时，

$$\rho_d(t) \pi R(t)^2 L = \rho_{d0} \pi R_0^2 L$$

从而

$$\rho_d(t) = \rho_{d0} \frac{R_0^2}{R(t)^2}$$

材料坐标下的平均水分浓度为

$$\bar{U}(t) = 2 \int_0^1 U(\xi, t) \xi \, d\xi$$

对式(13)乘以 2ξ 并在 $[0, 1]$ 上积分，再利用中心零通量和表面传质条件，得到整体收支关系

$$\frac{d\bar{U}}{dt} = \frac{2h_m}{R(t)} [C_a(t) - U_s]$$

该式表明平均水分浓度的变化完全由表面传质通量决定，可用于检查移动边界离散的守恒一致性。

Step 5 模型汇总

综上，建立移动边界变物性热湿耦合模型：

$$\left\{ \begin{array}{l} \rho(U)c_p(U)\Theta_t = \frac{1}{R(t)^2\xi} \frac{\partial}{\partial \xi} [\xi k(U)\Theta_\xi] \\ U_t = \frac{1}{R(t)^2\xi} \frac{\partial}{\partial \xi} [\xi D(\Theta, U)U_\xi] \\ \rho(U) = 760 + 90U \\ c_p(U) = 1850 + 2150 \frac{U}{U+1} \\ k(U) = 0.12 + 0.20 \frac{U}{U+1} \\ \Theta(\xi, 0) = T_0 \\ U(\xi, 0) = C \\ \Theta_\xi(0, t) = 0 \\ U_\xi(0, t) = 0 \\ -\frac{k(U_s)}{R(t)}\Theta_\xi(1, t) = h[\Theta_s - T_a(t)] \\ -\frac{D(\Theta_s, U_s)}{R(t)}U_\xi(1, t) = h_m[U_s - C_a(t)] \end{array} \right. \quad 0 < r < R$$

5.4.3 模型求解

移动边界通过 $\xi = r/R(t)$ 映射到固定区间 $[0, 1]$ 。为解析移动表面附近较大的梯度，在均匀参数 $x_i = i/N$ 上构造表面加密网格

$$\xi_i = \frac{1 - \exp(-4x_i)}{1 - \exp(-4)}, \quad i = 0, 1, \dots, N$$

以 ξ_i 为中心建立环状控制体，界面取相邻节点中点，几何权重为

$$w_i = \int_{a_i}^{b_i} \xi \, d\xi = \frac{b_i^2 - a_i^2}{2}$$

将式(12)和式(13)两端分别乘以 ξ ，并在第 i 个控制体上积分。内部界面温度与水分浓度通量变量取为

$$F_{i+1/2}^T = \xi_{i+1/2} k_{i+1/2} \frac{\Theta_{i+1} - \Theta_i}{\xi_{i+1} - \xi_i}, \quad k_{i+1/2} = \frac{k_i + k_{i+1}}{2}$$

$$F_{i+1/2}^C = \xi_{i+1/2} D_{i+1/2}^K \frac{U_{i+1} - U_i}{\xi_{i+1} - \xi_i}, \quad D_{i+1/2}^K = \int_0^1 D\left(\frac{\Theta_i + \Theta_{i+1}}{2}, U_i + s(U_{i+1} - U_i)\right) ds$$

这里的 $D_{i+1/2}^K$ 与问题二的定义相同，温度仍在界面处取算术平均。

中心对称条件给出

$$F_{-1/2}^T = F_{-1/2}^C = 0$$

由式(14)的移动表面交换条件，最外侧通量变量为

$$F_{N+1/2}^T = R(t)h[T_a(t) - \Theta_N], \quad F_{N+1/2}^C = R(t)h_m[C_a(t) - U_N]$$

于是得到移动边界模型的半离散方程

$$\frac{d\Theta_i}{dt} = \frac{F_{i+1/2}^T - F_{i-1/2}^T}{R(t)^2 w_i \rho_i c_{p,i}}, \quad \frac{dU_i}{dt} = \frac{F_{i+1/2}^C - F_{i-1/2}^C}{R(t)^2 w_i} \quad (15)$$

为写出物理状态的 Jacobian, 令

$$\mathbf{y} = \begin{bmatrix} \Theta \\ U \end{bmatrix}, \quad \mathbf{f} = \begin{bmatrix} \mathbf{f}^\Theta \\ \mathbf{f}^U \end{bmatrix}$$

则

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{y}} = \begin{bmatrix} \mathbf{J}_{\Theta\Theta} & \mathbf{J}_{\Theta U} \\ \mathbf{J}_{U\Theta} & \mathbf{J}_{UU} \end{bmatrix}$$

其中第一个下标表示右端方程的类型, 第二个下标表示被求导的状态变量。四个分块矩阵的含义与问题二中的 $J_{TT}, J_{TC}, J_{CT}, J_{CC}$ 一一对应。四个分块矩阵的求解流程与问题二一致, 区别在于每个内部通量导数都要乘以 $1/R(t)^2$ 。对于最外节点的边界项, 有

$$\left. \frac{\partial f_N^\Theta}{\partial \Theta_N} \right|_{\text{surface}} = -\frac{h}{R(t) \rho_N c_{p,N} w_N}, \quad \left. \frac{\partial f_N^U}{\partial U_N} \right|_{\text{surface}} = -\frac{h_m}{R(t) w_N}$$

温度和水分浓度组成统一状态向量, 同样采用带稀疏解析 Jacobian 的隐式 BDF 方法求解。

问题四最终采用 $N = 300$ 个表面加密区间, 相对容差和绝对容差分别为 2×10^{-10} 和 2×10^{-12} 。前 4h 最大内部步长为 5s, 随后为 300s。达标时间的定义仍为式(9)。

最终求得的结果是关于固定材料坐标 ξ_i 的函数, 而题目要求输出实际距离 r_j 处的状态。时刻 t 的对应材料坐标为

$$\xi_j(t) = \frac{r_j}{R(t)}$$

当 $r_j \leq R(t)$ 时, 先将固定实际位置换算为材料坐标 $\xi_j(t) = r_j/R(t)$, 再对同一时刻各计算节点上的水分浓度数据 $\{(\xi_i, U_i(t))\}$ 进行保形分段三次插值, 从而得到实际位置 r_j 处的水分浓度

$$C(r_j, t) = U\left(\frac{r_j}{R(t)}, t\right)$$

当 $r_j > R(t)$ 时, 该位置已经位于药材外部, 结果留空, 在药材表面始终取 $U(1, t)$ 。

5.4.4 求解结果与分析

基于式(9), 解得药材烘干所需时间为

$$184565.8800 \text{ s} = 51.2683 \text{ h}$$

向上保留四位小数并继续积分后, 报告烘干时间为 **51.2683h**。全域最大水分浓度 0.1500kg/kg。

每隔 6h、到药材中心距离每隔 0.5 cm 的水分浓度如表 9, 药材内部水分浓度每隔 60s 时间、0.1cm 距离的完整结果已保存到文件 *result4.xlsx* 中。

将 *result4.xlsx* 结果可视化得到图 10, 黑色实线表示当前药材表面, 表面至距离中心 2.0cm 的收缩外部区域以均匀斜线遮罩, 并以灰色虚线标出 $C = 0.15 \text{ kg/kg}$ 的干燥参考线。

表 9 药材烘干过程的水分浓度

时间/h	到药材中心的距离/cm			
	0 cm	0.5 cm	1 cm	药材表面
6	1.7212	1.5390	1.0230	0.4197
12	0.7398	0.6564	0.4091	0.1668
18	0.4101	0.3698	0.2403	0.0891
24	0.2860	0.2618	0.1794	0.0672
30	0.2270	0.2100	0.1496	0.0593
36	0.1933	0.1801	0.1319	0.0558
42	0.1717	0.1607	0.1202	0.0539
48	0.1565	0.1471	0.1118	0.0528
51.2683	0.1500	0.1413	0.1081	0.0524

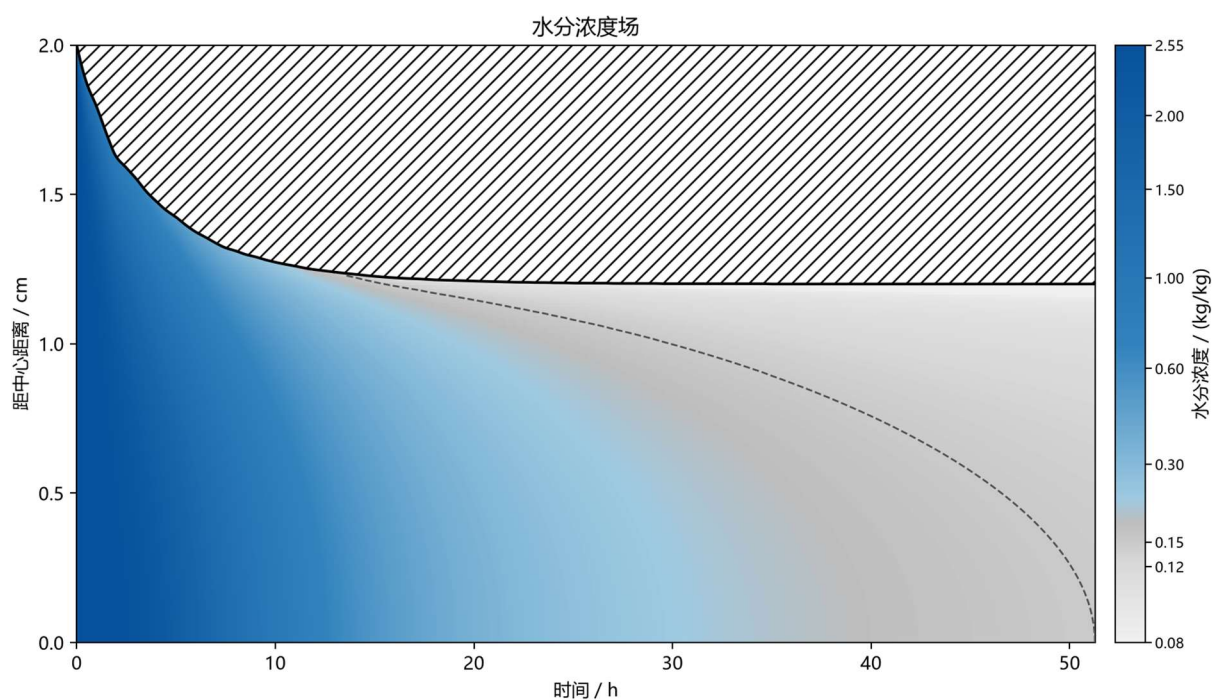


图 10 收缩药材水分浓度场变化图

六、模型检验与误差分析

本文的计算结果包括药材内部温度场、水分浓度场以及全域干燥终点。附件中没有药材内部温度、水分浓度场等完整观测，因此不能用实验真值评测模型。本节将误差分为三类：一是数值收敛性误差，包括网格密度、时间步长、Jacobian 和守恒残差等；二是参数敏感性误差，主要来自给定范围内的参数和边界扰动；三是模型假设带来的系统性误差。

6.1 数值收敛性误差

四个问题均采用有限体积离散和隐式 BDF 积分，正式网格密度依次为 $N = 6400, 400, 300, 300$ 。此处通过比较相邻网格来执行空间检验，通过收紧求解器步长和容差来执行时间检验，得到表 10。

表 10 四个问题的数值离散收敛性与守恒检验结果表

问题	空间检验	时间检验	最大收支残差
第一问	$\Delta C = 1.17 \times 10^{-5}$	$\Delta C = 4.32 \times 10^{-9}$	4.79×10^{-15}
第二问	$\Delta C = 7.53 \times 10^{-4}$	$\Delta C = 2.44 \times 10^{-8}$	4.88×10^{-15}
第三问	达标时刻差0.0563s	达标时刻差0.00245s	1.20×10^{-14}
第四问	达标时刻差0.1489s	达标时刻差0.0133s	7.99×10^{-15}

6.2 参数敏感性误差

单因素灵敏度采用 $S_{y,p} \approx \frac{y(1.2p) - y(0.8p)}{0.4y(p)}$ ，每次只改变一个参数，所得灵敏度的绝对值表示影响大小，正负号表示参数增大后的变化方向。问题一中参数扰动使用 $N = 800$ ，问题二中心差分使用 $N = 200$ 。主要结果如表 11。所得结果表明各参数对计算结果影响均在可控范围内。

表 11 关键传热传质参数的单因素敏感性与灵敏度分析表

参数扰动	主要输出变化
问题一 $h \times (0.8, 1.2)$	平均温度变化($-0.7027, +0.5480$)
问题一 $h_m \times (0.8, 1.2)$	平均水分浓度变化($+0.03758, -0.03265$)
问题一 $D \times (0.8, 1.2)$	平均水分浓度变化($+0.00884, -0.00684$)
问题二 3h 平均水分浓度	敏感性绝对值排序: $h_m > D > \rho c_p \approx h > k$

6.3 系统误差

系统误差主要来源于模型假设与实际过程不一致，本文模型中潜在来源包括：将药材视为均匀、各向同质的介质；采用轴对称的一维径向传热传质模型，忽略轴向差异及局部分均匀性；将复杂的表面对流传质过程等效为边界对流传质过程等等。

受篇幅限制，本文仅对长期边界扰动做分析。为考察实际波动的影响，令

$$B(t) = \bar{B} + \varepsilon^*(t)$$

其中 $\bar{B}$ 为尾段均值， $\varepsilon^*(t)$ 为稳定阶段观测值相对于均值边界的残差。计算取共同稳定阶段 $t \geq 8580$ s 的 98 个观测点，将温度和水分浓度残差成对组成 10 个观测点(600s)的时间块，进行 4 组固定种子的重采样，并将波动传入 Q3、Q4 求解器，得到表 12。

表 12 长期边界环境波动对于干燥终点预测的系统误差评估表

问题	均值尾段基准/h	平均波动±标准差/h	波动范围/h	平均变化/h
问题三	57.67893	57.67639±0.00147	57.67453~57.67796	-0.00253
问题四	51.26822	51.26699±0.00081	51.26606~51.26801	-0.00123

由表可知，长期边界波动阴气的最大波动分别为问题三 0.00253h 和问题四 0.00123h，相对预测时长的相对误差均小于 0.01%，在允许范围以内。

七、模型评价与推广

7.1 模型的优点

- (1) 模型具有明确的物理基础，从圆柱体内的能量守恒和水分守恒出发，以温度场和水分浓度场为基本状态变量，内部采用导热与水分扩散描述，表面采用有限速率的对流换热和有效传质边界描述，可推广至其他不同物性材料的情况。
- (2) 针对附件中的离散环境数据，模型使用初值固定的拉伸指数描述预热阶段的连续变化，并分别识别温度和环境水分的稳定分界点。分界点后平滑过渡接入各自的稳定尾段均值，避免了直接拼接产生的不连续，同时保留了温度和环境水分进入稳定阶段的时间差异。
- (3) 空间离散采用有限体积法，避免了有限差分在边界 $r=0$ 处的发散问题。相邻控制体之间共享同一个界面通量，因此内部通量能够成对抵消。对时间积分采用隐式BDF方法，并为变物性模型构造稀疏解析 Jacobian 矩阵加速求解器求解。
- (4) 当扩散系数随温度和水分浓度变化时，模型在相邻节点水分浓度区间内采用 Kirchhoff 积分平均，由此构造的界面通量能够在离散层面保持积分结构，适合水分浓度梯度较大或物性非线性较强的情形。

7.2 模型的不足

- (1) 模型把药材视为均匀、各向同性的圆柱体，只考虑径向传热传质，忽略轴向梯度和上下表面的热量、水分交换。当药材长度与直径处于同一数量级，或者上下表面暴露程度较高时，端部传热传质可能使实际干燥速度快于一维径向模型的预测。
- (2) 能量方程只考虑导热变化，没有显式加入水分蒸发吸热、水分迁移携带的焓以及气液两相间的能量交换。实际烘干中，表面或内部汽化可能降低局部温度，并通过温度相关的扩散系数进一步影响干燥速率。因此，当前模型可能高估升温速度或低估热湿耦合强度。

7.3 模型的推广

- (1) 本文的径向扩散模型可以统一写为 $u_t = \frac{1}{r^m} \frac{\partial}{\partial r} [r^m a(u, T) u_r]$ 。其中平板、无限长圆柱和球体分别对应 $m=0, 1, 2$ 。因此，只需改变几何权重、表面积与体积比以及中心条件的离散形式，就可以将模型推广到片状、条状、颗粒状或近似球形的中药材和食品。
- (2) 模型的控制方程和数值框架不依赖某一种具体药材。对于果蔬、谷物、木材或其他多孔介质，可以通过实验重新确定 $\rho(C)$ 、 $c_p(C)$ 、 $k(C)$ 、 $D(T, C)$ 、平衡水分浓度以及表面换热传质系数。环境边界也可以替换为多阶段升温、变风速、变湿度、间歇烘干等情况，从而比较不同烘干条件对内部梯度和完成时间的影响。

AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于赛题理解，文献查找，辅助编写代码、代码注释和调试代码，建模思路可行性分析，语言润色，文献引用格式整理，详细使用情况见支撑材料 *AI 工具使用详情.pdf*。

参考文献

- [1] RAMOS J I. Finite difference methods based on the Kirchhoff transformation and time linearization for the numerical solution of nonlinear reaction–diffusion equations[J]. *Computation*, 2024, 12(11): 218. DOI: 10.3390/computation12110218.
- [2] AKTER F, MUHURY R, SULTANA A, et al. A comprehensive review of mathematical modeling for drying processes of fruits and vegetables[J]. *International Journal of Food Science*, 2022, 2022: 6195257. DOI: 10.1155/2022/6195257.
- [3] SHAMPINE L F, REICHEL T M W. The MATLAB ODE suite[J]. *SIAM Journal on Scientific Computing*, 1997, 18(1): 1-22. DOI: 10.1137/S1064827594276424.
- [4] TULY S S, JOARDDER M U H, WELSH Z G, et al. Mathematical modelling of heat and mass transfer during jackfruit drying considering shrinkage[J]. *Energies*, 2023, 16(11): 4461. DOI: 10.3390/en16114461.
- [5] FRITSCH F N, BUTLAND J. A method for constructing local monotone piecewise cubic interpolants[J]. *SIAM Journal on Scientific and Statistical Computing*, 1984, 5(2): 300-304. DOI: 10.1137/0905021.
- [6] SERBAN R, HINDMARSH A C. CVODES: The sensitivity-enabled ODE solver in SUNDIALS[C]// *Proceedings of the ASME 2005 International Design Engineering Technical Conferences and Computers and Information in Engineering Conference*. Long Beach, CA, USA: ASME, 2005: 257-269. DOI: 10.1115/DETC2005-85597.
- [7] EYMARD R, GALLOUËT T, HERBIN R. Finite volume methods[M]// CIARLET P G, LIONS J L. *Handbook of Numerical Analysis: Vol. 7*. Amsterdam: Elsevier, 2000: 713-1018. DOI: 10.1016/S1570-8659(00)07005-8.

附录

附录 A 支撑材料文件列表

文件名	文件内容
AI 工具使用详情.pdf	本文在竞赛过程中使用 AI 工具的详细说明。
README.md	提交包边界、环境配置、复现命令、目录结构和清理说明。
environment.yml	Conda 环境定义及 Python 依赖版本约束。
run_all.py	四问一体化正式复现入口，支持快速冒烟测试和正式网格计算。

code/artifact_names.py	统一定义各问题结果、工作簿、图像和提交包级产物的文件名。
code/build_workbooks.py	根据各问题 JSON 结果载荷构建 result1.xlsx 至 result4.xlsx。
code/kirchhoff_flux.py	计算非线性含水率扩散中的 Kirchhoff 面平均通量及导数。
code/q1/solve_q1.py	第一问求解器：边界拟合、守恒型径向有限体积法、敏感性分析和图像生成。

<code>code/q1/export_result1.py</code>	将第一问结果载荷导出为题目规定格式的 re-sult1.xlsx 。
<code>code/q2/boundary_stage.py</code>	第二问使用的边界拟合、稳定阶段检测和分阶段边界构造工具。
<code>code/q2/solve_q2.py</code>	第二问耦合径向传热传质模型求解器。
<code>code/q2/run_sensitivity.py</code>	重新计算第二问参数敏感性诊断结果。
<code>code/q2/export_result2.py</code>	将第二问结果数组导出为题目提供格式的 re-sult2.xlsx 。

<code>code/q2/verify_result2.py</code>	独立核验第二问工作簿、结果数组和输入文件一致性。
<code>code/q3/boundary_stage.py</code>	第三问复用的边界拟合、稳定阶段检测和分阶段边界构造工具。
<code>code/q3/solve_q3.py</code>	第三问定位最大含水率达到 0.15 的时间，并生成 60 秒间隔结果。
<code>code/q3/verify_result3.py</code>	独立核验第三问结果工作簿、终点状态和输入依赖。

<code>code/q4/boundary_stage.py</code>	第四问复用的边界拟合、稳定阶段检测和分阶段边界构造工具。
<code>code/q4/solve_q4.py</code>	第四问材料坐标下变半径传热传质模型求解器。
<code>code/q4/build_workbook.py</code>	将第四问结果载荷写入题目规定格式的 result4.xlsx 。
<code>code/q4/verify_result4.py</code>	独立核验第四问工作簿、空白掩码、终点状态和输入文件。

<code>code/p6/run_p6.py</code>	生成论文第六节的收敛性、参数敏感性和边界系统误差分析表。
<code>code/p6/verify_p6.py</code>	核验第六节分析表格及其来源链接。
<code>code/verify_all.py</code>	对整个提交包执行输入、结果、工作簿、图像和目录结构的总核验。
<code>data/附件 1.xlsx</code>	题目附件 1：烘房温度和环境水分浓度观测数据。
<code>data/附件 2.xlsx</code>	题目附件 2：药材半径观测数据。

data/result1_template.xlsx	题目提供的第一问结果工作簿模板。
data/result2_template.xlsx	题目提供的第二问结果工作簿模板。
data/result3_template.xlsx	题目提供的第三问结果工作簿模板。
data/result4_template.xlsx	题目提供的第四问结果工作簿模板。
results/result1.xlsx	第一问完整结果工作簿。
results/result2.xlsx	第二问完整结果工作簿。
results/result3.xlsx	第三问完整结果工作簿。
results/result4.xlsx	第四问完整结果工作簿。

results/reproducibility_check.json	一体化复现记录，包含输入、源代码、结果和图像哈希及目录核验信息。
results/q1/q1_result_data.json	第一问结构化结果数组、网格和模型元数据。
results/q1/q1_result_full_precision.npz	第一问全精度数值数组。
results/q1/q1_validation.json	第一问守恒、时间步、空间网格和敏感性核验记录。
results/q1/q1_boundary_model_cv.csv	第一问候选边界模型交叉验证比较。
results/q1/q1_boundary_endpoint_comparison.csv	第一问边界拟合终点比较。

results/q2/q2_result_data.json	第二问结构化结果数组、网格和边界元数据。
results/q2/q2_result_full_precision.npz	第二问全精度数值数组。
results/q2/q2_validation.json	第二问守恒、时间步和空间网格核验记录。
results/q2/q2_validation_export.json	第二问导出工作簿的数值一致性核验记录。
results/q2/q2_parameter_sensitivity.json	第二问参数敏感性诊断及源代码哈希记录。
results/q3/q3_result_data.json	第三问结构化结果数组、达标时间和边界元数据。

results/q3/q3_result_full_precision.npz	第三问全精度数值数组。
results/q3/q3_validation.json	第三问守恒、时间步、网格和终点核验记录。
results/q3/q3_validation_export.json	第三问导出工作簿的数值一致性核验记录。
results/q4/q4_result_data.json	第四问结构化结果数组、半径曲线和模型元数据。
results/q4/q4_result_full_precision.npz	第四问全精度数值数组。
results/q4/q4_validation.json	第四问守恒、时间步、网格、掩码和终点核验记录。

results/q4/q4_validation_export.json	第四问导出工作簿的数值与空白掩码核验记录。
results/q4/q4_radius_method_comparison.csv	第四问半径边界不同表示方法的比较结果。
results/p6/p6_convergence.csv	第六节数值收敛性分析表。
results/p6/p6_sensitivity.csv	第六节参数敏感性分析表。
results/p6/p6_systematic_boundary.csv	第六节边界扰动系统误差分析表。
results/p6/p6_summary.json	第六节分析汇总及表格行数、样本数记录。
figures/q1/q1_fig_01_boundary_raw_scatter.png	第一问环境边界原始观测散点图。

figures/q1/q1_fig_02_boundary_stretched_exp_fit.png	第一问拉伸指数边界拟合图。
figures/q1/q1_fig_03_temperature_moisture_field.png	第一问温度场和含水率场图。
figures/q2/q2_fig_01_boundary_stability_detection.png	第二问温度、含水率稳定阶段检测图。
figures/q2/q2_fig_02_staged_boundary_fit.png	第二问分阶段边界拟合图。
figures/q2/q2_fig_03_parameter_sensitivity.png	第二问参数敏感性图。
figures/q2/q2_fig_04_temperature_moisture_field.png	第二问温度场和含水率场图。

figures/q3/q3_fig_01_drying_result.png	第三问干燥达标结果图。
figures/q4/q4_fig_01_radius_raw_scatter.png	第四问半径原始观测散点图。
figures/q4/q4_fig_02_radius_pchip_fit.png	第四问 PCHIP 半径拟合图。
figures/q4/q4_fig_03_shrinkage_moisture_field.png	第四问收缩过程含水率场图。

附录 B 程序代码

附录 B1: run_all.py
一个复现本文 A 题完整计算流程的入口程序
<pre> """从一个入口复现 A 题的完整计算流程。 默认模式使用论文中报告的网格和核验项。``--quick`` 保持相同的方程和输出格式， 但使用更小的网格执行隔离式冒烟测试。数值产物保存在 ``results/``, 论文图像写入 ``figures/``。 """ from __future__ import annotations import argparse import os import shutil import subprocess import sys import tempfile </pre>

```

from pathlib import Path

PACKAGE = Path(__file__).resolve().parent
CODE = PACKAGE / "code"
RESULTS = PACKAGE / "results"
FIGURES = PACKAGE / "figures"
if str(CODE) not in sys.path:
    sys.path.insert(0, str(CODE))
from artifact_names import FIGURE_NAMES, figure_name

# 提交包四个问题共用的边界窗口约定。
# 第一问是早期时段的辨识问题。第二至第四问使用附件 1 的完整记录，分别检测稳定阶段
# 并估计尾段均值；每条拉伸指数分支只拟合到各自的过渡点，再接入 1800 s 过渡段和
# 恒定尾段。
Q1_FIT_ENDPOINT_S = 1800.0

def run(script: Path, *args: str) -> None:
    command = [sys.executable, str(script), *args]
    print("+", " ".join(command), flush=True)
    env = os.environ.copy()
    env.setdefault("PYTHONDONTWRITEBYTECODE", "1")
    subprocess.run(command, cwd=PACKAGE, check=True, env=env)

def run_isolated_quick() -> None:
    """在临时副本中执行冒烟测试，以保持正式结果不变。"""
    with tempfile.TemporaryDirectory(prefix="solo_math_modeling_quick_") as temp:
        quick_package = Path(temp) / PACKAGE.name
        shutil.copytree(
            PACKAGE,
            quick_package,
            ignore=shutil.ignore_patterns("__pycache__"),
        )
        command = [sys.executable, str(quick_package / "run_all.py"),
                    "--quick", "--isolated-quick"]
        print("+", " ".join(command), flush=True)
        env = os.environ.copy()
        env.setdefault("PYTHONDONTWRITEBYTECODE", "1")
        subprocess.run(command, cwd=quick_package, check=True, env=env)
    print("临时副本中的快速冒烟测试通过；正式结果未被修改。",
          flush=True)

def check_main_figures() -> None:

```

```

missing = []
for question, figure_keys in FIGURE_NAMES.items():
    figure_folder = FIGURES / question
    figure_folder.mkdir(parents=True, exist_ok=True)
    for key in figure_keys:
        name = figure_name(question, key)
        figure_path = figure_folder / name
        if not figure_path.exists():
            missing.append(str(figure_path))
if missing:
    raise FileNotFoundError(f"Missing generated figures: {missing}")

def main() -> None:
    parser = argparse.ArgumentParser()
    parser.add_argument("--quick", action="store_true", help="使用较小网格运行冒烟测试")
    parser.add_argument("--_isolated-quick", action="store_true", help=argparse.SUPPRESS)
    args = parser.parse_args()

    if args.quick and not args._isolated_quick:
        run_isolated_quick()
        return

    if args.quick:
        q1_grids, q2_grids, q3_grids, q4_grids = [200, 400], [100, 200], [100, 200], [100, 200]
        common = []
        q1_extra = ["--skip-sensitivity"]
        q3_extra = []
        q4_extra = []
    else:
        # 这些是论文核验记录所使用的正式网格。
        q1_grids, q2_grids = [800, 1600, 3200, 6400], [200, 300, 400]
        q3_grids, q4_grids = [220, 300], [200, 300]
        common = ["--check-time"]
        q1_extra = ["--sensitivity-grid", "800"]
        q3_extra = ["--sensitivity"]
        q4_extra = ["--comparisons"]

    run(CODE / "q1" / "solve_q1.py", "--grids", *map(str, q1_grids),
        "--fit-window-s", str(int(Q1_FIT_ENDPOINT_S)), *common, *q1_extra)
    run(CODE / "q1" / "export_result1.py")
    run(CODE / "q2" / "solve_q2.py", "--grids", *map(str, q2_grids),
        "--boundary-mode", "staged", *common)
    if args.quick:
        run(CODE / "q2" / "run_sensitivity.py", "--grid", "100")
    else:

```

```

run(CODE / "q2" / "run_sensitivity.py", "--grid", "200")
run(CODE / "q2" / "export_result2.py")
run(CODE / "q3" / "solve_q3.py", "--grids", *map(str, q3_grids),
    "--boundary-mode", "staged", *common, *q3_extra)
run(CODE / "q4" / "solve_q4.py", "--grids", *map(str, q4_grids),
    "--boundary-mode", "staged", *common, *q4_extra)
run(CODE / "build_workbooks.py")
check_main_figures()
run(CODE / "q2" / "verify_result2.py")
run(CODE / "q3" / "verify_result3.py")
run(CODE / "q4" / "verify_result4.py")
run(CODE / "verify_all.py")
print("Integrated reproduction completed under", RESULTS)

```

```

if __name__ == "__main__":
    main()

```

附录 B2: code/q1/solve_q1.py

第一问核心求解代码

"""第一问：守恒型径向有限体积法与隐式 BDF 积分。

从当前目录运行：python solve_q1.py

依赖：numpy、scipy、openpyxl（只读）和 matplotlib。

result1.xlsx 由独立导出器生成；本求解器在 results 目录重新生成数值载荷和验证记录。

"""

```
from pathlib import Path
```

```
import argparse
```

```
import json
```

```
import time
```

```
import zipfile
```

```
import xml.etree.ElementTree as ET
```

```
import sys
```

```
import numpy as np
```

```
from scipy.integrate import solve_ivp
```

```
from scipy.optimize import least_squares
```

```
from scipy.sparse import diags, bmat, csr_matrix
```

```
try:
```

```
    import openpyxl
```

```
except ImportError: # 以下标准库读取器用于保持本求解器的可复现性。
```

```
    openpyxl = None
```

```
HERE = Path(__file__).resolve().parent
```

```
PACKAGE = HERE.parent.parent
```

```
if str(HERE.parent) not in sys.path:
```

```
    sys.path.insert(0, str(HERE.parent))
```

```

from artifact_names import artifact_name, figure_name
from kirchhoff_flux import scalar_face
OUT = PACKAGE / 'results' / 'q1'
INPUT = PACKAGE / 'data' / '附件 1.xlsx'
FIGURE_OUT = PACKAGE / 'figures' / 'q1'
OUT.mkdir(parents=True, exist_ok=True)
FIGURE_OUT.mkdir(parents=True, exist_ok=True)
R, RHO, CP, K, H, HM = 0.02, 820.0, 2600.0, 0.36, 25.0, 8e-7
ALPHA = K / (RHO * CP)
TIMES = np.arange(1801, dtype=float)
REPORT_TIMES = np.array([100, 300, 600, 900, 1200, 1500, 1800])
# 工作簿保留题目要求的 0.1 cm 径向输出；更密的径向采样另行保留，用于连续
# 颜色场图像。
PLOT_RADIAL_POINTS = 201
# 第一问只使用附件 1 的前 1800 s 数据拟合边界；第二问及以后由各自的
# boundary_stage.py 使用完整附件 1 数据拟合，再在稳定分界点前调用该拟合曲线。
DEFAULT_FIT_WINDOW_S = 1800.0

def read_environment():
    """以只读方式读取附件 1，不要求可写的电子表格程序。"""
    path = INPUT
    if openpyxl is not None:
        wb = openpyxl.load_workbook(path, data_only=True, read_only=True)
        records = list(wb.active.values)
        wb.close()
        data = np.array(records[1:], dtype=float)
    else:
        # 提供的第一个工作表包含一行表头和三列数值数据。
        namespace = {'x': 'http://schemas.openxmlformats.org/spread-
sheetml/2006/main'}
        with zipfile.ZipFile(path) as archive:
            root = ET.fromstring(archive.read('xl/worksheets/sheet1.xml'))
            records = []
            for row in root.findall('.//x:sheetData/x:row', namespace):
                values = []
                for cell in row.findall('x:c', namespace):
                    value = cell.find('x:v', namespace)
                    if value is not None:
                        values.append(float(value.text))
                if len(values) == 3:
                    records.append(values)
            data = np.array(records[1:], dtype=float)
        assert data.shape[1] == 3 and np.isfinite(data).all()
        assert np.all(np.diff(data[:, 0]) > 0)
        assert data[0, 0] == 0 and data[-1, 0] >= 1800
        return data

```

```

def stretched_exponential(t_s, y0, amplitude, tau_h, exponent):
    """单调输入平滑器，接口处的时间单位为秒。"""
    tau_s = 3600.0 * tau_h
    return y0 + amplitude * (1.0 - np.exp(-(t_s / tau_s) ** exponent))

def fit_environment(environment, fit_window_s):
    """在指定数据窗口内拟合两个环境边界，并返回连续输入。"""
    if not (1800.0 <= fit_window_s <= environment[-1, 0]):
        raise ValueError('fit_window_s must lie between 1800 s and the last attachment
record.')
    fit_mask = environment[:, 0] <= fit_window_s
    q1_mask = environment[:, 0] <= 1800.0
    smooth = environment.copy()
    diagnostics = {'method_name': 'stretched_exp',
                    'family': '初值固定的拉伸指数模型',
                    'fit_window_s': float(fit_window_s), 'series': {}}
    for column, name, scale in [(1, 'T_infty_C', 22.0), (2, 'C_infty_kg_per_kg', 0.03)]:
        t_fit = environment[fit_mask, 0]
        y_fit = environment[fit_mask, column]
        y0 = float(y_fit[0])

        def residual(x):
            return stretched_exponential(t_fit, y0, *x) - y_fit

        solution = least_squares(
            residual, x0=[scale, 0.7, 1.1],
            bounds=([1e-12, 1e-5, 0.1], [100.0, 100.0, 5.0]),
            xtol=1e-13, ftol=1e-13, gtol=1e-13, max_nfev=100000,
        )
        if not solution.success:
            raise RuntimeError(f'Boundary fit failed for {name}: {solution.message}')
        amplitude, tau_h, exponent = map(float, solution.x)
        fitted_all = stretched_exponential(environment[:, 0], y0, *solution.x)
        fitted_window = fitted_all[fit_mask]
        fitted_q1 = fitted_all[q1_mask]
        residual_window = fitted_window - y_fit
        residual_q1 = fitted_q1 - environment[q1_mask, column]
        smooth[:, column] = fitted_all
        diagnostics['series'][name] = {
            'initial_value': y0, 'amplitude': amplitude, 'tau_h': tau_h,
            'exponent': exponent,
            'fit_rmse': float(np.sqrt(np.mean(residual_window ** 2))),
            'fit_max_abs_error': float(np.max(np.abs(residual_window))),
            'q1_rmse': float(np.sqrt(np.mean(residual_q1 ** 2))),
            'q1_max_abs_error': float(np.max(np.abs(residual_q1))),
        }

```

```

        'value_at_1800_s': float(fitted_all[np.flatnonzero(environment[:, 0] ==
1800.0)[0]]),
    }
    return smooth, diagnostics

```

```

def make_fitted_boundary(diagnostics):

```

```

    """返回解析形式的拟合边界，而不是按节点插值得到的边界。"""

```

```

    series_for_column = {1: diagnostics['series']['T_infty_C'],
                          2: diagnostics['series']['C_infty_kg_per_kg']}

```

```

    def boundary(t_s, column):

```

```

        item = series_for_column[column]
        return stretched_exponential(t_s, item['initial_value'], item['amplitude'],
                                     item['tau_h'], item['exponent'])

```

```

    return boundary

```

```

def solve_field(n, field, environment, rtol=2e-10, atol=2e-12, max_step=5.0,
               h_factor=1.0, hm_factor=1.0, d_factor=1.0, boundary=None):

```

```

    """N 个区间、包含 r=0 和 r=R 的 N+1 个节点。

```

最后一项状态用于对边界交换进行面积平均，以检查守恒。

含水率界面采用 Kirchhoff 通量；所有界面通量会望远镜式相消。

```

    """

```

```

    assert n % 20 == 0

```

```

    r = np.linspace(0, R, n + 1)

```

```

    dr = R / n

```

```

    faces = (r[:-1] + r[1:]) / 2

```

```

    edges = np.r_[0.0, faces, R]

```

```

    volumes = np.diff(edges**2) / 2 # 体积 / (2*pi*长度)

```

```

    area = R**2 / 2

```

```

    factor = faces / dr

```

```

    initial, beta, column = ((28.0, h_factor * H / (RHO * CP), 1)

```

```

                           if field == 'T' else (2.55, hm_factor * HM, 2))

```

```

    boundary_jac = csr_matrix((-R * beta / area, ([0], [n])), shape=(1, n + 1))

```

```

    def coefficients(u):

```

```

        if field == 'T':

```

```

            return np.full_like(u, ALPHA), np.zeros_like(u)

```

```

        if np.min(u) <= 0:

```

```

            raise ValueError('Nonpositive moisture encountered; no clipping is ap-
plied.')

```

```

            diffusivity = d_factor * 7e-9 * np.exp(-0.89 / u)

```

```

            return diffusivity, diffusivity * 0.89 / u**2

```

```

    def rhs(t, state):

```

```

    u = state[:-1]
    flux = np.empty(n + 2)
    flux[0] = 0.0
    if field == 'T':
        face = np.full(n, ALPHA)
    else:
        face, _, _ = scalar_face(u[:-1], u[1:], coefficients)
    flux[1:-1] = factor * face * np.diff(u)
    ambient = boundary(t, column) if boundary is not None else np.interp(
        t, environment[:, 0], environment[:, column])
    flux[-1] = R * beta * (ambient - u[-1])
    return np.r_[np.diff(flux) / volumes, flux[-1] / area]

def jac(t, state):
    u = state[:-1]
    delta = np.diff(u)
    if field == 'T':
        df = np.full(n, ALPHA)
        df_left = df_right = np.zeros(n)
    else:
        df, df_left, df_right = scalar_face(u[:-1], u[1:], coefficients)
    left = factor * (df_left * delta - df)
    right = factor * (df_right * delta + df)
    main = np.r_[left, -R * beta] - np.r_[0.0, right]
    matrix = diags((-left / volumes[1:], main / volumes, right / volumes[:-1]),
        offsets=(-1, 0, 1), format='csr')
    return bmat([[matrix, None], [boundary_jac, csr_matrix((1, 1))]], format='csc')

state = np.r_[np.full(n + 1, initial), 0.0]
sampled = np.empty((1801, 21))
sampled[0] = initial
plot_count = min(PLOT_RADIAL_POINTS, n + 1)
plot_indices = np.linspace(0, n, plot_count).round().astype(int)
plot_values = np.empty((1801, plot_count))
plot_values[0] = initial
averages = np.empty(1801)
averages[0] = initial
snapshots = {}
indices = np.arange(21) * (n // 20)
max_balance_error, evaluations = 0.0, 0
breaks = (np.array([0.0, 1800.0]) if boundary is not None else
    np.unique(np.r_[0, environment[(environment[:, 0] > 0) &
        (environment[:, 0] < 1800)], 0],
1800]))
for start, stop in zip(breaks[:-1], breaks[1:]):
    evaluation_times = TIMES[(TIMES > start) & (TIMES <= stop)]
    sol = solve_ivp(rhs, (start, stop), state, method='BDF', t_eval=evaluation_times,
        rtol=rtol, atol=atol, max_step=max_step, jac=jac)

```

```

        if not sol.success:
            raise RuntimeError(sol.message)
        evaluations += sol.nfev
        # 附件节点的时间为整数秒，因此无需对终止状态插值。
        assert evaluation_times[-1] == stop
        state = sol.y[:, -1]
        rows = evaluation_times.astype(int)
        sampled[rows] = sol.y[indices].T
        plot_values[rows] = sol.y[plot_indices].T
        mean = volumes @ sol.y[:, -1] / area
        averages[rows] = mean
        max_balance_error = max(max_balance_error,
                                float(np.max(np.abs(mean - initial - sol.y[-1]))))
        for t in REPORT_TIMES[(REPORT_TIMES > start) & (REPORT_TIMES <=
stop)]:
            snapshots[int(t)] = sol.y[:, -1, np.flatnonzero(rows == t)[0]].copy()
            assert np.isfinite(sampled).all()
            return dict(values=sampled, r=r, plot_r=r[plot_indices], plot_values=plot_values,
                        profiles=np.array([snapshots[int(t)] for t in REPORT_TIMES]),
                        mean=averages, balance_error=max_balance_error, nfev=evalua-
tions)

def endpoint_metrics(result_t, result_c):
    """用于输入/参数敏感性表的统一标量结果。"""
    return {
        'T_center_1800_C': float(result_t['values'][-1, 0]),
        'T_surface_1800_C': float(result_t['values'][-1, -1]),
        'T_mean_1800_C': float(result_t['mean'][-1]),
        'C_center_1800_kg_per_kg': float(result_c['values'][-1, 0]),
        'C_surface_1800_kg_per_kg': float(result_c['values'][-1, -1]),
        'C_mean_1800_kg_per_kg': float(result_c['mean'][-1]),
    }

def metric_difference(candidate, reference):
    return {name: float(candidate[name] - reference[name]) for name in reference}

def run_sensitivity(n, raw_environment, fit_window_s, reference_t=None, refer-
ence_c=None):
    """一次只改变一个参数的情景分析；这是模型形式/参数敏感性，不是求解器
误差。"""
    primary_environment, primary_fit = fit_environment(raw_environment, fit_win-
dow_s)
    primary_boundary = make_fitted_boundary(primary_fit)
    if reference_t is None or reference_c is None:

```

```

        reference_t = solve_field(n, 'T', primary_environment, boundary=primary_boundary)
        reference_c = solve_field(n, 'C', primary_environment, boundary=primary_boundary)
        reference = endpoint_metrics(reference_t, reference_c)
        scenarios = []

    def add(name, description, result_t, result_c):
        metrics = endpoint_metrics(result_t, result_c)
        scenarios.append({'name': name, 'description': description, 'metrics': metrics,
                          'delta_from_reference': metric_difference(metrics, reference)})

    linear_t = solve_field(n, 'T', raw_environment)
    linear_c = solve_field(n, 'C', raw_environment)
    add('linear_input', '附件 1 的 60 s 观测节点，采用分段线性插值。',
        linear_t, linear_c)

    for window_s in (3600.0, 7200.0):
        scenario_environment, scenario_fit = fit_environment(raw_environment, window_s)
        scenario_boundary = make_fitted_boundary(scenario_fit)
        scenario_t = solve_field(n, 'T', scenario_environment, boundary=scenario_boundary)
        scenario_c = solve_field(n, 'C', scenario_environment, boundary=scenario_boundary)
        add(f'fit_window_{int(window_s)}s',
            f'初值固定的拉伸指数模型，使用附件 1 的 0 - {int(window_s)} s 数据。',
            scenario_t, scenario_c)

    for factor in (0.8, 1.2):
        scenario_t = solve_field(n, 'T', primary_environment, h_factor=factor,
                                boundary=primary_boundary)
        add(f'h_factor_{factor:.1f}', f'换热系数 h 乘以 {factor:.1f}。',
            scenario_t, reference_c)
    for factor in (0.8, 1.2):
        scenario_c = solve_field(n, 'C', primary_environment, hm_factor=factor,
                                boundary=primary_boundary)
        add(f'hm_factor_{factor:.1f}', f'有效传质系数 h_m 乘以 {factor:.1f}。',
            reference_t, scenario_c)
    for factor in (0.8, 1.2):
        scenario_c = solve_field(n, 'C', primary_environment, d_factor=factor,
                                boundary=primary_boundary)
        add(f'D_factor_{factor:.1f}', f'内部水分扩散系数 D(C) 乘以 {factor:.1f}。',
            reference_t, scenario_c)

    return {'reference': {'description': f'0-{int(fit_window_s)} s nonlinear boundary fit',

```

```

'grid_intervals': int(n), 'metrics': reference,
'boundary_fit': primary_fit}, 'scenarios': scenarios}

```

```

def make_boundary_figures(raw_environment, boundary_info):

```

```

    """保存第 5.1.1 节使用的两幅边界图。

```

第一幅图只展示完整的 0--14400 s 原始数据记录；第二幅图限制在第一问的拟合窗口内，只绘制选定的拉伸指数曲线。将两幅图分开可以避免把模型选择过程与后续物理场求解混在一起。

```

    """

```

```

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

```

```

plt.rcParams.update({'font.sans-serif': ['Microsoft YaHei', 'SimHei', 'DejaVu Sans'],
                    'axes.unicode_minus': False, 'font.size': 10})

```

```

t_s = np.asarray(raw_environment[:, 0], dtype=float)

```

```

labels = [('烘房温度', '温度 / ° C', 1, '#9b1c1f'),
          ('环境水分浓度', '水分浓度 / (kg/kg)', 2, '#2166ac')]

```

图 1 不叠加拟合曲线，只如实展示题目提供的 60 s 观测和后续平台阶段。

```

fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.5), constrained_layout=True)

```

```

for ax, (title, ylabel, column, color) in zip(axes, labels):

```

```

    ax.scatter(t_s, raw_environment[:, column], s=14,
              color=color, alpha=0.78, linewidths=0, label='附件 1 观测散点')

```

```

    ax.set(title=f'{title}: 0 - 14400 s 原始数据', xlabel='时间 / s', ylabel=ylabel,
          xlim=(0.0, float(t_s[-1])))

```

```

    ax.grid(alpha=0.22)

```

```

    ax.legend(fontsize=8, loc='best')

```

```

fig.savefig(FIGURE_OUT / figure_name('q1', 'boundary_raw'), dpi=220)

```

```

plt.close(fig)

```

图 2 只展示选定的拟合曲线，使用与第一问识别过程相同的 0--1800 s 观测。

此处不绘制其他候选曲线。

```

if not boundary_info.get('series'):

```

```

    return

```

```

fit_mask = t_s <= DEFAULT_FIT_WINDOW_S + 1e-9

```

```

t_fit = t_s[fit_mask]

```

```

grid = np.linspace(0.0, DEFAULT_FIT_WINDOW_S, 500)

```

```

fig, axes = plt.subplots(1, 2, figsize=(12.5, 4.5), constrained_layout=True)

```

```

for ax, (title, ylabel, column, _color), name in zip(

```

```

    axes, labels, ('T_infty_C', 'C_infty_kg_per_kg')):

```

```

    item = boundary_info['series'][name]

```

```

    y_fit = raw_environment[fit_mask, column]

```

```

    y_curve = stretched_exponential(grid, item['initial_value'], item['amplitude'],

```

```

        item['tau_h'], item['exponent'])
    ax.scatter(t_fit, y_fit, s=16, color='#444444', alpha=0.82,
               linewidths=0, label='0 - 1800 s 观测散点')
    ax.plot(grid, y_curve, color='#c51b8a', lw=2.0,
            label='初值固定的拉伸指数曲线')
    rmse = item['q1_rmse']
    rmse_text = f'RMSE={rmse:.6g}' if column == 1 else f'RMSE={rmse:.3e}'
    ax.set(title=f'{title}: 拉伸指数拟合 (0 - 1800 s)', xlabel='时间 / s', yla-
bel=ylabel,
          xlim=(0.0, DEFAULT_FIT_WINDOW_S))
    ax.text(0.03, 0.95, rmse_text, transform=ax.transAxes, va='top',
           bbox={'facecolor': 'white', 'alpha': 0.8, 'edgecolor': '0.7'})
    ax.grid(alpha=0.22)
    ax.legend(fontsize=8, loc='best')
    fig.savefig(FIGURE_OUT / figure_name('q1', 'boundary_fit'), dpi=220)
    plt.close(fig)

def make_figures(result_t, result_c, env):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    from matplotlib.colors import LinearSegmentedColormap, PowerNorm

    plt.rcParams.update({'font.sans-serif': ['Microsoft YaHei', 'SimHei', 'DejaVu Sans'],
                        'axes.unicode_minus': False, 'font.size': 10})
    # 工作簿存储 21 个半径位置 (0--2 cm, 间隔 0.1 cm), 图像则使用
solve_field
    # 保留的更密采样。
    time_min = TIMES / 60.0
    temperature_values = result_t.get('plot_values', result_t['values'])
    moisture_values = result_c.get('plot_values', result_c['values'])
    plot_r = result_t.get('plot_r')
    if plot_r is None:
        plot_r = np.linspace(0.0, R, temperature_values.shape[1])
    r_cm = np.asarray(plot_r) * 100.0
    temperature = temperature_values.T
    moisture = moisture_values.T

    temperature_cmap = LinearSegmentedColormap.from_list(
        'temperature_blue_red',
        ['#08306b', '#2171b5', '#f7f7f7', '#cb181d', '#67000d'])
    moisture_cmap = LinearSegmentedColormap.from_list(
        'moisture_gray_blue',
        ['#f0f0f0', '#bdbdbd', '#9ecae1', '#3182bd', '#08519c'])

    fig, axes = plt.subplots(1, 2, figsize=(13, 5.6), constrained_layout=True)
    temp_mesh = axes[0].pcolormesh(

```

```

        time_min, r_cm, temperature, shading='nearest', cmap=temperature_cmap,
        vmin=float(np.min(temperature)), vmax=float(np.max(temperature)))
moisture_mesh = axes[1].pcolormesh(
    time_min, r_cm, moisture, shading='nearest', cmap=moisture_cmap,
    norm=PowerNorm(gamma=1.8, vmin=float(np.min(moisture)),
                    vmax=float(np.max(moisture))))

axes[0].set(title='温度场', xlabel='时间 / min', ylabel='距中心距离 / cm',
            ylim=(0, R * 100.0))
axes[1].set(title='水分浓度场', xlabel='时间 / min', ylabel='距中心距离 / cm',
            ylim=(0, R * 100.0))
for ax in axes:
    ax.set_xlim(float(time_min[0]), float(time_min[-1]))
    ax.set_yticks(np.arange(0, R * 100.0 + 0.01, 0.5))

temp_bar = fig.colorbar(temp_mesh, ax=axes[0], pad=0.02)
temp_bar.set_label('温度 / ° C')
moisture_bar = fig.colorbar(moisture_mesh, ax=axes[1], pad=0.02)
moisture_bar.set_label('水分浓度 / (kg/kg)')
fig.savefig(Figure_OUT / figure_name('q1', 'temperature_moisture_field'),
            dpi=180)
plt.close(fig)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--grids', type=int, nargs='+', default=[800, 1600, 3200, 6400])
    parser.add_argument('--check-time', action='store_true')
    parser.add_argument('--boundary-mode', choices=('fit', 'linear'), default='fit',
                        help='使用非线性边界拟合（默认），或回退到原始线性输入。')
    parser.add_argument('--fit-window-s', type=float, default=DEFAULT_FIT_WINDOW_S,
                        help='附件 1 拟合时间窗口；求解仍在 1800 s 处停止。')
    parser.add_argument('--sensitivity-grid', type=int, default=800,
                        help='一次只改变一个参数的敏感性情景所使用的径向区间数。')
    parser.add_argument('--skip-sensitivity', action='store_true')
    args = parser.parse_args()
    raw_env = read_environment()
    if args.boundary_mode == 'fit':
        env, boundary_info = fit_environment(raw_env, args.fit_window_s)
        boundary_info['mode'] = 'nonlinear_fit'
        boundary_evaluator = make_fitted_boundary(boundary_info)
    else:
        env = raw_env.copy()
        boundary_info = {'mode': 'piecewise_linear', 'fit_window_s': None,

```

```

'series': {}, 'family': 'Attachment-1 piecewise-linear inter-
polation'}
    boundary_evaluator = None
    # 第 5.1.1 节使用这两幅图分别展示原始数据检查、模型比较和最终选定的边
    # 界
    # 表示。
    make_boundary_figures(raw_env, boundary_info)
    previous = None
    convergence = []
    results_by_grid = {}
    for n in args.grids:
        start = time.perf_counter()
        rt = solve_field(n, 'T', env, boundary=boundary_evaluator)
        rc = solve_field(n, 'C', env, boundary=boundary_evaluator)
        record = {'intervals': n, 'dr_m': R / n,
                  'balance_T': rt['balance_error'], 'balance_C': rc['balance_error']}
        if previous is not None:
            for name, cur, prev in [('T', rt, previous[0]), ('C', rc, previous[1])]:
                record[f'max_diff_{name}'] = float(np.max(np.abs(cur['values'] -
prev['values'])))
                record[f'table_diff_{name}'] = float(np.max(np.abs(
cur['values'][REPORT_TIMES][:, ::5] - prev['values'][RE-
PORT_TIMES][:, ::5])))
            convergence.append(record)
            previous = rt, rc
            results_by_grid[n] = (rt, rc)
            print(json.dumps(record), f'elapsed={time.perf_counter() - start:.2f}s',
flush=True)

    time_check = {}
    if args.check_time:
        for name, baseline in [('T', rt), ('C', rc)]:
            tight = solve_field(args.grids[-1], name, env, rtol=2e-11, atol=2e-13,
max_step=2.0, boundary=boundary_evaluator)
            time_check[name] = float(np.max(np.abs(baseline['values'] - tight['val-
ues'])))
        print('Time convergence:', time_check, flush=True)
    assert np.min(rt['values']) >= 28 - 1e-8
    assert np.max(rt['values']) <= np.max(env[env[:, 0] <= 1800, 1]) + 1e-8
    assert np.min(rc['values']) > 0 and np.max(rc['values']) <= 2.55 + 1e-8
    assert np.min(np.diff(rt['values'], axis=1)) > -1e-7
    assert np.max(np.diff(rc['values'], axis=1)) < 1e-7

    sensitivity = None
    if not args.skip_sensitivity and args.boundary_mode == 'fit':
        reference_t, reference_c = results_by_grid.get(args.sensitivity_grid, (None,
None))
        sensitivity = run_sensitivity(args.sensitivity_grid, raw_env, args.fit_window_s,
reference_t, reference_c)

```

```

elif not args.skip_sensitivity:
    print('Sensitivity report is skipped for --boundary-mode linear; run the default
fit mode.',
        flush=True)

    payload = {'r_cm': np.round(np.linspace(0, 2, 21), 1).tolist(),
               't_s': TIMES[1:].astype(int).tolist(),
               'T': np.round(rt['values'][1:], 4).tolist(),
               'C': np.round(rc['values'][1:], 4).tolist()}
    (OUT / artifact_name('q1', 'result_data')).write_text(
        json.dumps(payload, ensure_ascii=False), encoding='utf-8')
    summary = {'model': '1D radial; Kirchhoff moisture flux; effective Robin moisture
boundary; no latent heat',
               'moisture_face_flux': '8-point Gauss-Legendre Kirchhoff average in
C',
               'alpha_m2_s': ALPHA, 'D_initial_m2_s': float(7e-9 * np.exp(-0.89 /
2.55)),
               'grid_intervals': args.grids[-1], 'rtol': 2e-10, 'atol': 2e-12,
               'max_step_s': 5, 'boundary': boundary_info,
               'selected_boundary_method': 'stretched_exp',
               'boundary_method_comparison': artifact_name('q1', 'boundary_cv'),
               'fit_endpoint_comparison': artifact_name('q1', 'boundary_end-
point_comparison'),
               'environment_1800': env[env[:, 0] == 1800][0].tolist(),
               'raw_environment_1800': raw_env[raw_env[:, 0] == 1800][0].tolist(),
               'convergence': convergence, 'time_check': time_check,
               'mean_T_1800': float(rt['mean'][-1]), 'mean_C_1800':
float(rc['mean'][-1]),
               'table_times': REPORT_TIMES.tolist(), 'table_r_cm': [0, 0.5, 1, 1.5,
2],
               'table_T': rt['values'][REPORT_TIMES][:, ::5].tolist(),
               'table_C': rc['values'][REPORT_TIMES][:, ::5].tolist()}
    if sensitivity is not None:
        summary['sensitivity'] = sensitivity
    (OUT / artifact_name('q1', 'validation')).write_text(
        json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
    np.savez_compressed(
        OUT / artifact_name('q1', 'full_precision'), t=TIMES, r_cm=payload['r_cm'],
        T=rt['values'], C=rc['values'], mean_T=rt['mean'], mean_C=rc['mean'])
    make_figures(rt, rc, env)
    print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)

if __name__ == '__main__':
    main()

```

附录 B3: code/q2/solve_q2.py

第二问核心求解代码

"""第二问：全程采用附录 3 的耦合径向传热/传质模型。

从当前目录运行：`python solve\_q2.py --check-time`。

输入：data/附件 1.xlsx；Excel 导出器：export_result2.py。

"""

```
from pathlib import Path
import argparse
import hashlib
import json
import sys
import time
```

```
import numpy as np
import openpyxl
from scipy.integrate import solve_ivp
from scipy.signal import savgol_filter
from scipy.sparse import diags, bmat, csr_matrix
from threadpoolctl import threadpool_limits
```

```
HERE = Path(__file__).resolve().parent
PACKAGE = HERE.parent.parent
ROOT = PACKAGE
OUT = PACKAGE / 'results' / 'q2'
DATA = PACKAGE / 'data'
FIGURE_OUT = PACKAGE / 'figures' / 'q2'
OUT.mkdir(parents=True, exist_ok=True)
FIGURE_OUT.mkdir(parents=True, exist_ok=True)
if str(HERE) not in sys.path:
    sys.path.insert(0, str(HERE))
if str(HERE.parent) not in sys.path:
    sys.path.insert(0, str(HERE.parent))
from artifact_names import artifact_name, figure_name
from kirchhoff_flux import moisture_face
from boundary_stage import build_boundaries
R, H, HM = 0.02, 25.0, 8e-7
T0, C0 = 28.0, 2.55
END_TIME = 10800
REPORT_TIMES = np.arange(1800, END_TIME + 1, 1800)
PLOT_RADIAL_POINTS = 201
```

```
def read_environment():
    path = DATA / '附件 1.xlsx'
    wb = openpyxl.load_workbook(path, data_only=True, read_only=True)
    rows = list(wb.active.values)
    wb.close()
    env = np.array(rows[1:], dtype=float)
    assert env.shape[1] == 3 and np.isfinite(env).all()
```

```

assert env[0, 0] == 0 and np.all(np.diff(env[:, 0]) > 0)
assert env[-1, 0] >= END_TIME
return env

def properties(T, C):
    """T 以摄氏度存储，只有 Arrhenius 因子使用开尔文温度。

    返回 b=rho*cp、k、D 及其偏导数。
    支持复数输入，以便独立进行复步长 Jacobian 核验。
    """
    kelvin = T + 273.15
    if np.min(np.real(C)) <= 0 or np.min(np.real(kelvin)) <= 0:
        raise ValueError('Nonphysical state; no clipping or silent replacement is applied.')
    rho = 650 + 128 * C
    cp = 1450 + 2736 * C / (C + 1)
    k = 0.21 + 0.38 * C / (C + 1)
    d = 2.4e-3 * np.exp(-0.45 / C - 3850 / kelvin)
    b = rho * cp
    b_c = 128 * cp + rho * 2736 / (C + 1)**2
    k_c = 0.38 / (C + 1)**2
    d_c = d * 0.45 / C**2
    d_t = d * 3850 / kelvin**2
    return b, k, d, b_c, k_c, d_c, d_t

class CoupledModel:
    def __init__(self, n, env, tail='hold_last', boundary=None):
        assert n % 20 == 0
        self.n, self.m, self.env, self.tail = n, n + 1, env, tail
        self.boundary = boundary
        self.r = np.linspace(0, R, n + 1)
        self.dr = R / n
        self.faces = (self.r[:-1] + self.r[1:]) / 2
        self.w = np.diff(np.r_[0.0, self.faces, R]**2) / 2
        self.area = R**2 / 2
        self.factor = self.faces / self.dr
        self.balance_c = csr_matrix((-R * HM / self.area, ([0], [n])), shape=(1, self.m))
        self.zero_row = csr_matrix((1, self.m))

    def ambient(self, t):
        if self.boundary is not None:
            return float(self.boundary[0](t)), float(self.boundary[1](t))
        if t > self.env[-1, 0] and self.tail != 'hold_last':
            raise ValueError('Environment data exhausted. Explicitly select a tail assumption.')

```

在给定时间范围之外，`np.interp` 保持最后一条观测值不变。
 # 这一行为由 `tail='hold_last'` 显式指定，且在第二问最初 3 h 的计算中不会触发。

```
return np.interp(t, self.env[:, 0], self.env[:, 1]), np.interp(t, self.env[:, 0],
self.env[:, 2])
```

```
def rhs(self, t, state):
```

```
    m = self.m
    T, C = state[:m], state[m:2*m]
    b, k, d, *_ = properties(T, C)
    ta, ca = self.ambient(t)
    ft = np.empty(m + 1, dtype=state.dtype)
    fc = np.empty_like(ft)
    ft[0] = fc[0] = 0
    ft[1:-1] = self.factor * (k[:-1] + k[1:]) / 2 * np.diff(T)
    dface, *_ = moisture_face(T[:-1], T[1:], C[:-1], C[1:], properties)
    fc[1:-1] = self.factor * dface * np.diff(C)
    ft[-1] = R * H * (ta - T[-1])
    fc[-1] = R * HM * (ca - C[-1])
    return np.r_[np.diff(ft) / (self.w * b), np.diff(fc) / self.w, fc[-1] / self.area]
```

```
def flux_jacobian(self, left, right, boundary, denominator):
```

```
    diagonal = np.r_[left, boundary] - np.r_[0.0, right]
    return diags((-left / denominator[1:], diagonal / denominator,
                 right / denominator[:-1]), (-1, 0, 1), format='csr')
```

```
def jac(self, t, state):
```

```
    m = self.m
    T, C = state[:m], state[m:2*m]
    b, k, d, bc, kc, dc, dt = properties(T, C)
    delta_t, delta_c = np.diff(T), np.diff(C)
    kface = (k[:-1] + k[1:]) / 2
    dface, dcl, dcr, dtl, dtr = moisture_face(
        T[:-1], T[1:], C[:-1], C[1:], properties)
    factor = self.factor
    jtt = self.flux_jacobian(-factor*kface, factor*kface, -R*H, self.w*b)
    jtc = self.flux_jacobian(factor*0.5*kc[:-1]*delta_t,
                             factor*0.5*kc[1:]*delta_t, 0.0, self.w*b)
    jtc -= diags(self.rhs(t, state)[:m] * bc / b, format='csr')
    jcc = self.flux_jacobian(factor*(dcl*delta_c-dface),
                             factor*(dcr*delta_c+dface), -R*HM, self.w)
    jct = self.flux_jacobian(factor*dtl*delta_c,
                             factor*dtr*delta_c, 0.0, self.w)
    return bmat([[jtt, jtc, None], [jct, jcc, None],
                 [self.zero_row, self.balance_c, csr_matrix((1, 1))]], for-
mat='csc')
```

```
def check_jacobian(env):
```

```

model = CoupledModel(40, env)
fraction = model.r / R
state = np.r_[28 + 7*fraction**2, 2.55 - 0.8*fraction**2, 0.0]
analytic = model.jac(1200, state)
rng = np.random.default_rng(2026)
errors = []
for _ in range(3):
    direction = rng.normal(size=len(state))
    expected = np.imag(model.rhs(1200, state.astype(complex) + 1e-25j*direction)) / 1e-25
    actual = analytic @ direction
    errors.append(float(np.max(np.abs(actual-expected)) / max(np.max(np.abs(expected)), 1e-30)))
assert max(errors) < 1e-11, errors
return max(errors)

def solve_model(n, env, rtol=2e-10, atol=2e-12, max_step=5.0, end_time=END_TIME,
                boundary=None):
    model = CoupledModel(n, env, boundary=boundary)
    m, w, area = model.m, model.w, model.area
    times = np.arange(end_time + 1, dtype=float)
    indices = np.arange(21) * (n // 20)
    plot_count = min(PLOT_RADIAL_POINTS, m)
    plot_indices = np.linspace(0, n, plot_count).round().astype(int)
    state = np.r_[np.full(m, T0), np.full(m, C0), 0.0]
    temperatures = np.empty((len(times), 21))
    moistures = np.empty_like(temperatures)
    temperatures[0], moistures[0] = T0, C0
    plot_temperatures = np.empty((len(times), plot_count))
    plot_moistures = np.empty_like(plot_temperatures)
    plot_temperatures[0], plot_moistures[0] = T0, C0
    mean_t, mean_c = np.empty(len(times)), np.empty(len(times))
    mean_t[0], mean_c[0] = T0, C0
    profile_t, profile_c = {}, {}
    balance_error, heat_rate_error = 0.0, 0.0
    nfev, nlu = 0, 0
    breaks = np.unique(np.r_[0, env[(env[:, 0] > 0) & (env[:, 0] < end_time), 0],
end_time])
    for start, stop in zip(breaks[:-1], breaks[1:]):
        evaluation_times = times[(times > start) & (times <= stop)]
        assert evaluation_times[-1] == stop, 'Environment knots must be integer seconds.'
        sol = solve_ivp(model.rhs, (start, stop), state, method='BDF', jac=model.jac,
                        t_eval=evaluation_times, rtol=rtol, atol=atol,
max_step=max_step,
                        first_step=min(0.001, stop-start))
        if not sol.success:
            raise RuntimeError(sol.message)

```

```

state = sol.y[:, -1]
nfev += sol.nfev
nlu += sol.nlu
rows = evaluation_times.astype(int)
temperatures[rows] = sol.y[indices].T
moistures[rows] = sol.y[m+indices].T
plot_temperatures[rows] = sol.y[plot_indices].T
plot_moistures[rows] = sol.y[m+plot_indices].T
avg_t = np.sum(w[:, None] * sol.y[:m], axis=0) / area
avg_c = np.sum(w[:, None] * sol.y[m:2*m], axis=0) / area
mean_t[rows], mean_c[rows] = avg_t, avg_c
balance_error = max(balance_error, float(np.max(np.abs(avg_c-C0-sol.y[-
1]))))
derivative = model.rhs(stop, state)
capacity = properties(state[:m], state[m:2*m])[0]
input_rate = R*H*(model.ambient(stop)[0]-state[m-1])
heat_rate_error = max(heat_rate_error, float(abs(np.sum(w*capacity*deriva-
tive[:m])-input_rate)))
for report_t in REPORT_TIMES[(REPORT_TIMES > start) & (RE-
PORT_TIMES <= stop)]:
    j = np.flatnonzero(rows == report_t)[0]
    profile_t[int(report_t)] = sol.y[:m, j].copy()
    profile_c[int(report_t)] = sol.y[m:2*m, j].copy()
if stop % 3600 == 0:
    print(f'N={n}, t={stop/3600:g} h, T_center={state[0]:.6f}, '
          f'C_surface={state[2*m-1]:.6f}', flush=True)
assert np.isfinite(temperatures).all() and np.isfinite(moistures).all()
return dict(T=temperatures, C=moistures, t=times, r=model.r,
            plot_T=plot_temperatures, plot_C=plot_moistures,
            plot_r=model.r[plot_indices],
            mean_T=mean_t, mean_C=mean_c,
            profiles_T=np.array([profile_t[int(t)] for t in REPORT_TIMES if t
<= end_time]),
            profiles_C=np.array([profile_c[int(t)] for t in REPORT_TIMES if t
<= end_time]),
            moisture_balance_error=balance_error, heat_rate_balance_er-
ror=heat_rate_error,
            nfev=nfev, nlu=nlu)

def make_figures(result, env):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    from matplotlib.colors import LinearSegmentedColormap, PowerNorm
    plt.rcParams.update({'font.sans-serif': ['Microsoft YaHei', 'SimHei', 'DejaVu Sans'],
                        'axes.unicode_minus': False, 'font.size': 10})
    time_h = result['t'] / 3600.0
    r_cm = result.get('plot_r', result['r']) * 100.0

```

```

temperature = result.get('plot_T', result['T']).T
moisture = result.get('plot_C', result['C']).T
temperature_cmap = LinearSegmentedColormap.from_list(
    'temperature_blue_red', ['#08306b', '#2171b5', '#f7f7f7', '#cb181d', '#67000d'])
moisture_cmap = LinearSegmentedColormap.from_list(
    'moisture_gray_blue', ['#f0f0f0', '#bdbdbd', '#9ecae1', '#3182bd', '#08519c'])
fig, axes = plt.subplots(1, 2, figsize=(13, 5.6), constrained_layout=True)
temp_mesh = axes[0].pcolormesh(
    time_h, r_cm, temperature, shading='nearest', cmap=temperature_cmap,
    vmin=float(np.min(temperature)), vmax=float(np.max(temperature)))
moisture_mesh = axes[1].pcolormesh(
    time_h, r_cm, moisture, shading='nearest', cmap=moisture_cmap,
    norm=PowerNorm(gamma=1.8, vmin=float(np.min(moisture)),
                    vmax=float(np.max(moisture))))
axes[0].set(title='温度场', xlabel='时间 / h', ylabel='距中心距离 / cm', ylim=(0, R
* 100.0))
axes[1].set(title='水分浓度场', xlabel='时间 / h', ylabel='距中心距离 / cm',
ylim=(0, R * 100.0))
for ax in axes:
    ax.set_xlim(float(time_h[0]), float(time_h[-1]))
    ax.set_yticks(np.arange(0, R * 100.0 + 0.01, 0.5))
temp_bar = fig.colorbar(temp_mesh, ax=axes[0], pad=0.02)
temp_bar.set_label('温度 / ° C')
moisture_bar = fig.colorbar(moisture_mesh, ax=axes[1], pad=0.02)
moisture_bar.set_label('水分浓度 / (kg/kg)')
fig.savefig(Figure_OUT / figure_name('q2', 'temperature_moisture_field'),
dpi=180)
plt.close(fig)

def make_boundary_stage_figure(raw_env, boundary_info):
    """展示如何从平滑后的数据中检测两个稳定点。"""
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.sans-serif': ['Microsoft YaHei', 'SimHei', 'DejaVu Sans'],
                        'axes.unicode_minus': False, 'font.size': 10})
    specs = [
        (1, boundary_info['phase_temperature'], '烘房温度 / ° C', '#9b1c1f'),
        (2, boundary_info['phase_moisture'], '环境水分浓度 / (kg/kg)', '#2166ac'),
    ]
    fig, axes = plt.subplots(2, 1, figsize=(11, 7), sharex=True, constrained_layout=True)
    for ax, (column, phase, ylabel, color) in zip(axes, specs):
        y = raw_env[:, column]
        smooth_window = int(phase['smoothing_window_points'])
        smoothed = savgol_filter(y, smooth_window, 2, mode='interp')
        ax.scatter(raw_env[:, 0], y, s=10, color=color, alpha=0.45,
                    linewidths=0, label='原始观测散点')

```

```

ax.plot(raw_env[:, 0], smoothed, color='#444444', lw=1.5,
        label=f'{smooth_window}点 Savitzky - Golay 平滑')
for index, window in enumerate(phase['windows']):
    ax.axvspan(window['start_s'], window['start_s'] + phase['window_s'],
               color='#f0a202', alpha=0.10 if index else 0.24,
               label='连续稳定窗口' if index == 0 else None)
ax.axvline(phase['phase_s'], color='#d62728', ls='--', lw=1.3,
           label=f'判定分界点 {phase['phase_s']:.0f} s')
ax.text(0.015, 0.08,
        f'极差阈值={phase['range_tol']:.3g}\n 斜率阈值
={phase['slope_tol']:.3g}',
        transform=ax.transAxes, va='bottom',
        bbox={'facecolor': 'white', 'alpha': 0.82, 'edgecolor': '0.7'})
ax.set_ylabel(ylabel)
ax.grid(alpha=0.2)
ax.legend(fontsize=8, ncol=3, loc='best')
axes[1].set_xlabel('时间 / s')
fig.suptitle('独立稳定分界点的探索：平滑、窗口极差与斜率判据（相差 1500
s）')
fig.savefig(FIGURE_OUT / figure_name('q2', 'boundary_stability'), dpi=180)
plt.close(fig)

def make_staged_boundary_figure(raw_env, boundary_t, boundary_c, boundary_info):
    """将原始观测与最终的分段边界绘制在同一图中。"""
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    plt.rcParams.update({'font.sans-serif': ['Microsoft YaHei', 'SimHei', 'DejaVu Sans'],
                        'axes.unicode_minus': False, 'font.size': 10})
    grid = np.linspace(raw_env[0, 0], raw_env[-1, 0], 1600)
    specs = [
        (1, boundary_t, boundary_info['transition_temperature'],
         boundary_info['plateau_temperature'], '烘房温度 / ° C', '#9b1c1f'),
        (2, boundary_c, boundary_info['transition_moisture'],
         boundary_info['plateau_moisture'], '环境水分浓度 / (kg/kg)', '#2166ac'),
    ]
    fig, axes = plt.subplots(2, 1, figsize=(11, 7), sharex=True, constrained_layout=True)
    for ax, (column, boundary, transition, plateau, ylabel, color) in zip(axes, specs):
        left, right = transition['left_s'], transition['right_s']
        pre = grid <= left
        blend = (grid >= left) & (grid <= right)
        post = grid >= right
        ax.scatter(raw_env[:, 0], raw_env[:, column], s=10, color=color, alpha=0.42,
                  linewidths=0, label='附件 1 原始散点')
        ax.plot(grid[pre], boundary(grid[pre]), color='#1f77b4', lw=2.0,
               label='拉伸指数函数')

```

```

ax.plot(grid[blend], boundary(grid[blend]), color='#f28e2b', lw=2.4,
        label='线性平滑链接')
ax.plot(grid[post], boundary(grid[post]), color='#2a9d8f', lw=2.0,
        label='稳定尾端均值')
ax.axvspan(left, right, color='#f0a202', alpha=0.12, label='1800 s 平滑过渡区')
ax.axvline(left, color='#666666', ls=':', lw=1.0)
ax.axvline(right, color='#666666', ls=':', lw=1.0)
ax.set_ylabel(ylabel)
ax.grid(alpha=0.2)
ax.legend(fontsize=8, ncol=2, loc='best')
axes[1].set_xlabel('时间 / s')
fig.suptitle('原始散点与最终分段环境边界（全观测区间）')
fig.savefig(FIGURE_OUT / figure_name('q2', 'boundary_staged_fit'), dpi=180)
plt.close(fig)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--grids', type=int, nargs='+', default=[800, 1600, 3200, 6400])
    parser.add_argument('--check-time', action='store_true')
    parser.add_argument('--boundary-mode', choices=('staged', 'raw'), default='staged',
                        help='使用检测得到的平滑恒定阶段边界（默认），或
使用原始 60 s 节点。')
    parser.add_argument('--fit-endpoint-s', type=float, default=None,
                        help='可选的共同拟合终点；省略时分别拟合温度和含
水率到各自的过渡点。')
    args = parser.parse_args()
    raw_env = read_environment()
    boundary_t, boundary_c, env, boundary_info = build_boundaries(
        raw_env, mode=args.boundary_mode, fit_endpoint_s=args.fit_endpoint_s)
    boundary = None if args.boundary_mode == 'raw' else (boundary_t, boundary_c)
    jac_error = check_jacobian(env)
    print('Jacobian complex-step relative error:', jac_error, flush=True)
    convergence, previous = [], None
    for n in args.grids:
        start = time.perf_counter()
        result = solve_model(n, env, boundary=boundary)
        record = {'intervals': n, 'dr_m': R/n,
                  'moisture_balance_error': result['moisture_balance_error'],
                  'heat_rate_balance_error': result['heat_rate_balance_error']}
        if previous is not None:
            for key in ['T', 'C']:
                delta = np.abs(result[key]-previous[key])
                record[f'max_diff_{key}'] = float(np.max(delta))
                record[f'table_diff_{key}'] = float(np.max(delta[RE-
PORT_TIMES][:, ::5]))
            index = np.unravel_index(np.argmax(delta), delta.shape)

```

```

        record[fmax_diff_location_{key}] = {'time_s': int(index[0]), 'r_cm':
float(index[1]/10)}
        record['elapsed_s'] = time.perf_counter()-start
        convergence.append(record)
        previous = result
        print(json.dumps(record), flush=True)
    time_check = {}
    if args.check_time:
        tight = solve_model(args.grids[-1], env, rtol=2e-11, atol=2e-13, max_step=2.0,
                             boundary=boundary)
        time_check = {key: float(np.max(np.abs(result[key]-tight[key]))) for key in
['T', 'C']}
    print('Time convergence:', time_check, flush=True)
    assert np.min(result['T']) >= T0-1e-8
    assert np.max(result['T']) <= np.max(env[env[:, 0] <= END_TIME, 1])+1e-8
    assert np.min(result['C']) > 0 and np.max(result['C']) <= C0+1e-8
    assert np.max(np.diff(result['C'], axis=1)) < 1e-7
    # 受烘房实测波动影响，温度梯度局部可能发生反向。
    payload = {'r_cm': np.round(np.linspace(0, 2, 21), 1).tolist(),
               't_s': result['t'][1:].astype(int).tolist(),
               'T': np.round(result['T'][1:], 4).tolist(),
               'C': np.round(result['C'][1:], 4).tolist()}
    (OUT / artifact_name('q2', 'result_data')).write_text(
        json.dumps(payload, ensure_ascii=False), encoding='utf-8')
    np.savez_compressed(
        OUT / artifact_name('q2', 'full_precision'), t=result['t'], r_cm=payload['r_cm'],
        T=result['T'], C=result['C'], mean_T=result['mean_T'], mean_C=re-
sult['mean_C'])
    initial_props = properties(np.array([T0]), np.array([C0]))
    final_props = properties(result['T'][-1], result['C'][-1])
    summary = {'model': 'Appendix 3 throughout; 1D radial; Kirchhoff moisture flux;
effective moisture Robin boundary; no latent heat',
               'moisture_face_flux': '8-point Gauss-Legendre Kirchhoff average in C
at arithmetic face temperature',
               'input_sha256': hashlib.sha256((DATA/'附件
1.xlsx').read_bytes()).hexdigest(),
               'duration_s': END_TIME, 'grid_intervals': args.grids[-1],
               'rtol': 2e-10, 'atol': 2e-12, 'max_step_s': 5,
               'interpolation': ('detected independent stage boundaries: stretched ex-
ponential before '
                                'the temperature/moisture-specific split, 1800 s
blends, then separate '
                                'measured tail means'
                                if args.boundary_mode == 'staged' else
                                'piecewise linear at original 60 s knots'),
               'boundary_mode': args.boundary_mode,
               'boundary_metadata': boundary_info,
               'raw_environment_sha256': hashlib.sha256((DATA/'附件
1.xlsx').read_bytes()).hexdigest(),

```

```

        'tail_assumption': ('hold the detected tail mean after the 1800 s transi-
tion; '
                            'the phase point and tail mean are recorded in
boundary_metadata'
                            if args.boundary_mode == 'staged' else
                            'hold final observation after 14400 s'),
        'H_W_m2_K': H, 'HM_m_s': HM, 'jacobian_relative_error': jac_error,
        'convergence': convergence, 'time_check': time_check,
        'initial_properties': {key: float(value[0]) for key, value in zip(['rho_cp',
'k', 'D'], initial_props[:3])},
        'final_D_center_surface': [float(final_props[2][0]), float(fi-
nal_props[2][-1])],
        'environment_10800': env[env[:, 0] == END_TIME][0].tolist(),
        'mean_T_10800': float(result['mean_T'][-1]), 'mean_C_10800':
float(result['mean_C'][-1]),
        'table_times_h': (REPORT_TIMES/3600).tolist(), 'table_r_cm': [0, 0.5,
1, 1.5, 2],
        'table_T': result['T'][REPORT_TIMES][:, ::5].tolist(),
        'table_C': result['C'][REPORT_TIMES][:, ::5].tolist()}
(OUT / artifact_name('q2', 'validation')).write_text(
    json.dumps(summary, ensure_ascii=False, indent=2), encoding='utf-8')
make_figures(result, env)
if args.boundary_mode == 'staged':
    make_boundary_stage_figure(raw_env, boundary_info)
    make_staged_boundary_figure(raw_env, boundary_t, boundary_c, bound-
ary_info)
print(json.dumps(summary, ensure_ascii=False, indent=2), flush=True)

if __name__ == '__main__':
    # 稀疏带状系统不适合使用超额分配的稠密 BLAS 线程池。
    with threadpool_limits(limits=1):
        main()

```

附录 B4: code/q3/solve_q3.py

第三问核心求解代码

```

"""第三问：复用第二问方程，定位 max(C)=0.15，并每 60 s 导出结果。
python 第三问/solve_q3.py --check-time --sensitivity
"""

from pathlib import Path
import argparse
import importlib.util
import json
import hashlib
import sys
import time
import numpy as np
from scipy.integrate import solve_ivp

```

```

from scipy.interpolate import PchipInterpolator
from threadpoolctl import threadpool_limits

sys.dont_write_bytecode = True
HERE = Path(__file__).resolve().parent
PACKAGE = HERE.parent.parent
OUT = PACKAGE / 'results' / 'q3'
FIGURE_OUT = PACKAGE / 'figures' / 'q3'
OUT.mkdir(parents=True, exist_ok=True)
FIGURE_OUT.mkdir(parents=True, exist_ok=True)
PLOT_RADIAL_POINTS = 201
if str(HERE.parent) not in sys.path:
    sys.path.insert(0, str(HERE.parent))
from artifact_names import artifact_name, figure_name

def load_q2(root=None):
    spec = importlib.util.spec_from_file_location('q2_model', PACKAGE/'code'/'q2'/'solve_q2.py')
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    return module

def integrate(q2, env, n, rtol=2e-10, atol=2e-12, max_step=300., tail=None):
    return _integrate(q2, env, n, rtol=rtol, atol=atol, max_step=max_step, tail=tail,
                      boundary=None)

def _integrate(q2, env, n, rtol=2e-10, atol=2e-12, max_step=300., tail=None,
               boundary=None):
    class Model(q2.CoupledModel):
        def __init__(self, n, env):
            super().__init__(n, env, boundary=boundary)
            x = np.linspace(0, 1, n+1)
            self.r = q2.R * (-np.expml(-4*x)) / (-np.expml(-4))
            self.faces = (self.r[:-1] + self.r[1:])/2
            self.w = np.diff(np.r_[0., self.faces, q2.R]**2)/2
            self.factor = self.faces/np.diff(self.r)
        def ambient(self, t):
            if boundary is not None:
                return float(boundary[0](t)), float(boundary[1](t))
            if tail is not None and t > env[-1, 0]:
                return tail
            return super().ambient(t)
    model = Model(n, env)
    m = model.m
    state = np.r_[np.full(m, q2.T0), np.full(m, q2.C0), 0.0]
    radii = np.linspace(0, q2.R, 21)

```

```

plot_radial=np.linspace(0,q2.R,PLOT_RADIAL_POINTS)
def sample(y):
    return PchipInterpolator(model.r,y[:m],axis=0)(radial), PchipInterpolator(model.r,y[m:2*m],axis=0)(radial)
def sample_plot(y):
    return (PchipInterpolator(model.r,y[:m],axis=0)(plot_radial),
            PchipInterpolator(model.r,y[m:2*m],axis=0)(plot_radial))
times, out_t, out_c, avg_c = [0.], [np.full(21,28.)], [np.full(21,2.55)], [2.55]
plot_t, plot_c = [np.full(PLOT_RADIAL_POINTS,28.)], [np.full(PLOT_RADIAL_POINTS,2.55)]
balance, max_radial_increase = 0., 0.
event_t, event_y = None, None
snapshots = {}

def event(t,y):
    return np.max(y[m:2*m])-0.15
event.terminal, event.direction = True, -1

breaks = np.unique(np.r_[env[:,0],np.arange(21600,864001,21600)])
for start, stop in zip(breaks[:-1],breaks[1:]):
    sol = solve_ivp(model.rhs,(start,stop),state,method='BDF',jac=model.jac,
                    rtol=rtol,atol=atol,first_step=.001,
                    max_step=min(max_step,5.) if start < env[-1,0] else
max_step,
                    events=event,dense_output=True)
    if not sol.success:
        raise RuntimeError(sol.message)
    end = sol.t[-1]
    # 在积分器接受的每个状态上检查所有物理节点。
    assert np.min(sol.y[m:2*m]) > 0 and np.max(sol.y[m:2*m]) <= 2.55+1e-8
    max_radial_increase = max(max_radial_increase,float(np.max(np.diff(sol.y[m:2*m],axis=0))))
    balance = max(balance,float(np.max(np.abs(model.w@sol.y[m:2*m]/model.area-2.55-sol.y[-1]))))
    wanted = np.arange((int(start)//60+1)*60, end+1e-8,60.)
    if len(wanted):
        y = sol.sol(wanted)
        times.extend(wanted.tolist())
        sampled_t,sampled_c=sample(y)
        out_t.extend(sampled_t.T)
        out_c.extend(sampled_c.T)
        sampled_plot_t,sampled_plot_c=sample_plot(y)
        plot_t.extend(sampled_plot_t.T)
        plot_c.extend(sampled_plot_c.T)
        avg_c.extend((model.w@y[m:2*m]/model.area).tolist())
    state = sol.y[:, -1]
    if stop%21600 == 0:
        print(f'N={n}, t={end/3600:.4f} h, max
C={np.max(state[m:2*m]):.8f}',flush=True)

```

```

        if len(sol.t_events[0]):
            event_t, event_y = float(sol.t_events[0][0]), sol.y_events[0][0]
            break
    if event_t is None:
        raise RuntimeError('Threshold not reached within 240 h; no time is fabricated.')
    # 向上取整到小时的四位小数，确保公布时长对应的状态确实低于阈值。
    finish_h = np.ceil(event_t/3600*1e4)/1e4
    finish_s = float(finish_h*3600)
    if finish_s-event_t < 1e-7:
        finish_h += 1e-4
        finish_s = float(finish_h*3600)
    endsol = solve_ivp(model.rhs,(event_t,finish_s),event_y,method='BDF',jac=model.jac,
                        rtol=rtol,atol=atol,first_step=min(.001,(finish_s-event_t)/2))
    assert endsol.success
    final_y = endsol.y[:, -1]
    assert np.max(final_y[m:2*m]) < .15
    times.append(finish_s)
    sampled_t,sampled_c=sample(final_y)
    out_t.append(sampled_t)
    out_c.append(sampled_c)
    sampled_plot_t,sampled_plot_c=sample_plot(final_y)
    plot_t.append(sampled_plot_t)
    plot_c.append(sampled_plot_c)
    avg_c.append(float(model.w@final_y[m:2*m]/model.area))
    times, out_t, out_c = np.array(times),np.array(out_t),np.array(out_c)
    assert np.all(np.diff(times)>0)
    assert max_radial_increase < 1e-8
    table_indices = [i for i,t in enumerate(times) if t>0 and abs(t%21600)<1e-6]
    if len(times)-1 not in table_indices:
        table_indices.append(len(times)-1)
    return dict(t=times,T=out_t,C=out_c,mean_C=np.array(avg_c),
                plot_r_cm=plot_radii*100.0,plot_T=np.array(plot_t),plot_C=np.array(plot_c),
                event_s=event_t,
                finish_h=float(finish_h),finish_s=finish_s,
                final_max_C=float(np.max(final_y[m:2*m])),
                final_max_r_cm=float(model.r[np.argmax(final_y[m:2*m])]*100),
                balance_error=balance,max_radial_increase=max_radial_increase,
                table_t=times[table_indices],table_C=out_c[table_indices][:,:5])

def plot(result,out):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    from matplotlib.colors import LinearSegmentedColormap, PowerNorm
    plt.rcParams.update({'font.sans-serif':['Microsoft YaHei','SimHei','DejaVu Sans'],

```

```

        'axes.unicode_minus':False,'font.size':10})

time_h=result['t']/3600.0
r_cm=result['plot_r_cm']
moisture=result['plot_C'].T
moisture_cmap=LinearSegmentedColormap.from_list(
    'moisture_gray_blue',
    [(0.00,'#f0f0f0'),(0.13,'#d9d9d9'),(0.20,'#bdbdbd'),
     (0.24,'#9ecae1'),(0.50,'#3182bd'),(1.00,'#08519c')])
# 突出低浓度区间，同时保留含水率场从灰色到蓝色的单调颜色含义。
moisture_cmap.set_bad('#ffffff')
# 次线性映射将中位浓度放在视觉中部附近，使灰色区和蓝色区的面积观感
保持可比。
# 显示刻度从 0.08 kg/kg 开始；更低的数值裁剪到浅灰色端点，因此 0.08
是色标
# 的底部刻度。较温和的次线性指数使 0.12 和 0.15 仍靠近色标底部。
moisture_norm=PowerNorm(gamma=0.5,vmin=0.08,
                        vmax=float(np.nanmax(moisture)),clip=True)
moisture_ticks=np.array([0.08,0.12,0.15,0.30,0.60,1.00,1.50,2.00,2.55])
fig, ax = plt.subplots(figsize=(9.4,5.6),constrained_layout=True)
moisture_mesh=ax.pcolormesh(
    time_h,r_cm,moisture,shading='nearest',cmap=moisture_cmap,
    norm=moisture_norm)
ax.contour(time_h,r_cm,moisture,levels=[0.15],colors=['#4d4d4d'],
           linestyle='--',linewidths=1.0,zorder=6)
ax.set(title='水分浓度场',xlabel='时间 / h',ylabel='距中心距离 / cm',ylim=(0,2))
ax.set_xlim(float(time_h[0]),float(time_h[-1]))
ax.set_yticks(np.arange(0,2.01,.5))
moisture_bar=fig.colorbar(moisture_mesh,ax=ax,pad=.02,ticks=moisture_ticks)
moisture_bar.ax.tick_params(labelsize=8,pad=2)
moisture_bar.set_label('水分浓度 / (kg/kg)')
fig.savefig(FIGURE_OUT / figure_name('q3', 'drying_result'), dpi=180)
plt.close(fig)

def main():
    parser=argparse.ArgumentParser()
    parser.add_argument('--root',type=Path,default=PACKAGE/'data',
                        help='为兼容旧命令保留；一体化提交包使用其本地
data 目录。')
    parser.add_argument('--out',type=Path,default=OUT)
    parser.add_argument('--grids',type=int,nargs='+',default=[400,800,1600,3200])
    parser.add_argument('--check-time',action='store_true')
    parser.add_argument('--sensitivity',action='store_true')
    parser.add_argument('--boundary-mode',choices=('staged','raw'),default='staged',
                        help='使用分别检测的温度/含水率阶段边界（默认），
或使用原始 60 s 节点。')
    parser.add_argument('--fit-endpoint-s',type=float,default=None,

```

```

help='可选的共同拟合终点：默认使用各变量自身的过
渡点。')
args=parser.parse_args()
args.out.mkdir(parents=True,exist_ok=True)
q2=load_q2(args.root)
env=q2.read_environment()
if str(HERE) not in sys.path:
    sys.path.insert(0,str(HERE))
from boundary_stage import build_boundaries
boundary_t,boundary_c,solver_env,boundary_info=build_boundaries(
    env,mode=args.boundary_mode,fit_endpoint_s=args.fit_endpoint_s)
boundary=None if args.boundary_mode=='raw' else (boundary_t,boundary_c)
records,prev=[],None
for n in args.grids:
    start=time.perf_counter()
    result=_integrate(q2,solver_env,n,boundary=boundary)
    rec={'N':n,'event_s':result['event_s'],'finish_h':result['finish_h'],
        'balance_error':result['balance_error'],'elapsed_s':time.perf_counter()-
start}
    if prev is not None:
        count=min(len(prev['t']),len(result['t']))-1
        rec['event_diff_s']=abs(result['event_s']-prev['event_s'])
        rec['max_C_diff_60s']=float(np.max(abs(result['C'][:count]-
prev['C'][:count])))
        records.append(rec)
        prev=result
        print(json.dumps(rec),flush=True)
    checks={}
    if args.check_time:
        tight=_integrate(q2,solver_env,args.grids[-1],rtol=2e-11,atol=2e-
13,max_step=120.,
                        boundary=boundary)
        count=min(len(tight['t']),len(result['t']))-1
        checks={'event_diff_s':abs(tight['event_s']-result['event_s']),
            'max_C_diff':float(np.max(abs(tight['C'][:count]-re-
sult['C'][:count])))}
        sensitivity=[]
        if args.sensitivity:
            for name,tail in [('50C_0.05',(50.,.05)),('last_hour_mean',tu-
ple(env[env[:,0]>=10800,1:].mean(axis=0)))]:
                sensitivity_grid=args.grids[-1]
                test=_integrate(q2,solver_env,sensitivity_grid,tail=tail)
                baseline=next((r['event_s'] for r in records if r['N']==sensitiv-
ity_grid),None)
                sensitivity.append({'case':name,'T_tail':float(tail[0]),'C_tail':float(tail[1]),
                    'sensitivity_grid':sensitivity_grid,
                    'event_h':test['event_s']/3600,
                    'delta_h_at_grid':None if baseline is None else
(test['event_s']-baseline)/3600})

```

```

q2_diff={}
q2data=PACKAGE/'results/'q2/artifact_name('q2', 'full_precision')
if q2data.exists():
    with np.load(q2data) as reference:
        match=result['t']<=10800
        for field in ['T','C']:
            q2_diff[field]=float(np.max(abs(result[field][match]-reference[field][result['t'][match].astype(int)])))
        summary={'model':'same equations as Q2; Kirchhoff moisture flux; graded radial
finite volumes',
'moisture_face_flux':'inherited from Q2: 8-point Gauss-Legendre Kirchhoff average in C at arithmetic face temperature',
'grid_mapping':'r=R*(1-exp(-4*x))/(1-exp(-4)), x=i/N',
'output_interpolation':'PCHIP',
'input_sha256':hashlib.sha256((PACKAGE/'data/'附件
1.xlsx').read_bytes()).hexdigest(),
'q2_code_sha256':hashlib.sha256((PACKAGE/'code/'q2/'solve_q2.py').read_bytes()).hexdigest(),
'N':args.grids[-1],'rtol':2e-10,'atol':2e-12,'max_step_after_4h_s':300.,
'tail_T':float(solver_env[-1,1]),'tail_C':float(solver_env[-1,2]),
'boundary_mode':args.boundary_mode,'boundary_metadata':boundary_info,
'event_s':result['event_s'],'event_h':result['event_s']/3600,
'finish_s':result['finish_s'],'finish_h':result['finish_h'],
'final_max_C':result['final_max_C'],'final_max_r_cm':result['final_max_r_cm'],
'final_mean_C':float(result['mean_C'][-1]),
'max_radial_increase':result['max_radial_increase'],
'grid_checks':records,'time_check':checks,'tail_sensitivity':sensitivity,
'q2_overlap_max_diff':q2_diff,
'table_t_s':result['table_t'].tolist(),'table_C':result['table_C'].tolist(),
'output_time_rows':len(result['t'])-1}
(args.out / artifact_name('q3', 'validation')).write_text(
    json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
payload={'r_cm':np.round(np.linspace(0,2,21),1).tolist(),
't_s':np.round(result['t'][1:],4).tolist(),'C':np.round(result['C'][1:],4).tolist()}
(args.out / artifact_name('q3', 'result_data')).write_text(
    json.dumps(payload,ensure_ascii=False),encoding='utf-8')
np.savez_compressed(
    args.out / artifact_name('q3', 'full_precision'), t=result['t'], T=result['T'], C=result['C'], mean_C=result['mean_C'],
)
plot(result,args.out)
print(json.dumps(summary,ensure_ascii=False,indent=2),flush=True)

if __name__=='__main__':
    with threadpool_limits(limits=1):

```

main()

附录 B5: code/q4/solve_q4.py

第四问核心求解代码

```
"""第四问：材料坐标下的变半径传热/传质模型。
python 第四问/solve_q4.py --check-time --comparisons
"""

from pathlib import Path
import argparse, csv, hashlib, importlib.util, json, sys, time
import numpy as np
import openpyxl
from scipy.integrate import solve_ivp
from scipy.interpolate import PchipInterpolator, Akima1DInterpolator, CubicSpline, UnivariateSpline
from scipy.sparse import diags, bmat, csr_matrix
from threadpoolctl import threadpool_limits

sys.dont_write_bytecode=True
HERE=Path(__file__).resolve().parent
PACKAGE=HERE.parent.parent
DEFAULT_ROOT=PACKAGE/'data'
OUT=PACKAGE/'results'/'q4'
FIGURE_OUT=PACKAGE/'figures'/'q4'
OUT.mkdir(parents=True,exist_ok=True)
FIGURE_OUT.mkdir(parents=True,exist_ok=True)
if str(HERE) not in sys.path:
    sys.path.insert(0,str(HERE))
if str(HERE.parent) not in sys.path:
    sys.path.insert(0,str(HERE.parent))
from artifact_names import artifact_name, figure_name
from kirchhoff_flux import moisture_face
from boundary_stage import build_boundaries
R0,H,HM,T0,C0=.02,25.,8e-7,28.,2.55
OUTPUT_R=np.arange(20)*.001
PLOT_RADIAL_POINTS=201

def read_xlsx(path):
    w=openpyxl.load_workbook(path,data_only=True,read_only=True)
    rows=list(w.active.values)
    w.close()
    a=np.array(rows[1:],dtype=float)
    assert np.isfinite(a).all() and a[0,0]==0 and np.all(np.diff(a[:,0])>0)
    return a

def _radius_smoothing_factor(values):
    second=np.diff(np.asarray(values,dtype=float),n=2)
    center=np.median(second)
    sigma=1.4826*np.median(np.abs(second-center))/np.sqrt(6.)
```

```

if not np.isfinite(sigma) or sigma<=0:
    sigma=max(float(np.std(values))*1e-4,np.finfo(float).eps)
return float(len(values)*sigma**2)

def _radius_curve(name,t,r):
    if name=='linear':
        return lambda q: np.interp(q,t,r), {'method':name}
    if name=='pchip':
        return PchipInterpolator(t,r,extrapolate=False), {'method':name}
    if name=='akima':
        return Akima1DInterpolator(t,r,extrapolate=False), {'method':name}
    if name=='cubic_spline':
        return CubicSpline(t,r,bc_type='natural',extrapolate=False), {'method':name}
    if name=='smooth_spline':
        s=_radius_smoothing_factor(r)
        return UnivariateSpline(t,r,s=s,ext=2), {'method':name,'smoothing_factor':s}
    raise ValueError('Unknown radius diagnostic method: '+str(name))

def _radius_metrics(curve,t_train,r_train,t_test,r_test):
    prediction=np.asarray(curve(t_test),dtype=float)
    if not np.isfinite(prediction).all():
        raise ValueError('Radius candidate returned a non-finite value.')
    dense=np.linspace(float(t_test[0]),float(t_test[-1]),721)
    values=np.asarray(curve(dense),dtype=float)
    if not np.isfinite(values).all():
        raise ValueError('Radius candidate returned a non-finite dense value.')
    lo,hi=float(np.min(r_train)),float(np.max(r_train))
    overshoot=np.maximum(lo-values,0.)+np.maximum(values-hi,0.)
    return {
        'cv_rmse_cm':float(np.sqrt(np.mean((prediction-r_test)**2))*100.),
        'cv_mae_cm':float(np.mean(np.abs(prediction-r_test))*100.),
        'cv_max_abs_cm':float(np.max(np.abs(prediction-r_test))*100.),
        'overshoot_max_cm':float(np.max(overshoot)*100.),
        'roughness':float(np.mean(np.abs(np.diff(values,n=2))/(dense[1]-
dense[0])**2))),
    }

def radius_diagnostics(radius,out):
    """核验候选半径曲线，并绘制最终采用的 PCHIP 表示。"""
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    methods=['linear','pchip','akima','cubic_spline','smooth_spline']
    labels={'linear': '分段线性','pchip': 'PCHIP','akima': 'Akima',
            'cubic_spline': '自然三次样条','smooth_spline': '平滑样条'}
    colors={'linear': '#1f77b4','pchip': '#d62728','akima': '#2ca02c',
            'cubic_spline': '#9467bd','smooth_spline': '#ff7f0e'}
    t,r=radius[:,0],radius[:,1]
    hold=np.zeros(len(t),dtype=bool)

```

```

hold[1:-1:5]=True
rows=[]
for name in methods:
    try:
        curve,metadata=_radius_curve(name,t[~hold],r[~hold])
        metrics=_radius_metrics(curve,t[~hold],r[~hold],t[hold],r[hold])
        dense=np.linspace(t[0],t[-1],5000)
        values=np.asarray(curve(dense),dtype=float)
        row={'method':name,**metrics,
            'increasing_segments':int(np.sum(np.diff(values)>1e-9)),
            'min_radius_cm':float(np.min(values)*100.),
            'max_radius_cm':float(np.max(values)*100.),
            'fit_metadata':repr(metadata)}
    except Exception as exc:
        row={'method':name,'error':str(exc)}
    rows.append(row)

fieldnames=['cv_mae_cm','cv_max_abs_cm','cv_rmse_cm','fit_metadata',
            'increasing_segments','max_radius_cm','method','min_radius_cm',
            'overshoot_max_cm','roughness']
with (out / artifact_name('q4', 'radius_method_comparison')).open(
    'w',encoding='utf-8',newline='') as fh:
    writer=csv.DictWriter(fh,fieldnames=fieldnames)
    writer.writeheader()
    for row in rows:
        writer.writerow({key:row.get(key,"") for key in fieldnames})

plt.rcParams.update({'font.sans-serif':['Microsoft YaHei','SimHei','DejaVu Sans'],
                    'axes.unicode_minus':False,'font.size':10})
fig,ax=plt.subplots(figsize=(9,4.8),constrained_layout=True)
ax.scatter(t/3600.,r*100.,s=12,color='#4d4d4d',alpha=.65,label='附件 2 原始观测')
ax.set(xlabel='时间 / h',ylabel='半径 / cm',title='附件 2: 药材半径原始观测')
ax.grid(alpha=.2)
ax.legend(fontsize=9)
fig.savefig(FIGURE_OUT / figure_name('q4', 'radius_raw'), dpi=180)
plt.close(fig)

grid=np.linspace(t[0],t[-1],1800)
pchip_curve,_=_radius_curve('pchip',t,r)
fig,axes=plt.subplots(1,2,figsize=(13,4.8),constrained_layout=True)
for ax,xmax,title in [(axes[0],72.,'全观测区间'),(axes[1],24.,'前 24 h 放大')]:
    ax.scatter(t/3600.,r*100.,s=8,color='#555555',alpha=.45,label='附件 2 观测')
    ax.plot(grid/3600.,np.asarray(pchip_curve(grid))*100.,color='#d62728',lw=2.4,
            label='PCHIP (最终选定)')
    ax.set(xlabel='时间 / h',ylabel='半径 / cm',xlim=(0.,xmax),title=title)
    ax.grid(alpha=.2)
    ax.legend(fontsize=8)
fig.suptitle('半径数据与最终选定的 PCHIP 插值')

```

```

fig.savefig(FIGURE_OUT / figure_name('q4', 'radius_pchip_fit'), dpi=180)
plt.close(fig)
return rows

def load_q2(root=None):
    spec=importlib.util.spec_from_file_location('q2_model',PACK-
AGE/'code'/q2/'solve_q2.py')
    q2=importlib.util.module_from_spec(spec)
    spec.loader.exec_module(q2)
    return q2

def props4(T,C):
    K=T+273.15
    if np.min(C.real)<=0 or np.min(K.real)<=0:
        raise ValueError('Nonphysical state; no silent clipping.')
    rho=760+90*C
    cp=1850+2150*C/(C+1)
    k=.12+.20*C/(C+1)
    D=4.2e-4*np.exp(-.30/C-3850/K)
    return
rho*cp,k,D,90*cp+rho*2150/(C+1)**2,.20/(C+1)**2,D*.30/C**2,D*3850/K**2

class Model:
    def __init__(self,n,env,radius,material=props4,fixed_radius=None,linear_ra-
dius=False,tail=None,
                graded=True,boundary=None,radius_method=None):
        self.n,self.m=n,n+1
        self.env,self.radius_data,self.material=env,radius,material
        self.fixed_radius,self.linear_radius,self.tail=fixed_radius,linear_radius,tail
        self.boundary=boundary
        self.radius_method='linear' if linear_radius and radius_method is None else (ra-
dius_method or 'pchip')
        z=np.linspace(0,1,n+1)
        self.x=-np.expm1(-4*z)/(-np.expm1(-4)) if graded else z
        self.faces=(self.x[:-1]+self.x[1:])/2
        self.w=np.diff(np.r_[0.,self.faces,1.])**2)/2
        self.factor=self.faces/np.diff(self.x)
        if self.radius_method == 'pchip':
            self.curve=PchipInterpolator(radius[:,0],radius[:,1],extrapolate=False)
        elif self.radius_method == 'akima':
            self.curve=Akima1DInterpolator(radius[:,0],radius[:,1],extrapolate=False)
        elif self.radius_method == 'cubic_spline':
            self.curve=CubicSpline(radius[:,0],radius[:,1],bc_type='natural',extrapo-
late=False)
        elif self.radius_method == 'linear':
            self.curve=None
        else:
            raise ValueError('Unknown radius_method: '+str(self.radius_method))
        self.zero=csr_matrix((1,self.m))

```

```

def R(self,t):
    if self.fixed_radius is not None:
        return np.full_like(np.asarray(t,dtype=float),self.fixed_radius)
    t=np.clip(t,self.radius_data[0,0],self.radius_data[-1,0])
    return np.interp(t,self.radius_data[:,0],self.radius_data[:,1]) if self.radius_method=='linear' else self.curve(t)
def ambient(self,t):
    if self.boundary is not None:
        return float(self.boundary[0](t)),float(self.boundary[1](t))
    if self.tail is not None and t>self.env[-1,0]:
        return self.tail
    return np.interp(t,self.env[:,0],self.env[:,1]),np.interp(t,self.env[:,0],self.env[:,2])
def rhs(self,t,y):
    m=self.m
    T,C=y[:m],y[m:2*m]
    b,k,D,*_=self.material(T,C)
    R=self.R(t)
    ta,ca=self.ambient(t)
    ft=np.empty(m+1,dtype=y.dtype)
    fc=np.empty_like(ft)
    ft[0]=fc[0]=0
    ft[1:-1]=self.factor*(k[:-1]+k[1:])/2*np.diff(T)
    Df,*_=moisture_face(T[:-1],T[1:],C[:-1],C[1:],self.material)
    fc[1:-1]=self.factor*Df*np.diff(C)
    ft[-1]=R*H*(ta-T[-1])
    fc[-1]=R*HM*(ca-C[-1])
    return np.r_[np.diff(ft)/(R*R*self.w*b),np.diff(fc)/(R*R*self.w),2*HM/R*(ca-C[-1])]
def block(self,l,r,boundary,den):
    main=np.r_[l,boundary]-np.r_[0.,r]
    return diags((-l/den[1:],main/den,r/den[:-1]),(-1,0,1),format='csr')
def jac(self,t,y):
    m=self.m
    T,C=y[:m],y[m:2*m]
    b,k,D,bc,kc,Dc,Dt=self.material(T,C)
    R=self.R(t)
    f=self.factor
    dT,dC=np.diff(T),np.diff(C)
    kf=(k[:-1]+k[1:])/2
    Df,Dcl,Dcr,Dtl,Dtr=moisture_face(T[:-1],T[1:],C[:-1],C[1:],self.material)
    tt=self.block(-f*kf,f*kf,-R*H,R*R*self.w*b)
    tc=self.block(.5*f*kf[:-1]*dT,.5*f*kf[1:]*dT,0.,R*R*self.w*b)
    tc-=diags(self.rhs(t,y)[:m]*bc/b,format='csr')
    cc=self.block(f*(Dcl*dC-Df),f*(Dcr*dC+Df),-R*HM,R*R*self.w)
    ct=self.block(f*Dtl*dC,f*Dtr*dC,0.,R*R*self.w)
    fluxrow=csr_matrix(([-2*HM/R],[0],[m-1])),shape=(1,m))
    return bmat([[tt,tc,None],[ct,cc,None],[self.zero,fluxrow,csr_matrix((1,1))]],format='csc')

```

```

def unit_checks(q2,env,radius):
    rng=np.random.default_rng(2026)
    model=Model(40,env,radius)
    y=np.r_[28+7*model.x**2,2.55-.8*model.x**2,0.]
    errors=[]
    for t in [600.,21600.,100000.]:
        j=model.jac(t,y)
        for _ in range(2):
            v=rng.normal(size=len(y))
            exact=np.imag(model.rhs(t,y+1e-25j*v))/1e-25
            errors.append(float(np.max(abs(j@v-exact))/np.max(abs(exact))))
    assert max(errors)<1e-11
    fixed=Model(40,env,radius,material=q2.properties,fixed_radius=R0,graded=False)
    original=q2.CoupledModel(40,env)
    y=np.r_[28+7*fixed.x**2,2.55-.8*fixed.x**2,0.]
    delta=float(np.max(abs(fixed.rhs(1200,y)-original.rhs(1200,y))))
    assert delta<1e-10
    # 无环境驱动力时，均匀场在收缩过程中仍保持均匀。
    closed=Model(40,np.array([[0.,T0,C0],[259200.,T0,C0]]),radius)
    initial=np.r_[np.full(41,T0),np.full(41,C0),0.]
    invariant=max(float(np.max(abs(closed.rhs(t,initial)))) for t in
[0.,1800.,21600.,259200.])
    assert invariant==0.
    return {'jacobian_relative_error':max(errors),'fixed_radius_q2_rhs_max_difference':delta,'uniform_no_exchange_derivative':invariant}

def simulate(env,radius,n,rtol=2e-10,atol=2e-12,max_step=300.,**options):
    model=Model(n,env,radius,**options)
    m=model.m
    state=np.r_[np.full(m,T0),np.full(m,C0),0.]
    times=[0.]; tc=[]; cc=[]; mean=[]; radii=[]
    plot_r_cm=np.linspace(0.,R0*100.,PLOT_RADIAL_POINTS)
    plot_t=[]; plot_c=[]
    table_profiles=[]
    fixed_profile_x=np.linspace(0,1,401)
    def append(t,y):
        R=float(model.R(t))
        valid=OUTPUT_R<=R
        targets=np.r_[OUTPUT_R[valid]/R,1.]
        interp_t=PchipInterpolator(model.x,y[:m])
        interp_c=PchipInterpolator(model.x,y[m:2*m])
        vt=interp_t(targets)
        vc=interp_c(targets)
        trow,crow=np.full(21,np.nan),np.full(21,np.nan)
        trow[:20][valid],crow[:20][valid]=vt[:-1],vc[:-1]
        trow[-1],crow[-1]=vt[-1],vc[-1]
        tc.append(trow);cc.append(crow);radii.append(R);mean.append(float(2*model.w@y[m:2*m]))

```

```

    plot_valid=plot_r_cm<=R*100.+1e-12
    plot_t_row=np.full(PLOT_RADIAL_POINTS,np.nan)
    plot_c_row=np.full(PLOT_RADIAL_POINTS,np.nan)
    plot_x=plot_r_cm[plot_valid]/(R*100.)
    plot_t_row[plot_valid]=interp_t(plot_x)
    plot_c_row[plot_valid]=interp_c(plot_x)
    plot_t.append(plot_t_row);plot_c.append(plot_c_row)
append(0.,state)
def event(t,y):return float(np.max(y[m:2*m])).15)
event.terminal,event.direction=True,-1
breaks=np.unique(np.r_[env[:,0],radius[:,0],np.arange(21600,864001,21600)])
balance,radial_increase=0.,0.
event_s,event_y=None,None
for start,stop in zip(breaks[:-1],breaks[1:]):
    sol=solve_ivp(model.rhs,(start,stop),state,method='BDF',jac=model.jac,
                  rtol=rtol,atol=atol,first_step=.001,
                  max_step=min(5.,max_step) if start<env[-1,0] else max_step,
                  events=event,dense_output=True)
    if not sol.success:raise RuntimeError(sol.message)
    end=sol.t[-1]
    C=sol.y[m:2*m]
    assert C.min()>0 and C.max()<=C0+1e-8
    radial_increase=max(radial_increase,float(np.max(np.diff(C,axis=0))))
    balance=max(balance,float(np.max(abs(2*model.w@C-C0-sol.y[-1]))))
    wanted=np.arange((int(start)//60+1)*60,end+1e-8,60.)
    if len(wanted):
        ys=sol.sol(wanted)
        for i,t in enumerate(wanted):
            times.append(float(t));append(t,ys[:,i])
            if t%21600==0:
                table_profiles.append(PchipInterpolator(model.x,ys[m:2*m,i])(fixed_profile_x))
        state=sol.y[:, -1]
        if stop%21600==0 or len(sol.t_events[0]):
            print(f'N={n}, t={end/3600:.5f} h, R={float(model.R(end))*100:.6f} cm,
                  Cmax={np.max(state[m:2*m]):.8f}',flush=True)
            if len(sol.t_events[0]):
                event_s,event_y=float(sol.t_events[0][0]),sol.y_events[0][0]
                break
    if event_s is None:raise RuntimeError('No drying event within 240 h.')
    finish_h=float(np.ceil(event_s/3600*1e4)/1e4)
    finish_s=finish_h*3600
    if finish_s-event_s<1e-7:
        finish_h+=1e-4;finish_s=finish_h*3600
    last=solve_ivp(model.rhs,(event_s,finish_s),event_y,method='BDF',jac=model.jac,
                  rtol=rtol,atol=atol,first_step=min(.001,(finish_s-event_s)/2))
    assert last.success and np.max(last.y[m:2*m,-1])<.15
    times.append(finish_s);append(finish_s,last.y[:, -1])

```

```

    table_profiles.append(PchipInterpolator(model.x,last.y[m:2*m,-1])(fixed_profile_x))
    assert radial_increase<1e-8
    a=np.array(times)
    table_indices=[i for i,t in enumerate(a) if t>0 and abs(t%21600)<1e-7]
    if len(a)-1 not in table_indices:table_indices.append(len(a)-1)
    return {'t':a,'T':np.array(tc),'C':np.array(cc),'R':np.array(radii),'mean_C':np.array(mean),
            'plot_r_cm':plot_r_cm,'plot_T':np.array(plot_t),'plot_C':np.array(plot_c),
            'event_s':event_s,'finish_s':finish_s,'finish_h':finish_h,
            'final_max_C':float(np.max(last.y[m:2*m,-1])), 'max_radial_increase':radial_increase,
            'balance_error':balance,'table_indices':table_indices,
            'profile_x':fixed_profile_x,'profiles_C':np.array(table_profiles)}

def graph(result,out):
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    from matplotlib.colors import LinearSegmentedColormap, PowerNorm
    plt.rcParams.update({'font.sans-serif':['Microsoft YaHei','SimHei','DejaVu Sans'],
        'axes.unicode_minus':False,'font.size':10})
    time_h=result['t']/3600.
    r_cm=result['plot_r_cm']
    moisture=result['plot_C'].T
    moisture_cmap=LinearSegmentedColormap.from_list(
        'moisture_gray_blue',
        [(0.00,'#f0f0f0'),(0.13,'#d9d9d9'),(0.20,'#bdbdbd'),
        (0.24,'#9ecae1'),(0.50,'#3182bd'),(1.00,'#08519c')])
    moisture_cmap.set_bad('#ffffff')
    # 突出低浓度区间，同时保留含水率场从灰色到蓝色的单调颜色含义。
    # 次线性映射将中位浓度放在视觉中部附近，使灰色区和蓝色区的面积观感保持可比。
    # 显示刻度从 0.08 kg/kg 开始；更低的数值裁剪到浅灰色端点，因此 0.08 是色标
    # 的底部刻度。较温和的次线性指数使 0.12 和 0.15 仍靠近色标底部。
    moisture_norm=PowerNorm(gamma=0.5,vmin=0.08,
                            vmax=float(np.nanmax(moisture)),clip=True)
    moisture_ticks=np.array([0.08,0.12,0.15,0.30,0.60,1.00,1.50,2.00,2.55])
    fig,moisture_ax=plt.subplots(figsize=(9.4,5.4),constrained_layout=True)
    moisture_mesh=moisture_ax.pcolormesh(
        time_h,r_cm,moisture,shading='nearest',cmap=moisture_cmap,
        norm=moisture_norm)
    moisture_ax.contour(time_h,r_cm,moisture,levels=[0.15],colors=['#4d4d4d'],
        linestyle='--',linewidths=1.0,zorder=6)
    moisture_ax.set(xlabel='时间 / h',ylabel='距中心距离 / cm',
        ylim=(0,R0*100.),title='水分浓度场')
    moisture_ax.set_xlim(float(time_h[0]),float(time_h[-1]))

```

```

moisture_ax.set_yticks(np.arange(0,R0*100+.01,.5))
moisture_ax.fill_between(time_h,result['R']*100.,R0*100.,facecolor='none',
                        edgecolor='black',linewidth=0.,hatch='///',zorder=5)
moisture_ax.plot(time_h,result['R']*100.,color='black',lw=1.5,zorder=7,
                label='当前药材表面')
moisture_bar=fig.colorbar(moisture_mesh,ax=moisture_ax,pad=.02,
                        ticks=moisture_ticks)
moisture_bar.ax.tick_params(labelsize=8,pad=2)
moisture_bar.set_label('水分浓度 / (kg/kg)')
fig.savefig(FIGURE_OUT / figure_name('q4', 'shrinkage_moisture_field'), dpi=180)
plt.close(fig)

```

```
def main():
```

```

    p=argparse.ArgumentParser()
    p.add_argument('--root',type=Path,default=DEFAULT_ROOT)
    p.add_argument('--out',type=Path,default=OUT)
    p.add_argument('--grids',type=int,nargs='+',default=[400,800,1600,3200])
    p.add_argument('--check-time',action='store_true')
    p.add_argument('--comparisons',action='store_true')
    p.add_argument('--boundary-mode',choices=('staged','raw'),default='staged',
                    help='使用分别检测的温度/含水率阶段边界（默认），或使

```

```
用原始 60 s 节点。')
```

```

    p.add_argument('--fit-endpoint-s',type=float,default=None,
                    help='可选的共同拟合终点：默认使用各变量自身的过渡

```

```
点。')
```

```

    p.add_argument('--radius-method',choices=('pchip','linear','akima','cubic_spline'),de-
fault='pchip')

```

```

    a=p.parse_args();a.out.mkdir(parents=True,exist_ok=True)

```

```

    raw_env=read_xlsx(PACKAGE/'data/'附件 1.xlsx)

```

```

    boundary_t,boundary_c,env,boundary_info=build_boundaries(

```

```

        raw_env,mode=a.boundary_mode,fit_endpoint_s=a.fit_endpoint_s)

```

```

    boundary=None if a.boundary_mode=='raw' else (boundary_t,boundary_c)

```

```

    radius=read_xlsx(PACKAGE/'data/'附件 2.xlsx);radius[:,1]/=100

```

```

    assert radius[0,1]==R0 and np.all(np.diff(radius[:,1])<=0)

```

```

    radius_diagnostics(radius,a.out)

```

```

    q2=load_q2()

```

```

    checks=unit_checks(q2,env,radius)

```

```

    print('Model checks:',checks,flush=True)

```

```

    records=[];prev=None

```

```

    for n in a.grids:

```

```

        tick=time.perf_counter();r=simulate(env,radius,n,boundary=boundary,ra-
dius_method=a.radius_method)

```

```

        row={'N':n,'event_s':r['event_s'],'finish_h':r['finish_h'],'balance_error':r['bal-
ance_error'],'elapsed_s':time.perf_counter()-tick}

```

```

        if prev is not None:

```

```

            count=min(len(prev['t']),len(r['t']))-1

```

```

            row['event_diff_s']=abs(r['event_s']-prev['event_s'])

```

```

            assert np.array_equal(np.isnan(r['C'][:count]),np.isnan(prev['C'][:count]))

```

```

        row['max_output_C_diff']=float(np.nanmax(abs(r['C'][:count]-
prev['C'][:count])))
        records.append(row);prev=r;print(json.dumps(row),flush=True)
        timecheck={}
        if a.check_time:
            tight=simulate(env,radius,a.grids[-1],rtol=2e-11,atol=2e-
13,max_step=120.,boundary=boundary,
                            radius_method=a.radius_method)
            count=min(len(tight['t']),len(r['t']))-1
            timecheck={'event_diff_s':abs(r['event_s']-tight['event_s']),
                        'max_output_C_diff':float(np.nanmax(abs(r['C'][:count]-
tight['C'][:count])))}
            comparisons=[]
            if a.comparisons:
                for label,opts in [
                    ('appendix4_fixed_radius',{'fixed_radius':R0}),
                    ('appendix3_shrinking',{'material':q2.properties}),
                    ('linear_radius',{'linear_radius':True}),
                    ('tail_50C_0.05',{'tail':(50.,.05)})]:
                    comparison_boundary=None if 'tail' in opts else boundary
                    c=simulate(env,radius,800,boundary=comparison_boundary,**opts)
                    comparisons.append({'case':label,'N':800,'event_h':c['event_s']/3600})
            table_i=r['table_indices']
            table_C=r['C'][table_i][:,[0,5,10,20]]
            summary={'model':'homogeneous radial shrinkage; Appendix 4; material coordi-
nates; Kirchhoff moisture flux',
                    'moisture_face_flux':'8-point Gauss-Legendre Kirchhoff average in C at
arithmetic face temperature',
                    'environment_sha256':hashlib.sha256((PACKAGE/'data/'附件
1.xlsx').read_bytes()).hexdigest(),
                    'radius_sha256':hashlib.sha256((PACKAGE/'data/'附件
2.xlsx').read_bytes()).hexdigest(),
                    'R_interpolation':a.radius_method+'; hold last after 72
h','tail_T':float(env[-1,1]),'tail_C':float(env[-1,2]),
                    'boundary_mode':a.boundary_mode,'boundary_metadata':bound-
ary_info,
                    'N':a.grids[-1],'grid':'xi=(1-exp(-4*i/N))/(1-exp(-4))','rtol':2e-10,'at-
ol':2e-12,
                    'model_checks':checks,'grid_checks':rec-
ords,'time_check':timecheck,'comparisons':comparisons,
                    'event_s':r['event_s'],'event_h':r['event_s']/3600,'finish_h':r['fin-
ish_h'],'finish_s':r['finish_s'],
                    'final_max_C':r['final_max_C'],'final_R_cm':float(r['R'][-1]*100),'fi-
nal_mean_C':float(r['mean_C'][-1]),
                    'max_radial_increase':r['max_radial_increase'],'radius_extrapol-
ation_used':bool(r['finish_s']>radius[-1,0]),
                    'table_t_s':r['t'][table_i].tolist(),'table_R_cm':(r['R'][:ta-
ble_i]*100).tolist(),'table_C':table_C.tolist(),

```

```

        'output_rows':len(r['t'])-1,'outside_blank_count':int(np.isnan(r['C'][1:]).sum()))
    q3path=PACKAGE+'/results'/q3'/artifact_name('q3', 'validation')
    if q3path.exists():
        q3s=json.loads(q3path.read_text(encoding='utf-8'))
        summary['q3_finish_h']=q3s['finish_h']
        q3_n800=next((v['event_s']/3600 for v in q3s['grid_checks'] if
v['N']==800),None)
        summary['q3_event_h_N800']=q3_n800
        (a.out / artifact_name('q4', 'validation')).write_text(
            json.dumps(summary,ensure_ascii=False,indent=2),encoding='utf-8')
        payload={'fixed_r_cm':np.round(OUT-
PUT_R*100,1).tolist(),'t_s':np.round(r['t'][1:],4).tolist(),
                'R_cm':(r['R'][1:]*100).tolist(),
                'C':[[None if np.isnan(v) else round(float(v),4) for v in row] for row in
r['C'][1:]]}
        (a.out / artifact_name('q4', 'result_data')).write_text(
            json.dumps(payload,ensure_ascii=False,allow_nan=False),encoding='utf-8')
        np.savez_compressed(a.out / artifact_name('q4', 'full_precision'), t=r['t'], T=r['T'],
C=r['C'], R=r['R'], mean_C=r['mean_C'],
                        profile_x=r['profile_x'],profiles_C=r['profiles_C'],table_in-
dices=table_i)
        graph(r,a.out)
        print(json.dumps(summary,ensure_ascii=False,indent=2),flush=True)

if __name__=='__main__':
    with threadpool_limits(limits=1):main()

```